

ĐẠI HỌC QUỐC GIA THÀNH PHỐ HỒ CHÍ MINH
TRƯỜNG ĐẠI HỌC BÁCH KHOA THÀNH PHỐ HỒ CHÍ MINH

KHOA ĐIỆN - ĐIỆN TỬ
BỘ MÔN VIỄN THÔNG



ĐỒ ÁN MÔN HỌC

MULTICAST TRONG MẠNG 6LoWPAN

GVHD: TS. Võ Quế Sơn

SVTH: Dương Hồng Sơn

MSSV: 2312953

Thành phố Hồ Chí Minh, tháng 1 năm 2026

LỜI CẢM ƠN

Trước hết, em xin bày tỏ lòng biết ơn đến thầy Võ Quế Sơn – người đã hướng dẫn em trong suốt quá trình thực hiện đề tài. Trong thời gian làm đồ án, thầy không những hỗ trợ em về mặt chuyên môn mà còn cho em nhiều lời khuyên quý báu, giúp em hoàn thiện nội dung một cách tốt nhất. Bên cạnh đó, em cũng xin chân thành cảm ơn quý thầy cô trong khoa đã giảng dạy cho em những kiến thức bổ ích trong suốt thời gian học tập. Những kiến thức và kinh nghiệm mà thầy cô cung cấp chính là cơ sở quan trọng giúp em có đủ nền tảng để nghiên cứu và triển khai đề tài một cách hiệu quả. Mặc dù em đã cố gắng hết sức để hoàn thành đồ án 1 một cách tốt nhất nhưng do kiến thức và kinh nghiệm thực tế còn hạn chế nên chắc chắn không thể tránh khỏi những thiếu sót. Em rất mong nhận được sự góp ý, nhận xét và chỉ bảo thêm từ quý thầy cô để em có thể rút kinh nghiệm, hoàn thiện bản thân và thực hiện tốt hơn trong những đề tài sau. Em xin chân thành cảm ơn!

TP. Hồ Chí Minh, ngày 5 tháng 1 năm 2026

Sinh viên



Dương Hồng Sơn

TÓM TẮT ĐỀ TÀI

IoT ngày càng phát triển mạnh mẽ trong các lĩnh vực như nhà thông minh, giám sát môi trường và nông nghiệp thông minh. Các hệ thống IoT thường sử dụng mạng không dây công suất thấp với tài nguyên hạn chế về năng lượng và băng thông. Công nghệ 6LoWPAN cho phép truyền thông IPv6 trên mạng cảm biến không dây, tạo nền tảng kết nối hiệu quả cho các thiết bị IoT. Trong môi trường 6LoWPAN, nhu cầu truyền dữ liệu đến nhiều node là khá phổ biến. Nếu sử dụng phương thức unicast để gửi dữ liệu lần lượt đến từng node, lưu lượng truyền trong mạng sẽ tăng lên, kéo theo số lượng gói tin lớn hơn và mức tiêu thụ năng lượng cao hơn. Vì vậy, multicast được xem là giải pháp phù hợp nhằm tối ưu tài nguyên mạng và nâng cao hiệu quả truyền dữ liệu. Đề tài tập trung khảo sát các giao thức cơ bản liên quan đến multicast trong mạng 6LoWPAN như IPv6 multicast, UDP, MLD và cơ chế hoạt động của lớp thích nghi 6LoWPAN. Đồng thời, đề tài nghiên cứu các thuật toán multicast thường được sử dụng trong mạng công suất thấp như Flooding, Tree-based multicast và multicast dựa trên RPL. Thông qua việc xây dựng mô hình mô phỏng trên nền tảng Contiki-NG kết hợp với công cụ Cooja Simulator, đề tài thực hiện so sánh multicast và unicast dựa trên các thông số như độ trễ truyền dữ liệu, tỉ lệ mất gói và số lượng gói tin trong mạng. Kết quả thu được giúp đánh giá hiệu quả của multicast trong môi trường mạng IoT công suất thấp.

MỤC LỤC

1	GIỚI THIỆU	1
1.1	Tổng quan đề tài	1
1.2	Nhiệm vụ đề tài	1
1.3	Phạm vi và phương pháp nghiên cứu	2
1.3.1	Phạm vi nghiên cứu	2
1.3.2	Phương pháp nghiên cứu	2
1.4	Cấu trúc báo cáo	3
2	CƠ SỞ LÝ THUYẾT	4
2.1	Tổng quan về Internet of Things (IoT)	4
2.1.1	Ứng dụng của IoT trong thực tế	4
2.1.2	Kiến trúc cơ bản của hệ thống IoT	5
2.1.3	Đặc điểm của thiết bị IoT trong mạng công suất thấp	5
2.1.4	Yêu cầu về truyền thông trong mạng IoT	5
2.1.5	Liên hệ với đề tài nghiên cứu	5
2.2	Mạng LLN (Low-Power and Lossy Networks)	6
2.2.1	Đặc điểm của mạng LLN	6
2.2.2	Cấu trúc và mô hình truyền thông trong mạng LLN	7
2.2.3	Giao thức định tuyến RPL trong LLNs	7
2.2.4	Chuẩn IEEE 802.15.4 trong mạng LLN	8
2.2.5	Thách thức khi triển khai multicast trong mạng LLN	8
2.3	Giao thức IPv6 trong mạng IoT	9
2.4	6LoWPAN	9
2.5	UDP	9
2.6	Multicast IPv6	10
2.7	So sánh Unicast và Multicast trong mạng 6LoWPAN	10
2.7.1	Unicast trong mạng 6LoWPAN	11
2.7.2	Multicast trong mạng 6LoWPAN	11
3	THUẬT TOÁN MULTICAST TRONG 6LoWPAN	12
3.1	Tổng quan multicast trong 6LoWPAN	12
3.2	Trickle Multicast	12
3.3	SMRF (Stateless Multicast RPL Forwarding)	13
3.4	EMRF (Enhanced Multicast RPL Forwarding)	13
3.5	So sánh các thuật toán	14

4	XÂY DỰNG MÔ HÌNH VÀ MÔ PHỎNG	15
4.1	Giới thiệu	15
4.2	Môi trường mô phỏng	15
4.2.1	Giới thiệu Contiki-NG	15
4.2.2	Giới thiệu Cooja Simulator	16
4.2.3	Lý do lựa chọn môi trường mô phỏng	16
4.2.4	Cấu hình mô phỏng sử dụng trong đề tài	17
4.3	Mô hình mạng 6LoWPAN được xây dựng	17
4.3.1	Kiến trúc tổng thể của mạng	17
4.3.2	Vai trò của các node trong mô hình	18
4.3.3	Topology mạng mô phỏng	18
4.3.4	Các thông số cấu hình của mô hình	19
4.4	Thiết kế các kịch bản mô phỏng	20
4.4.1	Mục tiêu thiết kế kịch bản mô phỏng	20
4.4.2	Kịch bản 1: Truyền Unicast	20
4.4.3	Kịch bản 2: Trickle Multicast	21
4.4.4	Kịch bản 3: SMRF	21
4.4.5	Kịch bản 4: EMRF	21
4.4.6	Nguyên tắc đảm bảo so sánh công bằng giữa các kịch bản	22
4.5	Quy trình thực hiện mô phỏng trên Cooja	22
4.5.1	Mục tiêu của quy trình mô phỏng	22
4.5.2	Tạo simulation mới	22
4.5.3	Tạo mote type và biên dịch chương trình cho node	22
4.5.4	Thêm node vào mô hình mạng	23
4.5.5	Bố trí topology và kiểm tra kết nối giữa các node	23
4.5.6	Thực hiện chạy mô phỏng và quan sát log	24
4.5.7	Thu thập dữ liệu phục vụ đánh giá	25
4.6	Các chỉ tiêu đánh giá và cách thu thập dữ liệu	25
4.6.1	Mục tiêu đánh giá	25
4.6.2	Độ trễ truyền dữ liệu (Delay)	26
4.6.3	Tỉ lệ nhận gói và tỉ lệ mất gói (Delivery ratio / Packet loss)	26
4.6.4	Overhead truyền thông	27
4.6.5	Tiêu thụ năng lượng (ước lượng)	28
4.6.6	Khả năng mở rộng theo số node	29
5	KẾT QUẢ MÔ PHỎNG VÀ THỰC NGHIỆM	30
5.1	Kết quả mô phỏng	30
5.1.1	Kết quả kịch bản Unicast	30
5.1.2	Kết quả kịch bản Trickle Multicast	33
5.1.3	Kết quả kịch bản SMRF	35
5.1.4	Kết quả kịch bản EMRF	37
5.2	So sánh kết quả	39
5.2.1	So sánh theo độ trễ truyền dữ liệu	39
5.2.2	So sánh theo tỉ lệ nhận gói và tỉ lệ mất gói	40
5.2.3	So sánh theo overhead truyền thông	40
5.3	Phân tích nguyên nhân	41
5.4	Kiểm chứng trên FIT/IoT-LAB	42
5.4.1	Mục đích kiểm chứng	42
5.4.2	Thiết lập kiểm chứng	43
5.4.3	Kiểm chứng kịch bản Unicast	43
5.4.4	Trickle Multicast	45

5.4.5	Kiểm chứng kịch bản SMRF	46
5.4.6	Kiểm chứng kịch bản EMRF	47
5.4.7	Nhận xét chung	48
5.5	Đánh giá chung	49
6	KẾT LUẬN VÀ HƯỚNG PHÁT TRIỂN	50
6.1	Kết luận	50
6.2	Hạn chế	50
6.3	Hướng phát triển	51
6.4	Kết thúc	54
	PHỤ LỤC	56

DANH MỤC ẢNH MINH HỌA

2.1	Cấu trúc cây định tuyến DODAG trong RPL	7
2.2	So sánh truyền Unicast và Multicast trong mạng 6LoWPAN	11
4.1	Giao diện mô phỏng mạng trên Cooja Simulator	16
4.2	Topology mạng 6LoWPAN được xây dựng trên Cooja	19
4.3	Tạo simulation mới trên Cooja	23
4.4	Tạo mote type và biên dịch chương trình cho node trên Cooja	23
4.5	Thêm node vào mô hình mạng trong Cooja	24
4.6	Topology mạng mô phỏng gồm 1 node gốc và các node nhận	24
4.7	Giao diện mô phỏng trên Cooja: cửa sổ Network và Mote output trong quá trình chạy	25
5.1	Topology mô phỏng kịch bản unicast trong Cooja	30
5.2	Log gửi request của các node client	31
5.3	Log nhận response của các node client	31
5.4	Kết quả mô phỏng kịch bản Trickle Multicast trong Cooja	33
5.5	Kết quả mô phỏng kịch bản SMRF trong Cooja	35
5.6	Kết quả mô phỏng kịch bản EMRF trong Cooja	37
5.7	Lựa chọn node server cho kịch bản unicast trên FIT/IoT-LAB	43
5.8	Lựa chọn các node client cho kịch bản unicast trên FIT/IoT-LAB	44
5.9	Terminal log của server và client trong quá trình kiểm chứng kịch bản unicast trên FIT/IoT-LAB	44
5.10	Cấu hình 7 node cho kịch bản Trickle Multicast trên FIT/IoT-LAB	45
5.11	Terminal log của node root, các node intermediate và các node sink trong quá trình kiểm chứng Trickle Multicast trên FIT/IoT-LAB	45
5.12	Cấu hình node root, intermediate và sink cho kịch bản SMRF trên FIT/IoT-LAB	47
5.13	Terminal log trong quá trình thử kiểm chứng kịch bản EMRF trên FIT/IoT-LAB	48
6.1	Cấu hình mô phỏng unicast với radio medium đặt TX ratio = 60% và RX ratio = 60%	51
6.2	Cấu hình mô phỏng Trickle Multicast trong điều kiện tăng loss với TX ratio = 60% và RX ratio = 60%	53

DANH MỤC BẢNG

3.1	So sánh Trickle Multicast, SMRF và EMRF	14
4.1	Vai trò của các node trong mô phỏng	18
4.2	Các thông số cấu hình mô phỏng	20
5.1	Kết quả thống kê kích bản unicast	32
5.2	Kết quả thống kê kích bản Trickle Multicast	34
5.3	Kết quả thống kê kích bản SMRF	36
5.4	Kết quả thống kê kích bản EMRF	38
5.5	So sánh kết quả giữa các kích bản truyền dữ liệu	39
5.6	Kết quả tổng hợp kích bản unicast trên FIT/IoT-LAB	44
5.7	Kết quả tổng hợp kích bản Trickle Multicast trên FIT/IoT-LAB	46
5.8	Kết quả tổng hợp kích bản SMRF trên FIT/IoT-LAB	47
6.1	Kết quả thử nghiệm mở rộng cho kích bản unicast khi tăng loss trong radio medium	52
6.2	Kết quả thử nghiệm mở rộng cho kích bản Trickle Multicast khi tăng loss trong radio medium	53

DANH MỤC CHỮ VIẾT TẮT

IoT	Internet of Things
IPv6	Internet Protocol version 6
6LoWPAN	IPv6 over Low-Power Wireless Personal Area Networks
LLN / LLNs	Low-power and Lossy Network(s)
RPL	Routing Protocol for Low-Power and Lossy Networks
DAG	Directed Acyclic Graph
DODAG	Destination-Oriented Directed Acyclic Graph
DIO	DODAG Information Object
DIS	DODAG Information Solicitation
DAO	Destination Advertisement Object
IEEE	Institute of Electrical and Electronics Engineers
IEEE 802.15.4	Chuẩn mạng không dây công suất thấp dùng trong 6LoWPAN
MAC	Media Access Control
UDP	User Datagram Protocol
TCP	Transmission Control Protocol
MLD	Multicast Listener Discovery
NAT	Network Address Translation
SLAAC	Stateless Address Autoconfiguration
TTL	Time To Live
ACK	Acknowledgement
TX	Transmission (phát/gửi)
RX	Reception (thu/nhận)
SMRF	Stateless Multicast RPL Forwarding
EMRF	Enhanced Multicast RPL Forwarding
RTT	Round-Trip Time

Chương 1

GIỚI THIỆU

1.1 Tổng quan đề tài

IoT hiện nay đang được ứng dụng rộng rãi trong nhiều lĩnh vực như nhà thông minh, giám sát môi trường và nông nghiệp thông minh. Các thiết bị trong những hệ thống này thường hoạt động trong môi trường mạng không dây công suất thấp và tài nguyên hạn chế. Vì vậy, công nghệ 6LoWPAN được sử dụng nhằm hỗ trợ truyền thông IPv6 trên mạng cảm biến không dây, giúp các thiết bị IoT có thể kết nối và giao tiếp hiệu quả.

Trong mạng 6LoWPAN, nhu cầu gửi dữ liệu đến nhiều node là khá phổ biến. Khi sử dụng phương thức unicast để truyền dữ liệu lần lượt đến từng node, lưu lượng trong mạng sẽ tăng lên, đồng thời làm tiêu tốn nhiều năng lượng hơn. Vì vậy, multicast được xem là một giải pháp phù hợp trong môi trường mạng công suất thấp. Từ đó, đề tài được thực hiện nhằm nghiên cứu cơ chế multicast trong mạng 6LoWPAN và đánh giá hiệu quả của phương thức này thông qua mô phỏng trên nền tảng Contiki-NG.

1.2 Nhiệm vụ đề tài

Nội dung 1: Nghiên cứu cơ sở lý thuyết về mạng 6LoWPAN và multicast.

- Mục tiêu: nắm vững kiến thức nền tảng về IoT, mạng công suất thấp và kiến trúc 6LoWPAN. Hiểu rõ cơ chế hoạt động của multicast trong mạng IPv6 và vai trò của multicast trong hệ thống IoT.
- Ý tưởng thực hiện: đọc các tài liệu liên quan đến IPv6, lớp thích nghi 6LoWPAN, giao thức UDP và cơ chế quản lý nhóm MLD. Phân tích sự khác nhau giữa unicast và multicast trong việc truyền dữ liệu đến nhiều node. Làm cơ sở lý thuyết cho việc nghiên cứu các thuật toán multicast ở phần sau.

Nội dung 2: Khảo sát và phân tích các thuật toán multicast.

- Mục tiêu: hiểu được nguyên lý hoạt động của các thuật toán multicast phổ biến trong mạng công suất thấp và đánh giá ưu, nhược điểm của từng phương pháp.
- Ý tưởng thực hiện: nghiên cứu các thuật toán như Flooding, Tree-based multicast, multicast dựa trên RPL. Phân tích cơ chế truyền gói tin của từng thuật toán, cách xây dựng đường truyền trong mạng và mức độ tiêu thụ tài nguyên. So sánh các thuật toán dựa trên các tiêu chí như độ trễ, số lượng gói tin và độ tin cậy.

Nội dung 3: Xây dựng và mô phỏng mạng 6LoWPAN trên nền tảng Contiki-NG.

- Mục tiêu: triển khai mô hình mạng 6LoWPAN trong môi trường mô phỏng nhằm kiểm tra hoạt động thực tế của multicast và unicast.

- Ý tưởng thực hiện: cài đặt Contiki-NG và Cooja Simulator. Thiết kế mô hình gồm một node gốc (border router) và nhiều node cảm biến. Thực hiện các kịch bản truyền dữ liệu sử dụng unicast và multicast. Quan sát và ghi nhận quá trình truyền dữ liệu giữa các node.

Nội dung 4: Thu thập, phân tích và đánh giá kết quả.

- Mục tiêu: đánh giá hiệu quả của multicast so với unicast trong mạng 6LoWPAN dựa trên số liệu mô phỏng.
- Ý tưởng thực hiện: thu thập các thông số như độ trễ truyền dữ liệu, tỉ lệ mất gói và số lượng gói tin trong mạng. So sánh kết quả giữa các phương pháp truyền. Đưa ra nhận xét, đánh giá hiệu quả và đề xuất hướng phát triển cho đề tài.

1.3 Phạm vi và phương pháp nghiên cứu

1.3.1 Phạm vi nghiên cứu

Đề tài tập trung nghiên cứu cơ chế multicast trong mạng 6LoWPAN và ứng dụng của multicast trong môi trường mạng IoT công suất thấp. Nội dung nghiên cứu bao gồm các giao thức liên quan như IPv6 multicast, UDP, MLD và lớp thích nghi 6LoWPAN. Đồng thời, đề tài khảo sát một số thuật toán multicast phổ biến trong mạng công suất thấp như Flooding, Tree-based multicast và multicast dựa trên RPL.

Phần thực nghiệm của đề tài được thực hiện thông qua mô phỏng trên nền tảng Contiki-NG và công cụ Cooja Simulator. Mô hình mạng được xây dựng gồm một node gốc (border router) và nhiều node cảm biến nhằm đánh giá quá trình truyền dữ liệu giữa các node. Đề tài không triển khai hệ thống trên phần cứng thực tế mà chỉ giới hạn trong môi trường mô phỏng. Việc đánh giá hiệu quả multicast được thực hiện dựa trên các thông số thu thập được từ quá trình mô phỏng như độ trễ truyền dữ liệu, tỉ lệ mất gói và số lượng gói tin trong mạng.

1.3.2 Phương pháp nghiên cứu

- Phương pháp nghiên cứu tài liệu: thu thập và nghiên cứu các tài liệu, giáo trình, bài báo khoa học liên quan đến IoT, 6LoWPAN, IPv6 multicast và các thuật toán multicast trong mạng công suất thấp.
- Phương pháp phân tích và tổng hợp: phân tích nguyên lý hoạt động của các giao thức và thuật toán multicast. So sánh, tổng hợp các đặc điểm, ưu điểm và hạn chế của từng phương pháp.
- Phương pháp mô phỏng thực nghiệm: xây dựng mô hình mạng trên nền tảng Contiki-NG và thực hiện các kịch bản truyền dữ liệu sử dụng unicast và multicast. Thu thập dữ liệu từ quá trình mô phỏng để đánh giá hiệu quả.
- Phương pháp đánh giá và so sánh: dựa trên các thông số đo được như độ trễ, tỉ lệ mất gói và số lượng gói tin để so sánh hiệu quả giữa multicast và unicast, từ đó đưa ra nhận xét và kết luận.

1.4 Cấu trúc báo cáo

Chương 1: Giới thiệu

Trình bày tổng quan về đề tài, lý do chọn đề tài, mục tiêu nghiên cứu, nhiệm vụ thực hiện, phạm vi và phương pháp nghiên cứu. Đồng thời giới thiệu cấu trúc tổng thể của báo cáo.

Chương 2: Cơ sở lý thuyết

Trình bày các kiến thức nền tảng liên quan đến đề tài như Internet of Things (IoT), mạng công suất thấp, giao thức IPv6, lớp thích nghi 6LoWPAN, giao thức UDP và cơ chế multicast trong mạng IP. Chương này làm cơ sở cho việc nghiên cứu các thuật toán multicast ở các chương sau.

Chương 3: Các thuật toán multicast trong mạng 6LoWPAN

Phân tích nguyên lý hoạt động của các thuật toán multicast phổ biến trong mạng công suất thấp như Flooding, Tree-based multicast và multicast dựa trên RPL. So sánh ưu điểm, hạn chế và khả năng ứng dụng của từng phương pháp.

Chương 4: Xây dựng mô hình và mô phỏng trên nền tảng Contiki-NG

Giới thiệu nền tảng Contiki-NG và công cụ Cooja Simulator. Trình bày cách xây dựng mô hình mạng 6LoWPAN, thiết kế các kịch bản truyền dữ liệu sử dụng unicast và multicast, cũng như các thông số cấu hình trong quá trình mô phỏng.

Chương 5: Kết quả mô phỏng và đánh giá

Trình bày các kết quả thu thập được từ mô phỏng, bao gồm độ trễ truyền dữ liệu, tỉ lệ mất gói và số lượng gói tin trong mạng. Thực hiện phân tích và so sánh hiệu quả giữa multicast và unicast, từ đó rút ra nhận xét và đánh giá.

Chương 6: Kết luận và hướng phát triển

Tổng kết các nội dung đã thực hiện trong đề tài, nêu những kết quả đạt được, hạn chế còn tồn tại và đề xuất hướng phát triển trong tương lai.

Chương 2

CƠ SỞ LÝ THUYẾT

2.1 Tổng quan về Internet of Things (IoT)

Trong những năm gần đây, IoT đã trở thành một trong những xu hướng công nghệ phát triển mạnh mẽ trên toàn thế giới. IoT được hiểu là mô hình kết nối các thiết bị vật lý với Internet, cho phép các thiết bị này có thể thu thập, trao đổi và xử lý dữ liệu một cách tự động mà không cần sự can thiệp trực tiếp của con người. Nhờ vào sự phát triển của mạng Internet, vi xử lý công suất thấp và công nghệ truyền thông không dây, IoT ngày càng được ứng dụng rộng rãi trong nhiều lĩnh vực khác nhau của đời sống.

Các thiết bị trong hệ thống IoT có thể là cảm biến nhiệt độ, độ ẩm, ánh sáng, thiết bị đo chất lượng không khí, camera giám sát, thiết bị điều khiển đèn, quạt, máy bơm nước, ... Những thiết bị này được gắn địa chỉ mạng và có khả năng giao tiếp với nhau thông qua các giao thức truyền thông phù hợp. Dữ liệu thu thập được từ các cảm biến sẽ được truyền về máy chủ hoặc nền tảng đám mây để xử lý, phân tích và hiển thị cho người dùng.

Trong phạm vi đề tài này, IoT được xem xét dưới góc độ mạng cảm biến không dây công suất thấp, nơi các thiết bị giao tiếp thông qua IPv6 trên nền 6LoWPAN. Việc nghiên cứu multicast trong môi trường này giúp đánh giá khả năng truyền dữ liệu nhóm hiệu quả cho các ứng dụng IoT thực tế.

2.1.1 Ứng dụng của IoT trong thực tế

IoT hiện nay được ứng dụng trong rất nhiều lĩnh vực, có thể kể đến như [1]:

- Nhà thông minh (Smart Home): Các thiết bị như đèn, điều hòa, camera, cảm biến chuyển động được kết nối với nhau và có thể điều khiển từ xa thông qua điện thoại.
- Nông nghiệp thông minh (Smart Agriculture): Cảm biến được sử dụng để đo độ ẩm đất, nhiệt độ môi trường, từ đó tự động điều khiển hệ thống tưới tiêu.
- Giám sát môi trường: Hệ thống cảm biến được triển khai để theo dõi chất lượng không khí, mức độ ô nhiễm hoặc cảnh báo cháy rừng.
- Thành phố thông minh (Smart City): Ứng dụng trong quản lý giao thông, chiếu sáng công cộng, quản lý bãi đỗ xe.
- Công nghiệp 4.0: Các thiết bị và máy móc trong nhà máy được kết nối để giám sát và tối ưu hóa quy trình sản xuất. [1]

Từ các ứng dụng trên có thể thấy rằng IoT giúp nâng cao hiệu quả quản lý, tiết kiệm chi phí và tăng mức độ tự động hóa trong nhiều lĩnh vực.

2.1.2 Kiến trúc cơ bản của hệ thống IoT

Một hệ thống IoT thông thường gồm ba lớp chính [1]:

1. *Lớp cảm biến (Perception Layer)*: Đây là lớp thấp nhất của hệ thống IoT, bao gồm các thiết bị cảm biến và thiết bị chấp hành (actuator). Nhiệm vụ chính của lớp này là thu thập dữ liệu từ môi trường như nhiệt độ, độ ẩm, ánh sáng, áp suất,... Sau đó chuyển dữ liệu này đến lớp mạng để xử lý tiếp.

2. *Lớp mạng (Network Layer)*: Lớp này có nhiệm vụ truyền dữ liệu từ các thiết bị cảm biến đến máy chủ hoặc hệ thống xử lý trung tâm. Tùy theo quy mô và yêu cầu của hệ thống, lớp mạng có thể sử dụng các công nghệ như Wi-Fi, Ethernet, 4G/5G hoặc các mạng không dây công suất thấp như IEEE 802.15.4. Trong môi trường mạng công suất thấp, 6LoWPAN được sử dụng để hỗ trợ truyền IPv6 trên nền IEEE 802.15.4. [6], [7]

3. *Lớp ứng dụng (Application Layer)*: Đây là lớp cao nhất, nơi dữ liệu được xử lý, phân tích và hiển thị cho người dùng. Lớp này có thể là một website, ứng dụng di động hoặc hệ thống quản lý trên nền tảng đám mây.

Kiến trúc phân lớp này giúp hệ thống IoT hoạt động linh hoạt và dễ mở rộng.

2.1.3 Đặc điểm của thiết bị IoT trong mạng công suất thấp

Khác với các thiết bị mạng truyền thống như máy tính hoặc server, các node trong mạng IoT thường có cấu hình phần cứng hạn chế. Cụ thể:

- Bộ nhớ RAM và Flash nhỏ.
- Tốc độ xử lý thấp.
- Hoạt động bằng pin nên yêu cầu tiết kiệm năng lượng.
- Băng thông truyền dẫn thấp. [2],[5]

Do những hạn chế này, việc sử dụng các giao thức mạng truyền thống cần được điều chỉnh hoặc tối ưu lại để phù hợp. Ví dụ, IPv6 được sử dụng thay vì IPv4 vì khả năng mở rộng địa chỉ lớn hơn và hỗ trợ multicast tốt hơn. Tuy nhiên, do kích thước header IPv6 khá lớn, nên cần có lớp thích nghi 6LoWPAN để nén header và giảm kích thước gói tin. [6], [7]

2.1.4 Yêu cầu về truyền thông trong mạng IoT

Trong hệ thống IoT, việc truyền dữ liệu cần đảm bảo các yêu cầu sau:

- Tiết kiệm năng lượng: vì phần lớn thiết bị sử dụng pin.
- Độ trễ thấp: đặc biệt trong các hệ thống cảnh báo.
- Giảm lưu lượng mạng: tránh gây tắc nghẽn.
- Khả năng mở rộng: hệ thống có thể thêm nhiều node mà vẫn hoạt động ổn định. [3], [5]

Một trong những vấn đề quan trọng là khi cần gửi dữ liệu hoặc lệnh đến nhiều node cùng lúc. Nếu sử dụng unicast để gửi riêng lẻ đến từng node, số lượng gói tin sẽ tăng lên đáng kể khi số node lớn. Điều này làm tăng tiêu thụ năng lượng và lưu lượng mạng. Vì vậy, multicast được xem là một giải pháp phù hợp trong môi trường IoT công suất thấp. Multicast cho phép gửi một gói tin đến nhiều node thông qua một địa chỉ nhóm, giúp tối ưu tài nguyên mạng. [3], [13]

2.1.5 Liên hệ với đề tài nghiên cứu

Trong đề tài “Multicast trong mạng 6LoWPAN”, việc nghiên cứu IoT đóng vai trò nền tảng. 6LoWPAN được thiết kế nhằm hỗ trợ IPv6 trong mạng công suất thấp – vốn là môi trường phổ

biến của IoT. Do đó, việc hiểu rõ đặc điểm của IoT giúp làm rõ lý do tại sao multicast lại quan trọng và cần được nghiên cứu trong mạng 6LoWPAN.

Từ những phân tích trên có thể thấy rằng IoT không chỉ là xu hướng công nghệ mà còn đặt ra nhiều yêu cầu đặc biệt về giao thức truyền thông. Điều này chính là cơ sở để nghiên cứu và đánh giá hiệu quả của multicast trong môi trường mạng công suất thấp.

2.2 Mạng LLN (Low-Power and Lossy Networks)

Trong các hệ thống Internet of Things (IoT), phần lớn các thiết bị đầu cuối là cảm biến hoặc node có cấu hình thấp và hoạt động bằng pin. Những thiết bị này thường được triển khai với số lượng lớn và phân bố trong phạm vi rộng. Môi trường mạng mà các thiết bị này hoạt động được gọi là mạng công suất thấp và không ổn định, hay còn gọi là LLN (Low-Power and Lossy Network). LLN là loại mạng có các đặc điểm về phần cứng hạn chế và môi trường truyền dẫn không đáng tin cậy. Đây là môi trường mạng đặc trưng của 6LoWPAN và cũng là nền tảng chính của đề tài nghiên cứu. [4], [5]

Với các đặc điểm như tài nguyên hạn chế và chất lượng liên kết không ổn định, mạng LLNs đặt ra nhiều thách thức cho việc triển khai multicast. Do đó, các thuật toán multicast trong mạng 6LoWPAN cần được thiết kế nhằm giảm số lượng gói tin, tiết kiệm năng lượng và đảm bảo khả năng mở rộng. [5], [13]

2.2.1 Đặc điểm của mạng LLN

Mạng LLN có những đặc điểm nổi bật sau:

1. Công suất tiêu thụ thấp

Các node trong mạng LLN thường sử dụng nguồn pin và được thiết kế để hoạt động trong thời gian dài (vài tháng đến vài năm). Vì vậy, việc tiết kiệm năng lượng là yêu cầu quan trọng hàng đầu. Mọi giao thức hoạt động trong mạng LLN đều phải tối ưu hóa để giảm số lượng gói tin truyền và giảm thời gian truyền.

2. Băng thông hạn chế

So với mạng Ethernet hoặc Wi-Fi truyền thống, mạng LLN có tốc độ truyền dữ liệu thấp hơn nhiều. Ví dụ, chuẩn IEEE 802.15.4 chỉ hỗ trợ tốc độ tối đa khoảng 250 kbps. Điều này khiến việc truyền dữ liệu lớn hoặc truyền nhiều gói tin liên tục có thể gây tắc nghẽn mạng. [5], [6]

3. Tỷ lệ mất gói cao (Lossy)

Do sử dụng sóng vô tuyến công suất thấp, tín hiệu trong mạng LLN dễ bị ảnh hưởng bởi:

- Nhiễu sóng
- Vật cản
- Khoảng cách truyền
- Điều kiện môi trường

Vì vậy, hiện tượng mất gói tin xảy ra thường xuyên hơn so với mạng có dây hoặc Wi-Fi.

4. Tài nguyên phần cứng hạn chế

Các node trong mạng LLN thường có:

- RAM chỉ vài KB
- Bộ nhớ Flash vài chục KB
- CPU công suất thấp

Điều này khiến việc triển khai các giao thức phức tạp như TCP gặp nhiều khó khăn. [2], [3], [5]

2.2.2 Cấu trúc và mô hình truyền thông trong mạng LLN

Trong mạng LLN, thường có một node đóng vai trò root (nút gốc), còn các node còn lại kết nối với nhau theo cấu trúc phân cấp để hình thành mạng định tuyến nhiều chặng. Trong môi trường 6LoWPAN, cấu trúc này thường được quản lý bởi giao thức RPL [3], [4]. Các node sẽ chọn parent để gửi dữ liệu lên root. Việc truyền dữ liệu trong mạng LLN có thể diễn ra theo hai hướng:

- Upward traffic: từ node cảm biến lên root
- Downward traffic: từ root gửi xuống các node

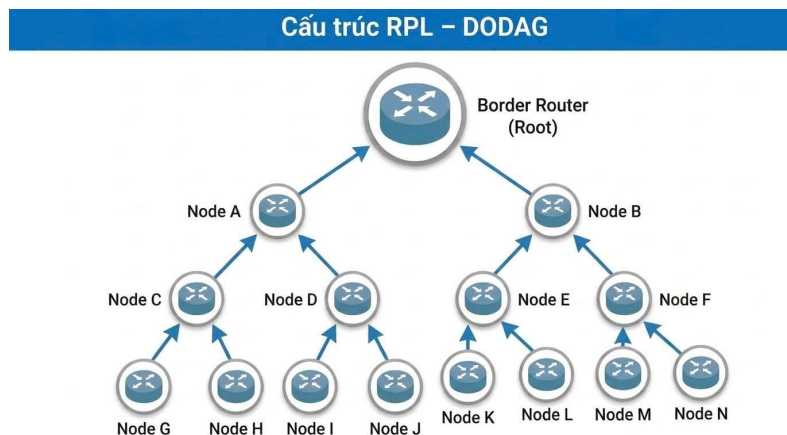
Multicast thường thuộc nhóm traffic hướng xuống (downward traffic), khi root cần gửi dữ liệu đến nhiều node cùng lúc. [3]

2.2.3 Giao thức định tuyến RPL trong LLNs

Trong mạng LLNs, do đặc điểm băng thông thấp, tài nguyên hạn chế và tỉ lệ mất gói cao, các giao thức định tuyến truyền thống như OSPF hay RIP không phù hợp. Vì vậy, IETF đã đề xuất giao thức RPL (Routing Protocol for Low-Power and Lossy Networks) dành riêng cho môi trường mạng công suất thấp. RPL được thiết kế để hoạt động hiệu quả trong các mạng IoT sử dụng IPv6 và 6LoWPAN. [3]

1. Cấu trúc DODAG

RPL xây dựng cấu trúc mạng theo dạng cây có hướng không chu trình gọi là DODAG (Destination-Oriented Directed Acyclic Graph). Trong cấu trúc này một node đóng vai trò Root (thường là Border Router). Các node khác sẽ chọn Parent để kết nối lên Root. Mỗi node có một giá trị gọi là Rank, thể hiện vị trí của node trong cây. Rank càng nhỏ thì node càng gần Root. Cấu trúc DODAG giúp tổ chức mạng rõ ràng, hạn chế vòng lặp, tối ưu đường đi trong mạng công suất thấp. [3]



Hình 2.1: Cấu trúc cây định tuyến DODAG trong RPL

2. Các loại bản tin trong RPL

RPL sử dụng các bản tin điều khiển sau:

- DIO (DODAG Information Object): dùng để quảng bá thông tin về cấu trúc mạng và giá trị Rank.
- DIS (DODAG Information Solicitation): node mới gửi yêu cầu để nhận thông tin mạng.
- DAO (Destination Advertisement Object): dùng để thông báo đường đi ngược từ node về Root.

Nhờ các bản tin này, mạng có thể tự tổ chức và tự cập nhật khi có node tham gia hoặc rời khỏi mạng. [3]

3. Vai trò của RPL trong multicast

Trong môi trường LLNs, nếu sử dụng phương pháp flooding để truyền multicast, số lượng gói tin sẽ tăng nhanh và gây tiêu tốn năng lượng. RPL có thể hỗ trợ xây dựng cây phân phối dữ liệu (multicast tree), giúp:

- Giảm số lần truyền lại gói tin
- Hạn chế trùng lặp
- Tiết kiệm năng lượng
- Tăng hiệu quả truyền dữ liệu trong 6LoWPAN [3], [13]

Trong đề tài này, RPL đóng vai trò quan trọng vì cấu trúc DODAG của RPL có thể được tận dụng để xây dựng các thuật toán multicast hiệu quả, giúp giảm hiện tượng flooding và tiết kiệm năng lượng trong mạng 6LoWPAN.

2.2.4 Chuẩn IEEE 802.15.4 trong mạng LLN

IEEE 802.15.4 là chuẩn truyền thông vật lý và MAC được sử dụng phổ biến trong mạng LLN. Đây là nền tảng của các công nghệ như Zigbee, Thread và 6LoWPAN. Một số thông số quan trọng của IEEE 802.15.4:

- Tốc độ truyền tối đa: 250 kbps
- Kích thước frame tối đa: 127 byte
- Hoạt động ở băng tần 2.4 GHz [6]

Tuy nhiên, do kích thước frame chỉ 127 byte, sau khi trừ phần header MAC, dữ liệu còn lại để chứa IPv6 rất hạn chế. Trong khi đó, header IPv6 đã chiếm 40 byte. Điều này gây ra vấn đề lớn khi triển khai IPv6 trực tiếp trên IEEE 802.15.4. Chính vì vậy, 6LoWPAN được thiết kế để giải quyết vấn đề này bằng cách nén header IPv6, phân mảnh gói tin lớn và tối ưu hóa truyền dữ liệu. [6], [7]

2.2.5 Thách thức khi triển khai multicast trong mạng LLN

Việc triển khai multicast trong mạng LLN gặp nhiều thách thức do đặc điểm của mạng:

1. Nguy cơ gây bão gói tin (Broadcast Storm)

Nếu sử dụng phương pháp flooding đơn giản để gửi multicast, mỗi node sẽ truyền lại gói tin cho các node lân cận. Điều này có thể gây ra tình trạng lặp lại gói tin và làm tăng lưu lượng mạng đáng kể. [13]

2. Tiêu tốn năng lượng

Mỗi lần truyền gói tin đều tiêu thụ năng lượng. Nếu multicast không được tối ưu, số lượng truyền lại gói tin sẽ tăng và làm giảm tuổi thọ pin của thiết bị. [13]

3. Quản lý nhóm multicast

Trong mạng IPv6, việc quản lý thành viên nhóm multicast sử dụng giao thức MLD (Multicast Listener Discovery). Tuy nhiên, trong môi trường LLN, việc duy trì và cập nhật thông tin nhóm cần được tối ưu để tránh tạo thêm lưu lượng điều khiển. [12], [13]

* Vai trò của LLN trong đề tài

Mạng LLN đóng vai trò là môi trường hoạt động chính. Các thuật toán multicast được nghiên cứu và mô phỏng đều hoạt động trong môi trường:

- Băng thông thấp

- Tài nguyên hạn chế
- Tỷ lệ mất gói cao

Do đó, việc hiểu rõ đặc điểm của LLN giúp đánh giá chính xác hiệu quả của multicast so với unicast trong quá trình mô phỏng trên nền tảng Contiki-NG.

2.3 Giao thức IPv6 trong mạng IoT

Trong các hệ thống IoT dựa trên nền tảng IP, IPv6 được sử dụng làm giao thức mạng chính nhờ khả năng mở rộng địa chỉ và hỗ trợ multicast hiệu quả. Tuy nhiên, do đặc điểm công suất thấp của mạng cảm biến, IPv6 cần được tối ưu khi triển khai trong môi trường 6LoWPAN.

* Đặc điểm của IPv6

IPv6 có độ dài địa chỉ 128 bit, cho phép cung cấp một số lượng địa chỉ rất lớn. Điều này giúp mỗi thiết bị IoT có thể được cấp một địa chỉ riêng biệt mà không lo cạn kiệt. [8], [9] Một số đặc điểm nổi bật của IPv6 là không cần NAT, hỗ trợ tốt multicast, header đơn giản hơn IPv4, tự động cấu hình địa chỉ (Stateless Address Autoconfiguration – SLAAC). Đặc biệt, IPv6 loại bỏ broadcast và thay thế bằng multicast, điều này rất phù hợp với môi trường IoT. [9], [10]

* Cấu trúc header IPv6

Header IPv6 có độ dài cố định 40 byte và gồm các trường:

- Version (4 bit): phiên bản giao thức (luôn là 6).
- Traffic Class (8 bit): phân loại lưu lượng.
- Flow Label (20 bit): định danh luồng dữ liệu.
- Payload Length (16 bit): độ dài phần dữ liệu.
- Next Header (8 bit): xác định giao thức lớp trên (UDP, TCP, ...).
- Hop Limit (8 bit): số bước nhảy tối đa.
- Source Address (128 bit).
- Destination Address (128 bit).

So với IPv4, IPv6 không có trường checksum trong header, giúp giảm xử lý và tăng hiệu suất. [8]

* Cơ chế địa chỉ trong IPv6

IPv6 hỗ trợ ba loại địa chỉ chính là unicast, multicast, anycast. IPv6 không hỗ trợ broadcast. Điều này giúp giảm lưu lượng không cần thiết trong mạng. Trong đề tài này, multicast IPv6 là nội dung trọng tâm vì nó cho phép gửi dữ liệu đến một nhóm node cùng lúc thay vì gửi riêng lẻ từng node.

2.4 6LoWPAN

6LoWPAN (IPv6 over Low-Power Wireless Personal Area Networks) là lớp thích nghi cho phép truyền IPv6 trên nền mạng IEEE 802.15.4. Vì khung dữ liệu của IEEE 802.15.4 chỉ tối đa 127 byte, trong khi header IPv6 đã 40 byte, nên cần cơ chế nén. Chức năng của 6LoWPAN: nén header IPv6, phân mảnh và ghép mảnh gói tin, tối ưu hóa truyền dữ liệu [6], [7]. Nhờ 6LoWPAN, các thiết bị công suất thấp vẫn có thể sử dụng IPv6 hiệu quả.

2.5 UDP

UDP (User Datagram Protocol) là giao thức tầng vận chuyển không kết nối. [11]

* *Đặc điểm:*

- Không thiết lập kết nối
- Không đảm bảo truyền tin
- Header nhỏ (8 byte)
- Độ trễ thấp

Cấu trúc header gồm: Source Port, Destination Port, Length, Checksum [11]. Trong IoT, UDP phù hợp hơn TCP vì tiết kiệm được năng lượng, giảm overhead, phù hợp mạng công suất thấp. Trong mạng 6LoWPAN, UDP thường được ưu tiên sử dụng thay cho TCP do có header nhỏ và không yêu cầu thiết lập kết nối. Điều này giúp giảm overhead và phù hợp với các ứng dụng multicast trong môi trường mạng công suất thấp.

2.6 Multicast IPv6

Multicast là phương thức truyền dữ liệu từ một node nguồn đến một nhóm các node đích thông qua một địa chỉ nhóm. Thay vì gửi nhiều gói unicast riêng lẻ, multicast chỉ cần gửi một gói duy nhất đến địa chỉ nhóm. Điều này giúp giảm số lượng gói tin, giảm tiêu thụ năng lượng và tăng hiệu quả sử dụng băng thông.

* *Cấu trúc địa chỉ Multicast IPv6*

- Địa chỉ multicast IPv6 có tiền tố FF00::/8
- Cấu trúc gồm:
 - 8 bit đầu: 11111111 (FF)
 - 4 bit: Flags
 - 4 bit: Scope
 - 112 bit: Group ID
- Scope xác định phạm vi hoạt động:
 - 1: Node-local
 - 2: Link-local
 - 5: Site-local
 - E: Global [9]

* *Giao thức MLD*

MLD (Multicast Listener Discovery) là giao thức được sử dụng trong IPv6 để quản lý thành viên của nhóm multicast. Chức năng chính của MLD: thông báo node tham gia hay rời nhóm multicast, router cập nhật danh sách thành viên [12]. Trong môi trường 6LoWPAN, MLD cần được tối ưu để giảm lưu lượng điều khiển. Giao thức MLD giúp các node trong mạng 6LoWPAN quản lý việc tham gia hoặc rời khỏi nhóm multicast. Thông tin này là cơ sở để các thuật toán multicast quyết định node nào cần chuyển tiếp gói tin, từ đó ảnh hưởng trực tiếp đến hiệu quả truyền dữ liệu và mức tiêu thụ năng lượng.

2.7 So sánh Unicast và Multicast trong mạng 6LoWPAN

Trong mạng 6LoWPAN nói riêng và mạng IoT công suất thấp nói chung, việc truyền dữ liệu từ một node đến nhiều node khác là nhu cầu thường xuyên xảy ra, ví dụ như gửi lệnh điều khiển bật/tắt nhiều thiết bị cùng lúc hoặc gửi thông báo cấu hình đến toàn bộ mạng. Hai phương thức truyền dữ liệu phổ biến là unicast và multicast. Mỗi phương thức có nguyên lý hoạt động, ưu điểm và hạn chế riêng.

2.7.1 Unicast trong mạng 6LoWPAN

Unicast là phương thức truyền dữ liệu từ một node nguồn đến một node đích duy nhất. Mỗi gói tin sẽ chứa địa chỉ IPv6 cụ thể của node nhận. Trong trường hợp cần gửi dữ liệu đến nhiều node, node nguồn phải gửi nhiều gói tin riêng biệt đến từng node đích. Ví dụ, nếu cần gửi dữ liệu đến 5 node, node nguồn phải thực hiện 5 lần truyền riêng biệt. [3]

* *Ưu điểm:*

- Đảm bảo dữ liệu đến đúng một node cụ thể
- Kiểm soát truyền dẫn tốt hơn
- Dễ triển khai và cấu hình

* *Hạn chế:*

- Tăng số lượng gói tin trong mạng
- Tiêu tốn nhiều năng lượng (đặc biệt trong mạng cảm biến)
- Tăng nguy cơ tắc nghẽn khi số node lớn

Trong mạng LLNs, nơi các node thường hoạt động bằng pin và có tài nguyên hạn chế, việc gửi nhiều gói unicast liên tiếp sẽ làm giảm thời gian sống của mạng.

2.7.2 Multicast trong mạng 6LoWPAN

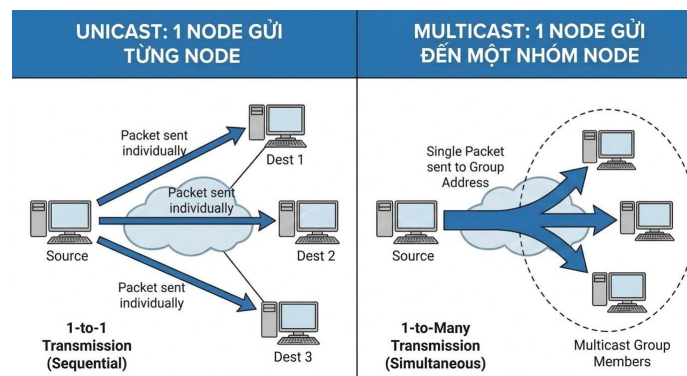
Multicast là phương thức truyền dữ liệu từ một node nguồn đến một nhóm node cùng tham gia một địa chỉ multicast. Thay vì gửi nhiều gói riêng lẻ như unicast, node nguồn chỉ cần gửi một gói tin đến địa chỉ multicast, và các node thuộc nhóm sẽ nhận được dữ liệu. Địa chỉ multicast IPv6 có tiền tố FF00::/8. Các node muốn nhận dữ liệu sẽ đăng ký tham gia nhóm multicast thông qua cơ chế MLD.

* *Ưu điểm:*

- Giảm số lượng gói tin truyền trong mạng
- Tiết kiệm băng thông
- Giảm tiêu thụ năng lượng
- Phù hợp cho truyền dữ liệu đồng thời đến nhiều node

* *Hạn chế:*

- Không có cơ chế xác nhận (ACK) như unicast
- Khó kiểm soát việc mất gói
- Phức tạp hơn trong việc quản lý nhóm



Hình 2.2: So sánh truyền Unicast và Multicast trong mạng 6LoWPAN

Chương 3

THUẬT TOÁN MULTICAST TRONG 6LoWPAN

3.1 Tổng quan multicast trong 6LoWPAN

Trong mạng 6LoWPAN/LLNs, multicast được dùng khi cần gửi dữ liệu đến một nhóm node (ví dụ gửi lệnh điều khiển theo nhóm, thông báo cấu hình, cảnh báo). Mục tiêu của multicast là giảm số gói tin so với unicast gửi lần lượt từng node, từ đó tiết kiệm băng thông và năng lượng. Tuy nhiên, do đặc thù LLNs có liên kết không ổn định, băng thông thấp và node hạn chế tài nguyên, multicast phải được thiết kế sao cho không phát tán tràn lan gây nghẽn và hao pin [5], [13]. Các thuật toán multicast trong Contiki/Contiki-NG thường tập trung giải quyết 3 vấn đề: (1) Giảm số lần phát lại (overhead), (2) đảm bảo độ phủ (delivery ratio), và (3) hoạt động được trên cấu trúc RPL/DODAG [3], [14].

3.2 Trickle Multicast

* Ý tưởng

Trickle Multicast dựa trên Trickle algorithm – một cơ chế “phát lại có kiểm soát” bằng cách tự suppress (không phát lại) khi mạng đã có đủ mức dư thừa. Thay vì “ai nhận cũng phát lại” như flooding, Trickle Multicast cho node chờ một khoảng thời gian ngẫu nhiên, trong lúc đó node lắng nghe xem có bao nhiêu hàng xóm đã phát lại cùng gói. Nếu nghe thấy đủ nhiều, node sẽ không cần phát lại nữa. [13]

* Nguyên lý hoạt động

Một node khi nhận được gói multicast mới sẽ:

1. Bắt đầu một interval có độ dài I (thường thay đổi theo thời gian).
2. Chọn ngẫu nhiên một thời điểm $t \in [I/2, I]$ để “có thể phát lại”.
3. Trong khoảng chờ, node đếm số lần nghe thấy gói giống nhau (counter c).
4. Khi đến thời điểm t :
 - Nếu $c < k$ (k là redundancy constant) $\rightarrow$ node phát lại.
 - Nếu $c \geq k \rightarrow$ node suppress, không phát lại.

Cơ chế này giúp hạn chế broadcast storm vì trong một khu vực chỉ cần một số node phát lại là đủ phủ. [13]

* Ưu điểm:

- Giảm trùng lặp mạnh ở mạng mật độ cao.
- Tự thích nghi theo mức dư thừa của mạng.

- Không cần xây dựng cây multicast riêng.

* *Nhược điểm:*

- Có thể tăng độ trễ vì phải chờ đến thời điểm phát lại.
- Trong môi trường mất gói cao, cơ chế “nghe đủ rồi” đôi khi làm giảm độ phủ nếu các gói nghe bị mất.

⇒ Trickle Multicast phù hợp để giảm overhead trong mạng dày đặc, chấp nhận độ trễ tăng nhẹ để đổi lấy tiết kiệm năng lượng.

3.3 SMRF (Stateless Multicast RPL Forwarding)

* *Ý tưởng*

SMRF là cơ chế forward multicast dựa trên RPL/DODAG và mang tính stateless (ít/không duy trì trạng thái multicast phức tạp). Thay vì flooding, SMRF chỉ forward multicast theo quan hệ parent-child trong DODAG nhằm giới hạn gói tin đi đúng “nhánh” cần thiết. Nói đơn giản là SMRF dùng cây RPL có sẵn để chuyển multicast xuống các nhánh, tránh phát tán toàn mạng. [3], [14]

* *Nguyên lý hoạt động*

Trong DODAG, mỗi node có: parent (cha) hướng lên root, children (con) hướng xuống Quy tắc forward cơ bản của SMRF:

- Nếu node nhận multicast từ parent → node forward xuống các child.
- Nếu nhận từ hướng “không đúng quan hệ” (ví dụ từ một node không phải parent theo logic DODAG) → node có thể bỏ qua để tránh loop và giảm trùng.

SMRF nhờ vậy mà giảm số lần forward so với flooding, vì chỉ các node nằm trên nhánh liên quan mới forward. [14]

* *Ưu điểm:*

- Overhead thấp hơn flooding/trickle trong nhiều trường hợp.
- Không yêu cầu nhiều bộ nhớ lưu trạng thái multicast.
- Tận dụng trực tiếp cấu trúc RPL nên phù hợp 6LoWPAN.

* *Nhược điểm:*

- Phụ thuộc vào độ ổn định của DODAG.
- Nếu nhánh có link yếu hoặc thay đổi, có thể làm giảm delivery ratio.
- Truyền multicast xuống (downward) thường khó hơn upward trong LLNs.

⇒ SMRF phù hợp khi mạng đã chạy RPL ổn định và cần cơ chế multicast nhẹ, tối ưu overhead.

3.4 EMRF (Enhanced Multicast RPL Forwarding)

* *Ý tưởng*

EMRF (thường tương ứng với ESMRF trong Contiki-NG) là phiên bản “nâng cao” dựa trên SMRF, nhằm cải thiện hạn chế của SMRF trong một số tình huống, đặc biệt liên quan đến việc gửi multicast và độ phủ trong môi trường lossy. Mục tiêu của EMRF là giữ tinh thần forward theo DODAG nhưng linh hoạt hơn, giúp tăng delivery ratio và mở rộng khả năng gửi multicast trong mạng. [14]

* Nguyên lý hoạt động

EMRF vẫn dựa trên cấu trúc parent-child của RPL để forward multicast, nhưng bổ sung logic để:

- Tránh bỏ sót node do forward quá cứng nhắc.
- Hỗ trợ tốt hơn khi multicast được phát sinh từ các node không phải root (tùy cấu hình/triển khai).
- Cân bằng giữa overhead và độ tin cậy: chấp nhận tăng nhẹ số lần forward để tăng độ phủ.

* Ưu điểm:

- Delivery ratio thường cao hơn SMRF trong môi trường lossy.
- Linh hoạt hơn trong việc phân phối multicast theo DODAG.
- Phù hợp nghiên cứu so sánh hiệu quả các hướng multicast dựa trên RPL.

* Nhược điểm:

- Phức tạp hơn SMRF, có thể tăng overhead so với SMRF.
- Phụ thuộc cấu hình RPL và chất lượng DODAG.

⇒ EMRF phù hợp khi ưu tiên độ phủ/độ tin cậy multicast cao hơn, chấp nhận thêm một phần overhead để đổi lấy kết quả ổn định hơn.

3.5 So sánh các thuật toán

Vì trong phạm vi mô phỏng nên có thể so sánh theo các tiêu chí:

- Độ trễ (Delay)
- Tỷ lệ nhận gói (Delivery ratio)
- Overhead (tổng số lần TX / số gói phát sinh trong mạng)
- Tiêu thụ năng lượng (ước lượng qua TX/RX hoặc công cụ đo năng lượng nếu có)
- Khả năng mở rộng theo số node

Bảng 3.1: So sánh Trickle Multicast, SMRF và EMRF

Tiêu chí	Trickle Multicast	SMRF	EMRF
Ý tưởng chính	Suppress dựa trên mức dư thừa	Forward theo DODAG, stateless	Nâng cao SMRF để tăng độ phủ
Overhead	Thấp-TB	Thấp	TB
Delivery ratio	TB (phụ thuộc loss)	TB	Cao hơn
Độ trễ	Có thể cao hơn	Thấp-TB	TB
Độ phức tạp	TB	Thấp	Cao hơn

* Nhận xét:

- Trickle Multicast phù hợp giảm trùng lặp, nhưng có thể tăng độ trễ do cơ chế interval.
- SMRF nhẹ và overhead thấp, phù hợp mạng RPL ổn định, nhưng delivery ratio phụ thuộc DODAG và link.
- EMRF cải thiện tính linh hoạt/độ phủ so với SMRF, thích hợp khi môi trường lossy và cần kết quả ổn định hơn.

Chương 4

XÂY DỰNG MÔ HÌNH VÀ MÔ PHỎNG

4.1 Giới thiệu

Sau khi tìm hiểu các kiến thức cơ bản về mạng 6LoWPAN, IPv6 multicast và một số thuật toán multicast ở các chương trước, trong chương này sẽ tiến hành xây dựng mô hình và mô phỏng mạng trên phần mềm Contiki-NG và Cooja. Đây là phần quan trọng để kiểm tra lại các nội dung lý thuyết đã trình bày, đồng thời quan sát cách các node hoạt động trong môi trường mạng mô phỏng.

Nội dung chính của chương này là giới thiệu môi trường mô phỏng được sử dụng, cách xây dựng mô hình mạng 6LoWPAN và thiết kế các kịch bản truyền dữ liệu giữa các node. Trong đó, mô hình mạng sẽ gồm một node gốc đóng vai trò phát dữ liệu và nhiều node còn lại đóng vai trò node nhận. Từ mô hình này, đề tài sẽ thực hiện các trường hợp truyền dữ liệu khác nhau để phục vụ cho việc so sánh và đánh giá hiệu quả của multicast trong mạng 6LoWPAN. Bên cạnh đó, chương này cũng trình bày các thông số được sử dụng trong quá trình mô phỏng như số lượng node, thời gian mô phỏng, chu kỳ gửi gói tin và cách thu thập kết quả. Những nội dung này sẽ là cơ sở để sang chương tiếp theo thực hiện phân tích kết quả mô phỏng dựa trên các tiêu chí như độ trễ, tỉ lệ mất gói và số lượng gói tin trong mạng.

Thông qua chương này, đề tài làm rõ quá trình triển khai từ phần lý thuyết sang phần mô phỏng thực tế, từ đó tạo nền tảng cho việc đánh giá và rút ra nhận xét ở các chương sau.

4.2 Môi trường mô phỏng

4.2.1 Giới thiệu Contiki-NG

Contiki-NG là một hệ điều hành mã nguồn mở được phát triển cho các thiết bị IoT và mạng cảm biến không dây công suất thấp. Hệ điều hành này hỗ trợ tốt cho các giao thức mạng thường dùng trong lĩnh vực IoT như IPv6, 6LoWPAN, RPL và UDP. Nhờ đó, Contiki-NG rất phù hợp để nghiên cứu và mô phỏng các mô hình mạng công suất thấp, đặc biệt là các bài toán truyền dữ liệu trong mạng 6LoWPAN.

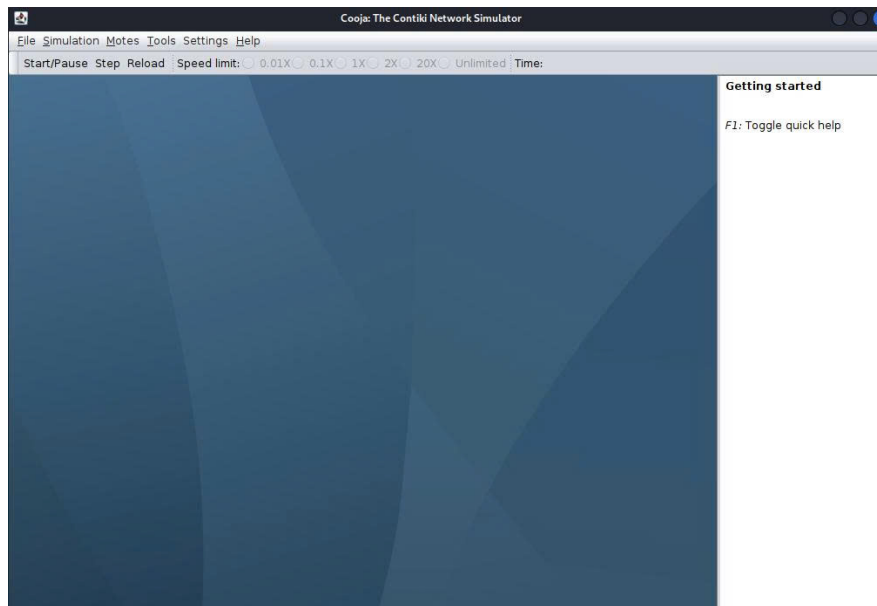
Trong đề tài này, việc lựa chọn Contiki-NG giúp quá trình xây dựng mô hình trở nên thuận tiện hơn vì hệ điều hành này đã tích hợp sẵn nhiều thành phần liên quan đến mạng IPv6 và các giao thức định tuyến cho IoT. Bên cạnh đó, Contiki-NG còn cho phép người dùng xây dựng các chương trình chạy trên từng node, từ đó dễ dàng tạo ra các kịch bản truyền dữ liệu unicast và multicast để kiểm tra hoạt động của mạng. Ngoài ra, Contiki-NG còn hỗ trợ tốt cho việc kiểm tra hành vi của các node trong quá trình truyền thông. Thông qua các chương trình được nạp vào node mô phỏng, người thực hiện có thể quan sát quá trình gửi, nhận và chuyển tiếp gói tin giữa các node trong mạng. Điều này rất phù hợp với mục tiêu của đề tài là đánh giá hoạt động

của multicast trong môi trường mạng 6LoWPAN. Trong phạm vi đề án này, Contiki-NG được sử dụng làm nền tảng chính để xây dựng mô hình mô phỏng, triển khai các node trong mạng và thực hiện các kịch bản truyền dữ liệu phục vụ cho quá trình đo đạc, so sánh và đánh giá kết quả.

4.2.2 Giới thiệu Cooja Simulator

Cooja Simulator là công cụ mô phỏng đi kèm với Contiki, được sử dụng rộng rãi trong việc nghiên cứu và kiểm thử các giao thức mạng dành cho IoT. Cooja cho phép tạo ra nhiều node ảo trong cùng một môi trường mô phỏng, từ đó người dùng có thể xây dựng các mô hình mạng cảm biến không dây với số lượng node khác nhau và quan sát quá trình hoạt động của mạng theo thời gian thực. Một ưu điểm lớn của Cooja là cho phép theo dõi chi tiết hoạt động của từng node trong quá trình mô phỏng. Người dùng có thể xem log của từng node, theo dõi quá trình gửi và nhận gói tin, quan sát topology mạng và kiểm tra cách các node liên lạc với nhau thông qua môi trường truyền vô tuyến [15]. Nhờ đó, Cooja rất phù hợp để mô phỏng mạng 6LoWPAN và đánh giá các phương thức truyền dữ liệu như unicast và multicast.

Trong đề tài này, Cooja được sử dụng để xây dựng topology mạng gồm một node gốc và nhiều node cảm biến. Sau khi nạp chương trình vào từng node, quá trình mô phỏng sẽ được thực hiện trên Cooja nhằm quan sát cách các gói tin được truyền trong mạng. Các kết quả thu được từ log mô phỏng sẽ là cơ sở để phân tích các thông số như độ trễ, tỉ lệ mất gói và số lượng gói tin phát sinh trong hệ thống. Có thể thấy rằng Cooja đóng vai trò là công cụ hỗ trợ rất quan trọng trong phần thực nghiệm của đề tài. Nhờ sử dụng Cooja, đề tài có thể kiểm tra hoạt động của mô hình mạng trong môi trường mô phỏng mà không cần triển khai trực tiếp trên phần cứng thật, từ đó tiết kiệm thời gian và giúp quá trình thử nghiệm trở nên thuận tiện hơn.



Hình 4.1: Giao diện mô phỏng mạng trên Cooja Simulator

4.2.3 Lý do lựa chọn môi trường mô phỏng

Trong đề tài này, em lựa chọn Contiki-NG kết hợp với Cooja Simulator để thực hiện mô phỏng do đây là bộ công cụ phù hợp với hướng nghiên cứu về mạng 6LoWPAN trong môi trường IoT công suất thấp. So với việc triển khai trực tiếp trên phần cứng thật, mô phỏng giúp quá trình thử nghiệm trở nên đơn giản hơn, dễ theo dõi hơn và tiết kiệm thời gian hơn. Bên cạnh đó, Contiki-NG hỗ trợ tốt cho các giao thức cần thiết trong đề tài như IPv6, 6LoWPAN, UDP và RPL. Trong khi đó, Cooja cho phép tạo mô hình gồm nhiều node, theo dõi log và quan sát quá

trình truyền gói tin giữa các node một cách trực quan. Điều này rất có lợi cho việc kiểm tra hoạt động của multicast và unicast trong mạng mô phỏng.

Một lý do khác là môi trường Contiki-NG/Cooja phù hợp để thực hiện các kịch bản thử nghiệm khác nhau mà vẫn giữ nguyên mô hình mạng. Ví dụ, người thực hiện có thể sử dụng cùng một topology nhưng thay đổi phương thức truyền dữ liệu hoặc thay đổi số lượng node để đánh giá sự khác biệt về hiệu năng. Nhờ vậy, kết quả thu được có tính so sánh tốt hơn và dễ phân tích hơn trong các chương sau. Với các ưu điểm trên, Contiki-NG và Cooja được xem là lựa chọn phù hợp để triển khai phần thực nghiệm của đề tài, đặc biệt trong bối cảnh đề án chỉ tập trung vào mô phỏng mà chưa triển khai trên phần cứng thực tế. Điều này cũng phù hợp với phạm vi nghiên cứu mà đề tài đã đặt ra từ đầu.

4.2.4 Cấu hình mô phỏng sử dụng trong đề tài

Trong phần mô phỏng, đề tài sử dụng môi trường Contiki-NG kết hợp với Cooja để xây dựng mạng 6LoWPAN gồm một node gốc và nhiều node cảm biến. Node gốc đóng vai trò phát dữ liệu hoặc điều phối quá trình truyền thông trong mạng, trong khi các node còn lại đóng vai trò nhận dữ liệu và tham gia vào quá trình chuyển tiếp gói tin khi cần thiết. Cách xây dựng mô hình này phù hợp với định hướng ban đầu của đề tài là mô phỏng hoạt động của unicast và multicast trong mạng 6LoWPAN. Trong quá trình mô phỏng, các thông số cơ bản như số lượng node, thời gian mô phỏng, chu kỳ gửi gói tin và loại giao thức được giữ ở mức phù hợp để thuận tiện cho việc quan sát và thu thập kết quả. Việc lựa chọn các thông số này nhằm đảm bảo mô hình vừa đủ đơn giản để dễ triển khai, vừa đủ rõ ràng để phục vụ cho việc đánh giá hiệu quả truyền dữ liệu trong mạng.

Ngoài ra, do mục tiêu chính của đề tài là so sánh hiệu quả giữa unicast và multicast nên các kịch bản mô phỏng cần được xây dựng trên cùng một mô hình mạng. Điều này giúp giảm bớt sai lệch khi đánh giá kết quả, vì các yếu tố như topology, số node và thời gian mô phỏng đều được giữ tương đối giống nhau giữa các lần thử nghiệm. Các số liệu thu được từ môi trường mô phỏng sẽ được sử dụng trong chương sau để phân tích các chỉ tiêu như độ trễ truyền dữ liệu, tỉ lệ mất gói và số lượng gói tin trong mạng. Đây là những tiêu chí quan trọng để đánh giá hiệu quả hoạt động của multicast trong mạng 6LoWPAN.

4.3 Mô hình mạng 6LoWPAN được xây dựng

4.3.1 Kiến trúc tổng thể của mạng

Trong đề tài này, mô hình mạng 6LoWPAN được xây dựng trên môi trường mô phỏng Cooja với cấu trúc gồm một node gốc và nhiều node cảm biến. Node gốc đóng vai trò là điểm khởi tạo quá trình truyền dữ liệu trong mạng, trong khi các node còn lại đóng vai trò là node nhận hoặc node trung gian tham gia chuyển tiếp gói tin khi cần thiết. Mô hình này được lựa chọn vì phù hợp với đặc điểm của mạng IoT công suất thấp, nơi các thiết bị thường có tài nguyên hạn chế nhưng vẫn cần trao đổi dữ liệu với nhau trong cùng một hệ thống. Về mặt truyền thông, các node trong mạng sử dụng nền tảng IPv6 kết hợp với cơ chế nén tiêu đề của 6LoWPAN để truyền dữ liệu trên môi trường mạng cảm biến không dây. Bên cạnh đó, mô hình cũng có thể sử dụng RPL để tổ chức cấu trúc định tuyến giữa các node, từ đó hỗ trợ quá trình truyền dữ liệu từ node gốc đến các node nhận trong mạng. Việc xây dựng mô hình theo hướng này giúp đề tài có thể khảo sát được hoạt động của cả unicast và multicast trong cùng một môi trường mạng.

Mô hình tổng thể được thiết kế theo hướng đơn giản nhưng vẫn đảm bảo thể hiện được các thành phần cơ bản của mạng 6LoWPAN. Cụ thể, hệ thống gồm một node trung tâm làm nguồn phát dữ liệu và một nhóm node xung quanh làm đích nhận dữ liệu. Trong một số trường hợp, các node nằm xa node gốc có thể nhận dữ liệu thông qua một hoặc nhiều node trung gian. Điều

này giúp mô phỏng gần hơn với thực tế của mạng cảm biến không dây, nơi dữ liệu thường không chỉ truyền trực tiếp một chặng mà có thể phải đi qua nhiều hop trước khi đến đích. Thông qua mô hình trên, đề tài có thể xây dựng các kịch bản truyền dữ liệu khác nhau để quan sát cách các gói tin được lan truyền trong mạng. Đây là cơ sở để thực hiện việc so sánh giữa unicast và multicast, cũng như đánh giá đặc điểm hoạt động của các thuật toán multicast được trình bày ở chương trước.

4.3.2 Vai trò của các node trong mô hình

Trong mô hình mạng được xây dựng, các node được chia thành các vai trò khác nhau để phục vụ cho quá trình mô phỏng. Trước hết là node gốc, đây là node có vai trò quan trọng nhất trong hệ thống. Node này chịu trách nhiệm khởi tạo mạng, phát sinh dữ liệu và gửi gói tin đến các node còn lại trong mạng theo từng kịch bản mô phỏng đã đặt ra. Trong các trường hợp khảo sát unicast, node gốc sẽ gửi dữ liệu lần lượt đến từng node đích. Trong các trường hợp khảo sát multicast, node gốc sẽ gửi dữ liệu đến một nhóm node nhận thông qua cơ chế multicast.

Bên cạnh node gốc là các node cảm biến hoặc node nhận dữ liệu. Đây là các node nằm trong phạm vi mạng và có nhiệm vụ nhận dữ liệu được gửi từ node gốc. Tùy theo cách bố trí topology, một số node có thể chỉ đóng vai trò là node đích, trong khi một số node khác ngoài việc nhận dữ liệu còn có thể tham gia chuyển tiếp gói tin đến các node ở xa hơn. Việc tồn tại các node trung gian như vậy giúp mô hình phản ánh đúng hơn đặc điểm của mạng nhiều hop trong môi trường 6LoWPAN.

Ngoài ra, trong trường hợp sử dụng giao thức định tuyến RPL, mỗi node trong mạng còn tham gia vào quá trình hình thành cấu trúc định tuyến, từ đó xác định đường truyền dữ liệu phù hợp trong hệ thống. Điều này đặc biệt quan trọng đối với các thuật toán multicast dựa trên DODAG như SMRF hoặc EMRF, vì hiệu quả của việc chuyển tiếp gói tin phụ thuộc khá nhiều vào cấu trúc định tuyến của mạng.

Việc phân chia rõ vai trò của từng node giúp cho quá trình mô phỏng trở nên dễ quan sát hơn, đồng thời cũng thuận lợi cho việc phân tích kết quả sau này. Khi xảy ra khác biệt về độ trễ, tỉ lệ nhận gói hay số lượng gói tin, người thực hiện có thể dựa vào vai trò của từng node để giải thích nguyên nhân một cách hợp lý hơn.

Node	Vai trò
Node 1	Root / Sender
Node 2	Receiver / Forwarder
Node 3	Receiver / Forwarder
Node 4	Receiver
Node 5	Receiver
Node 6	Receiver
Node 7	Receiver

Bảng 4.1: Vai trò của các node trong mô phỏng

4.3.3 Topology mạng mô phỏng

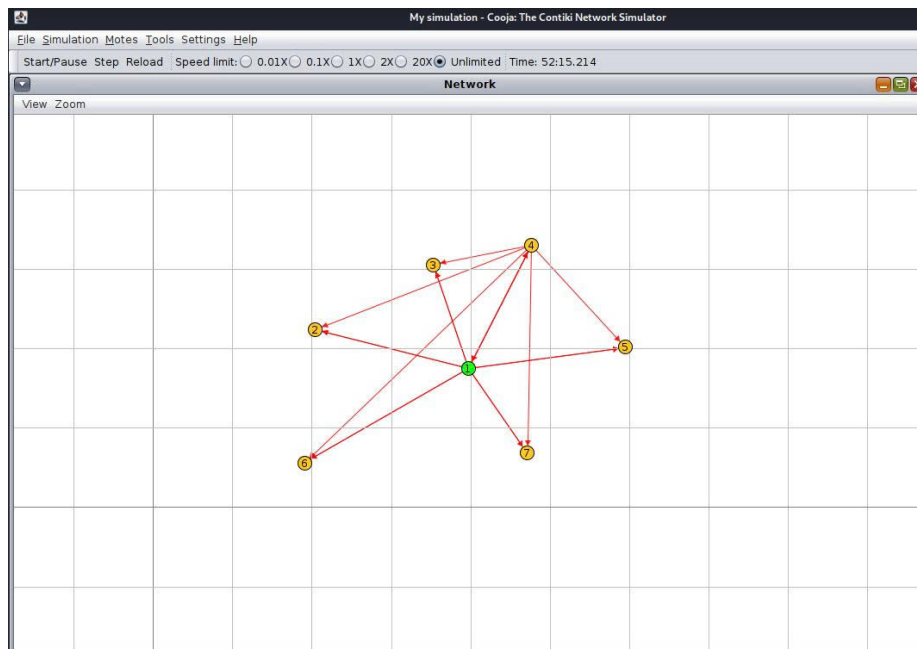
Topology của mạng trong đề tài được xây dựng theo dạng đơn giản, gồm một node gốc đặt ở vị trí trung tâm hoặc gần trung tâm và các node còn lại được bố trí xung quanh trong phạm vi truyền thông không dây. Cách bố trí này nhằm tạo điều kiện cho các node có thể liên lạc với

nhau trong môi trường mô phỏng, đồng thời cho phép quan sát được quá trình truyền dữ liệu từ node gốc đến các node nhận trong các trường hợp unicast và multicast.

Trong giai đoạn đầu của quá trình mô phỏng, topology được lựa chọn theo hướng tương đối đơn giản để dễ triển khai và theo dõi kết quả. Cụ thể, mô hình có thể sử dụng khoảng 7 node, trong đó gồm 1 node gốc và 6 node còn lại đóng vai trò là các node nhận. Với số lượng node này, đề tài vẫn có thể quan sát được hoạt động cơ bản của mạng 6LoWPAN mà không làm cho mô hình trở nên quá phức tạp.

Về vị trí đặt node, các node có thể được bố trí theo dạng cụm hoặc phân tán vừa phải trong không gian mô phỏng. Nếu khoảng cách giữa các node đủ gần, dữ liệu có thể được truyền trực tiếp từ node gốc đến node nhận. Ngược lại, nếu một số node được đặt xa hơn, quá trình truyền dữ liệu có thể phải đi qua node trung gian. Điều này giúp mô hình phản ánh được đặc điểm nhiều hop, vốn là một đặc trưng thường gặp trong mạng cảm biến không dây.

Việc lựa chọn topology ổn định và không quá phức tạp sẽ giúp quá trình đánh giá kết quả trở nên rõ ràng hơn. Khi tiến hành so sánh giữa unicast và multicast, đề tài sẽ giữ nguyên topology này để đảm bảo tính công bằng giữa các lần mô phỏng. Nhờ đó, sự khác biệt trong kết quả thu được chủ yếu đến từ cơ chế truyền dữ liệu, thay vì do thay đổi cấu trúc mạng.



Hình 4.2: Topology mạng 6LoWPAN được xây dựng trên Cooja

4.3.4 Các thông số cấu hình của mô hình

Để quá trình mô phỏng được thực hiện ổn định và thuận tiện cho việc so sánh kết quả, mô hình mạng trong đề tài được thiết lập với một số thông số cơ bản. Các thông số này bao gồm số lượng node trong mạng, thời gian mô phỏng, chu kỳ gửi gói tin, loại giao thức sử dụng và vai trò của từng node trong mô hình. Việc xác định rõ các thông số từ đầu sẽ giúp quá trình triển khai mô phỏng có tính nhất quán hơn giữa các kịch bản khác nhau.

Trong đề tài này, mô hình thử nghiệm ban đầu được xây dựng với một node gốc và một số node cảm biến xung quanh. Các node đều hoạt động trong môi trường mô phỏng Cooja trên nền Contiki-NG. Quá trình truyền dữ liệu giữa các node sử dụng nền tảng IPv6/6LoWPAN, kết hợp với giao thức định tuyến phù hợp để hỗ trợ liên lạc trong mạng nhiều hop khi cần thiết. Tùy theo từng kịch bản, dữ liệu sẽ được gửi theo cơ chế unicast hoặc multicast để quan sát sự khác biệt về hiệu năng.

Ngoài ra, để đảm bảo việc so sánh được công bằng, các thông số như số lượng node, thời gian chạy mô phỏng và chu kỳ gửi dữ liệu sẽ được giữ giống nhau giữa các kịch bản thử nghiệm. Chỉ có cơ chế truyền dữ liệu hoặc thuật toán multicast là được thay đổi để phục vụ mục đích đánh giá. Đây là nguyên tắc quan trọng giúp kết quả mô phỏng có độ tin cậy cao hơn và dễ phân tích hơn trong chương tiếp theo. Các thông số cấu hình cụ thể của mô hình sẽ được trình bày dưới dạng bảng để thuận tiện cho việc theo dõi và đối chiếu trong quá trình phân tích kết quả.

Thông số	Thiết lập sử dụng
Công cụ mô phỏng	Cooja Simulator
Nền tảng	Contiki-NG
Loại mote	Cooja mote
Node gốc (root)	1
Node trung gian / chuyển tiếp	2
Node nhận (sink/receiver)	4
Giao thức mạng	IPv6/6LoWPAN
Giao thức định tuyến	RPL
Giao thức truyền tải	UDP
Số lần gửi dữ liệu từ node gốc	100 lần
Chu kỳ gửi gói	1 s
Kịch bản truyền dữ liệu	Unicast, Trickle Multicast, SMRF, EMRF

Bảng 4.2: Các thông số cấu hình mô phỏng

4.4 Thiết kế các kịch bản mô phỏng

4.4.1 Mục tiêu thiết kế kịch bản mô phỏng

Trong phần mô phỏng, đề tài xây dựng các kịch bản truyền dữ liệu nhằm so sánh sự khác nhau giữa cơ chế unicast và multicast trong mạng 6LoWPAN. Các kịch bản được thiết kế theo hướng giữ nguyên mô hình mạng đã xây dựng ở mục trước, sau đó thay đổi phương thức truyền dữ liệu hoặc thuật toán multicast để đánh giá sự khác biệt về hiệu năng.

Mục tiêu chính của các kịch bản mô phỏng là quan sát và thu thập số liệu liên quan đến quá trình truyền dữ liệu trong mạng, bao gồm độ trễ truyền, tỉ lệ nhận gói (hoặc mất gói) và số lượng gói tin phát sinh trong hệ thống. Trên cơ sở đó, đề tài sẽ phân tích ưu điểm và hạn chế của từng phương án truyền dữ liệu trong môi trường mạng công suất thấp.

4.4.2 Kịch bản 1: Truyền Unicast

Trong kịch bản unicast, node gốc đóng vai trò là nguồn phát dữ liệu và thực hiện gửi gói tin đến từng node đích theo địa chỉ riêng biệt. Nghĩa là, nếu mạng có nhiều node nhận, node gốc sẽ phải thực hiện nhiều lần gửi khác nhau để mỗi node đều nhận được dữ liệu.

Mục đích của kịch bản unicast là tạo ra trường hợp tham chiếu (baseline) để so sánh với multicast. Trong mạng 6LoWPAN, unicast thường dễ triển khai và kiểm soát, tuy nhiên số lượng gói tin phát sinh sẽ tăng theo số lượng node nhận. Do đó, khi số node nhận tăng, cơ chế unicast có thể gây tăng overhead truyền thông và làm ảnh hưởng đến hiệu năng của mạng.

Trong mô hình mô phỏng, kịch bản unicast được thiết kế sao cho node gốc gửi dữ liệu theo chu kỳ cố định, lần lượt đến từng node còn lại. Mỗi lần gửi thành công, node nhận sẽ ghi nhận được dữ liệu ở phía đích. Các số liệu được thu thập ở kịch bản này bao gồm tổng số gói gửi từ node gốc, tổng số gói nhận tại các node và độ trễ truyền dữ liệu từ nguồn đến đích.

4.4.3 Kịch bản 2: Trickle Multicast

Trong kịch bản Trickle Multicast, node gốc gửi dữ liệu đến một nhóm node thông qua cơ chế multicast. Khác với unicast, multicast cho phép node gốc chỉ cần gửi một gói tin đến địa chỉ multicast (multicast group), và các node tham gia nhóm sẽ nhận được nội dung đó.

Điểm đặc trưng của Trickle Multicast là cơ chế “giảm trùng lặp” trong quá trình phát lại gói tin. Cụ thể, khi một node đã nghe thấy thông tin tương tự được truyền trong mạng đủ nhiều, node đó có thể hạn chế hoặc không phát lại gói tin nữa để giảm sự dư thừa. Cơ chế này giúp giảm overhead truyền thông trong mạng, tuy nhiên đôi lại có thể làm tăng độ trễ trong một số trường hợp do cơ chế hoạt động theo khoảng thời gian (interval) và có bước “suppress” phát lại.

Kịch bản này được thiết kế nhằm quan sát sự đánh đổi giữa việc giảm gói tin dư thừa và độ trễ truyền dữ liệu trong mạng. Trong mô hình mô phỏng, node gốc sẽ gửi gói multicast theo chu kỳ, các node tham gia nhóm sẽ nhận dữ liệu, đồng thời quá trình lan truyền gói tin có thể bị giảm bớt nhờ cơ chế suppress. Các số liệu cần thu thập gồm độ trễ trung bình, tỉ lệ nhận gói của các node trong nhóm và số lượng gói tin phát sinh trong toàn mạng.

4.4.4 Kịch bản 3: SMRF

SMRF là một cơ chế multicast hoạt động dựa trên cấu trúc định tuyến của RPL (DODAG). Ý tưởng chính của SMRF là sử dụng thông tin “cha-con” trong cây định tuyến để quyết định node nào sẽ chuyển tiếp gói multicast, từ đó giảm tình trạng phát lại tràn lan trong mạng. Đặc điểm của SMRF là “stateless”, tức là hạn chế việc lưu trạng thái phức tạp liên quan đến multicast ở mỗi node, giúp tiết kiệm tài nguyên và giảm overhead.

Trong kịch bản SMRF, node gốc sẽ gửi gói multicast đến nhóm node nhận. Khi gói tin lan truyền trong mạng, các node sẽ dựa trên hướng của DODAG để chuyển tiếp gói tin theo cách phù hợp. Nhờ đó, SMRF thường có overhead thấp và hoạt động khá hiệu quả trong các mạng có cấu trúc định tuyến ổn định. Tuy nhiên, hiệu năng có thể bị ảnh hưởng nếu cấu trúc DODAG thay đổi hoặc chất lượng liên kết không ổn định, dẫn đến giảm tỉ lệ nhận gói ở một số node.

Kịch bản SMRF được thiết kế để đánh giá hiệu năng multicast trong trường hợp forward dựa theo DODAG. Các chỉ tiêu cần thu thập gồm: độ trễ truyền dữ liệu, tỉ lệ nhận gói của các node trong nhóm và tổng số gói tin phát sinh trong mạng.

4.4.5 Kịch bản 4: EMRF

EMRF được xem là phiên bản cải tiến từ SMRF, nhằm tăng độ phủ và cải thiện tỉ lệ nhận gói trong môi trường truyền không ổn định. So với SMRF, cơ chế cải tiến thường cho phép quá trình chuyển tiếp linh hoạt hơn để hạn chế tình trạng một số node trong nhóm không nhận được dữ liệu do đường truyền kém hoặc topology thay đổi.

Trong kịch bản này, node gốc vẫn gửi dữ liệu theo cơ chế multicast đến nhóm node nhận. Tuy nhiên, cách forward gói tin trong mạng sẽ được điều chỉnh theo hướng “tăng độ tin cậy”, chấp nhận việc overhead có thể tăng nhẹ so với SMRF. Mục tiêu là đạt được delivery ratio tốt hơn, đặc biệt khi mạng có nhiều hop hoặc có tỉ lệ mất gói cao.

Kịch bản EMRF được thiết kế nhằm quan sát sự khác biệt giữa phương án “overhead thấp” (SMRF) và phương án “tăng độ tin cậy” (EMRF). Các số liệu thu thập tương tự các kịch bản

trước, gồm độ trễ, tỉ lệ nhận gói và số lượng gói tin phát sinh trong mạng.

4.4.6 Nguyên tắc đảm bảo so sánh công bằng giữa các kịch bản

Để việc so sánh giữa các kịch bản mang tính khách quan, đề tài áp dụng nguyên tắc giữ nguyên các điều kiện mô phỏng cơ bản, chỉ thay đổi phương thức truyền dữ liệu hoặc thuật toán multicast. Cụ thể, trong tất cả các kịch bản:

- Giữ nguyên topology mạng và vị trí các node.
- Giữ nguyên số lượng node tham gia mô phỏng.
- Giữ nguyên thời gian mô phỏng và chu kỳ gửi dữ liệu.
- Giữ nguyên kích thước gói tin (payload) và các thông số truyền thông cơ bản.
- Chỉ thay đổi cơ chế truyền: unicast hoặc multicast, và thay đổi thuật toán multicast (Trickle, SMRF, EMRF).

Việc giữ nguyên các thông số trên giúp đảm bảo rằng sự khác biệt trong kết quả thu được chủ yếu đến từ cơ chế truyền dữ liệu, từ đó làm rõ hiệu quả của multicast trong mạng 6LoWPAN cũng như sự khác nhau giữa các thuật toán multicast.

4.5 Quy trình thực hiện mô phỏng trên Cooja

4.5.1 Mục tiêu của quy trình mô phỏng

Trong đề tài này, phần mô phỏng được thực hiện nhằm kiểm tra và quan sát quá trình truyền dữ liệu trong mạng 6LoWPAN với các cơ chế truyền khác nhau. Do đó, quy trình mô phỏng cần đảm bảo hai yêu cầu chính: (1) mô hình mạng được xây dựng đúng theo topology đã thiết kế, và (2) các kịch bản mô phỏng được thực hiện nhất quán để kết quả thu được có thể so sánh được.

Quy trình mô phỏng trên Cooja được thực hiện theo các bước cơ bản gồm: tạo simulation mới, tạo loại node (mote type), thêm node vào mô hình, bố trí topology, chạy mô phỏng, quan sát log và thu thập dữ liệu. Các bước này giúp đảm bảo quá trình mô phỏng có tính lặp lại và dễ kiểm tra khi cần chạy lại để đối chiếu kết quả.

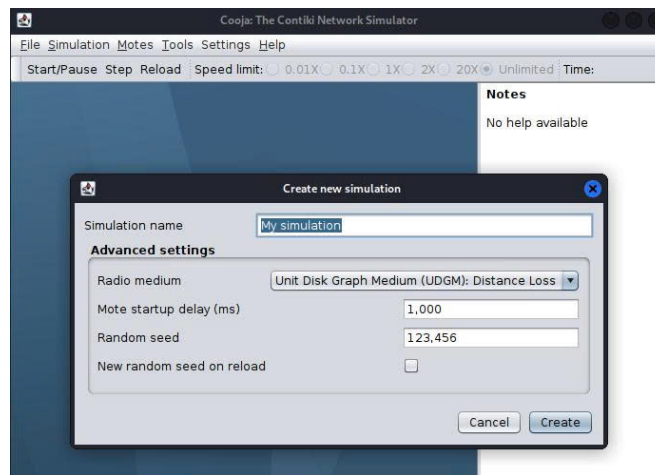
4.5.2 Tạo simulation mới

Bước đầu tiên của quá trình mô phỏng là tạo một simulation mới trên Cooja. Trong bước này, người thực hiện sẽ đặt tên cho simulation và lựa chọn môi trường truyền thông vô tuyến (radio medium) phù hợp. Việc lựa chọn radio medium ảnh hưởng đến cách các node giao tiếp với nhau trong mô phỏng (ví dụ dựa theo khoảng cách và tỉ lệ mất gói). Sau khi tạo simulation thành công, Cooja sẽ hiển thị môi trường làm việc để người thực hiện bắt đầu thêm các node vào mạng.

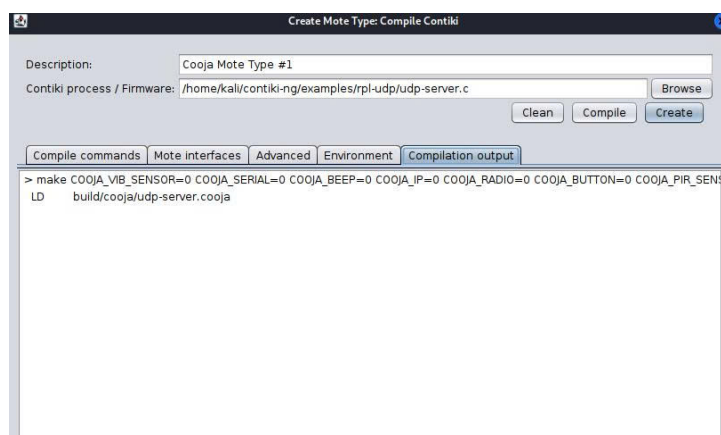
4.5.3 Tạo mote type và biên dịch chương trình cho node

Sau khi tạo simulation, bước tiếp theo là tạo mote type (loại node) để Cooja có thể nạp chương trình vào node mô phỏng. Mỗi mote type sẽ tương ứng với một chương trình chạy trên node (ví dụ node gốc hoặc node nhận). Trong đề tài này, mô hình gồm một node gốc và nhiều node còn lại, do đó thường sẽ cần tối thiểu hai mote type: (1) mote type cho node gốc và (2) mote type cho các node nhận.

Trong quá trình tạo mote type, người thực hiện chọn file chương trình tương ứng và tiến hành biên dịch (compile) trên Cooja. Nếu biên dịch thành công, Cooja sẽ cho phép tạo mote type và sử dụng mote type đó để thêm các node vào mô hình mạng. Việc tách mote type theo vai trò node giúp quá trình mô phỏng rõ ràng hơn, vì log của node gốc và node nhận thường có nội dung khác nhau, phục vụ trực tiếp cho việc quan sát và thu thập số liệu.



Hình 4.3: Tạo simulation mới trên Cooja



Hình 4.4: Tạo mote type và biên dịch chương trình cho node trên Cooja

4.5.4 Thêm node vào mô hình mạng

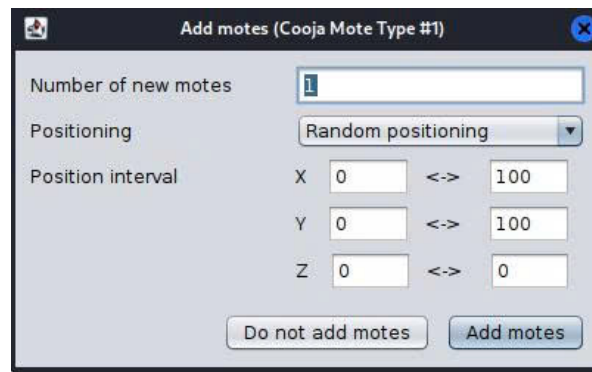
Sau khi tạo mote type, người thực hiện tiến hành thêm các node vào mô hình mạng. Trong đề tài này, mô hình tối thiểu gồm một node gốc và một số node nhận. Khi thêm node, Cooja cho phép lựa chọn số lượng node cần tạo và cách đặt vị trí node (ngẫu nhiên hoặc tự bố trí). Để thuận tiện cho quan sát topology và dễ chụp hình minh họa, các node thường được bố trí theo dạng phân tán quanh node gốc.

Việc lựa chọn số lượng node trong mô phỏng sẽ ảnh hưởng trực tiếp đến kết quả so sánh. Trong giai đoạn đầu, đề tài sử dụng số lượng node vừa phải để đảm bảo mô hình dễ triển khai và dễ theo dõi. Sau khi mô hình chạy ổn định, có thể tăng số lượng node ở các lần chạy sau để kiểm tra khả năng mở rộng và ảnh hưởng của mật độ node đến hiệu năng truyền multicast.

4.5.5 Bố trí topology và kiểm tra kết nối giữa các node

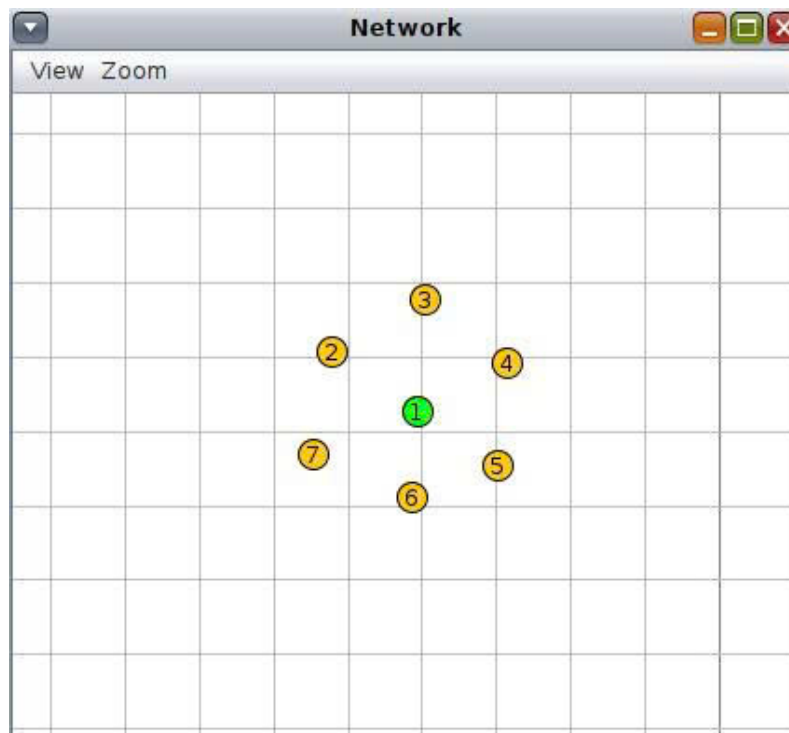
Sau khi các node đã được tạo, người thực hiện tiến hành bố trí lại vị trí node (nếu cần) để topology mạng hiển thị rõ ràng hơn. Cửa sổ Network trong Cooja cho phép quan sát sơ đồ các node, cũng như các liên kết vô tuyến giữa chúng. Việc quan sát topology giúp kiểm tra nhanh xem các node có nằm trong vùng liên lạc hay không, đồng thời giúp người thực hiện xác định được mạng đang có xu hướng một hop hay nhiều hop.

Trong đề tài này, topology được bố trí theo hướng đảm bảo các node đều có khả năng giao tiếp với node gốc hoặc thông qua node trung gian. Điều này giúp quá trình truyền dữ liệu unicast/multicast xảy ra ổn định và tạo điều kiện để quan sát rõ sự khác nhau về số lượng gói



Hình 4.5: Thêm node vào mô hình mạng trong Cooja

phát sinh cũng như độ trễ truyền trong mạng.

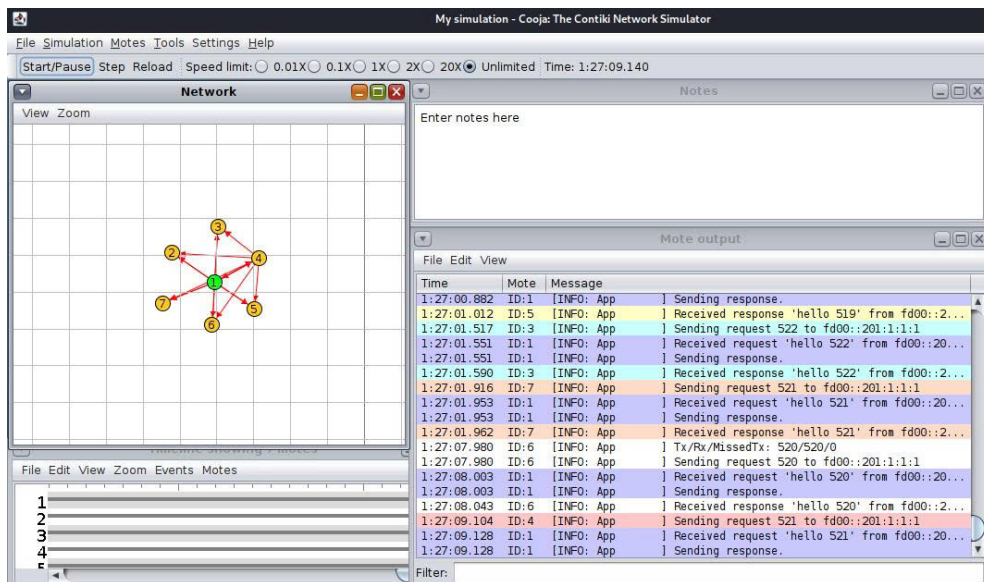


Hình 4.6: Topology mạng mô phỏng gồm 1 node gốc và các node nhận

4.5.6 Thực hiện chạy mô phỏng và quan sát log

Sau khi hoàn tất topology, người thực hiện bắt đầu chạy mô phỏng bằng cách khởi động simulation trên Cooja. Trong quá trình chạy, Cooja cho phép quan sát log của từng node thông qua cửa sổ Mote output hoặc log tổng. Log này thường hiển thị các thông tin như thời điểm node gửi gói, node nhận gói, địa chỉ nguồn/đích và các sự kiện liên quan đến truyền thông trong mạng.

Đối với đề tài này, việc quan sát log đóng vai trò quan trọng vì đây là nguồn dữ liệu để xác định số lần gửi, số lần nhận và (nếu có) tính toán độ trễ dựa trên thời điểm gửi/nhận. Tùy theo từng kịch bản, log sẽ thể hiện khác nhau: unicast thường thể hiện việc gửi lần lượt theo từng đích, trong khi multicast thể hiện một lần gửi nhưng nhiều node cùng nhận (hoặc nhận theo từng hop).



Hình 4.7: Giao diện mô phỏng trên Cooja: cửa sổ Network và Mote output trong quá trình chạy

4.5.7 Thu thập dữ liệu phục vụ đánh giá

Sau khi mô phỏng chạy trong khoảng thời gian đã thiết lập, người thực hiện tiến hành thu thập dữ liệu để phục vụ đánh giá. Dữ liệu có thể được thu từ log của các node (số lượng gói gửi/nhận, thời điểm gửi/nhận) hoặc từ các thống kê trong môi trường mô phỏng (nếu sử dụng các công cụ hỗ trợ thống kê của Cooja).

Trong đề tài, các dữ liệu cần thu thập chủ yếu gồm: (1) tổng số gói tin phát sinh trong mạng, (2) tỉ lệ nhận gói của các node (hoặc tỉ lệ mất gói), và (3) độ trễ truyền dữ liệu. Các dữ liệu này sẽ được tổng hợp thành bảng số liệu và biểu đồ ở chương tiếp theo để so sánh giữa các kịch bản unicast và multicast, cũng như giữa các thuật toán multicast khác nhau.

Để đảm bảo kết quả có tính so sánh, mỗi kịch bản mô phỏng nên được chạy với cùng một topology và cùng các thông số thời gian/chu kỳ gửi. Sau đó, chỉ thay đổi cơ chế truyền (unicast hoặc multicast) hay thuật toán multicast (Trickle, SMRF, EMRF) để đánh giá sự khác biệt về hiệu năng.

Tóm lại, quy trình mô phỏng trên Cooja trong đề tài được thực hiện theo các bước chính: tạo simulation, tạo mote type và biên dịch chương trình, thêm node vào mô hình, bố trí topology, chạy mô phỏng, quan sát log và thu thập dữ liệu. Các bước này đảm bảo việc mô phỏng có thể lặp lại và kết quả thu được có thể sử dụng trực tiếp để phân tích, so sánh ở chương tiếp theo.

4.6 Các chỉ tiêu đánh giá và cách thu thập dữ liệu

4.6.1 Mục tiêu đánh giá

Sau khi xây dựng mô hình và thiết kế các kịch bản mô phỏng, đề tài cần các chỉ tiêu cụ thể để so sánh hiệu năng giữa unicast và multicast, cũng như giữa các thuật toán multicast. Các chỉ tiêu được lựa chọn phải phản ánh được mức độ hiệu quả của việc truyền dữ liệu trong mạng 6LoWPAN, đồng thời phù hợp với dữ liệu có thể thu thập từ môi trường mô phỏng Cooja.

Trong đề tài này, các chỉ tiêu chính gồm: độ trễ truyền dữ liệu, tỉ lệ nhận gói/tỉ lệ mất gói, và overhead (số lượng gói tin phát sinh trong mạng). Ngoài ra, đề tài có thể xem xét thêm yếu tố khả năng mở rộng khi tăng số lượng node.

4.6.2 Độ trễ truyền dữ liệu (Delay)

Khái niệm:

Độ trễ truyền dữ liệu được hiểu là khoảng thời gian tính từ lúc node gốc gửi gói tin cho đến lúc node đích nhận được gói tin đó. Chỉ tiêu này phản ánh tốc độ truyền thông trong mạng.

Cách tính:

Với mỗi gói tin có số thứ tự (sequence number) xác định, độ trễ được tính:

$$Delay_i = t_{receive,i} - t_{send,i}$$

Trong đó:

- $t_{send,i}$: thời điểm node gốc gửi gói thứ i
- $t_{receive,i}$: thời điểm node nhận nhận gói thứ i

Độ trễ trung bình:

$$Delay_{avg} = \frac{1}{N} \sum_{i=1}^N Delay_i$$

Cách thu thập dữ liệu trong Cooja:

- Theo dõi log ở Mote output
- Ghi lại thời điểm node gốc in log “SEND ...” và thời điểm node nhận in log “RECV ...” tương ứng cùng số thứ tự gói.

4.6.3 Tỷ lệ nhận gói và tỷ lệ mất gói (Delivery ratio / Packet loss)

Khái niệm:

Tỷ lệ nhận gói phản ánh mức độ tin cậy: trong tổng số gói “đáng lẽ phải nhận”, các node nhận được bao nhiêu gói thành công. Tỷ lệ mất gói là phần còn lại bị thất bại do mất gói trên đường truyền hoặc do lỗi chuyển tiếp.

Cách tính:

Nếu mô phỏng có:

- tổng số gói gửi từ node gốc: P_{send}
- tổng số gói nhận thành công tại các node: P_{recv}

Tỷ lệ nhận gói:

$$Delivery\ Ratio = \frac{P_{recv}}{P_{expected}} \times 100\%$$

Trong đó $P_{expected}$ là số gói ‘kỳ vọng’:

- Với unicast: $P_{expected}$ thường bằng tổng số gói node gốc gửi đến từng node (tùy cách bạn thiết kế).
- Với multicast: nếu node gốc gửi K lần cho một nhóm gồm M node, thì

$$P_{expected} = K \times M.$$

Tỉ lệ mất gói:

$$Packet Loss Ratio = 100\% - Delivery Ratio$$

Cách thu thập dữ liệu trong Cooja:

- Dựa vào log “RECV ...” trên Mote output để đếm tổng gói nhận thành công.
- Dựa vào số lần “SEND ...” hoặc số gói dự kiến theo cấu hình (chu kỳ gửi $\times$ thời gian mô phỏng) để tính số gói kỳ vọng.

** Khả năng mở rộng:*

Khả năng mở rộng phản ánh khi số lượng node tăng lên thì hiệu năng của cơ chế truyền có bị giảm mạnh hay không (delay tăng, loss tăng, overhead tăng...).

** Cách đánh giá trong đề tài:*

- Giữ nguyên kịch bản và thông số, chỉ thay đổi số lượng node (ví dụ: 7 node $\rightarrow$ 10 node).
- So sánh các chỉ tiêu delay, delivery ratio và overhead giữa hai trường hợp.

$\Rightarrow$ Trong chương này, đề tài đã trình bày môi trường mô phỏng sử dụng Contiki-NG và Cooja, đồng thời xây dựng mô hình mạng 6LoWPAN gồm một node gốc và nhiều node còn lại để mô phỏng quá trình truyền dữ liệu trong mạng. Trên cơ sở mô hình đã xây dựng, đề tài đã thiết kế các kịch bản mô phỏng gồm unicast và các cơ chế multicast khác nhau để phục vụ cho việc so sánh và đánh giá.

Bên cạnh đó, chương này cũng đưa ra các chỉ tiêu đánh giá quan trọng như độ trễ truyền dữ liệu, tỉ lệ nhận gói/tỉ lệ mất gói và overhead truyền thông, kèm theo phương pháp thu thập dữ liệu từ môi trường mô phỏng. Đây là cơ sở để chương tiếp theo trình bày kết quả mô phỏng và phân tích sự khác nhau giữa các kịch bản, từ đó rút ra nhận xét về hiệu quả của multicast trong mạng 6LoWPAN.

4.6.4 Overhead truyền thông

Trong mạng 6LoWPAN công suất thấp, overhead truyền thông phản ánh mức độ “tốn tài nguyên truyền dẫn” của hệ thống để hoàn thành việc truyền dữ liệu. Overhead càng cao thì số lần truyền/nhận (TX/RX) càng nhiều, làm tăng xác suất va chạm gói, tăng độ trễ và làm tăng tiêu thụ năng lượng của các node. Vì vậy, overhead là một chỉ tiêu quan trọng để so sánh giữa unicast và multicast, cũng như giữa các thuật toán multicast khác nhau.

Trong phạm vi đề tài, do dữ liệu thu thập chủ yếu từ log mô phỏng trên Cooja, overhead được đánh giá theo hai mức:

- **Overhead tại node nguồn (root/nguồn phát):** số lần node nguồn phải phát gói ra mạng để phân phối dữ liệu.
- **Overhead ứng dụng/toàn hệ thống (ước lượng):** tổng số bản tin phát sinh liên quan trực tiếp đến kịch bản truyền dữ liệu (ví dụ request/response đối với unicast, hoặc số lần gửi của root và số lần chuyển tiếp của node trung gian đối với multicast nếu có log).

a) Overhead trong kịch bản Unicast

Với kịch bản unicast dạng UDP request/response, overhead có thể thống kê trực tiếp từ log:

- P_{send} : tổng số request do các node client gửi đi.
- P_{recv} : tổng số response nhận về.

Khi đó, tổng overhead ứng dụng được ước lượng:

$$Overhead_{app} = P_{send} + P_{recv}$$

Ngoài ra, nếu chỉ xét lượng gói phát sinh theo chiều gửi request:

$$Overhead_{send} = P_{send}$$

b) Overhead trong kịch bản Multicast

Đối với multicast, do một gói gửi có thể được nhiều node trong nhóm nhận, overhead tại node nguồn được tính bằng:

- K : số lần node root gửi gói multicast trong khoảng thời gian mô phỏng.

Do đó:

$$Overhead_{root} = K$$

Trong trường hợp có node trung gian (intermediate) tham gia chuyển tiếp, overhead toàn mạng có thể tăng do các lần forward. Khi log cho phép quan sát, overhead toàn mạng có thể ước lượng theo:

$$Overhead_{network} \approx K + K_{fwd}$$

trong đó K_{fwd} là số lần node trung gian chuyển tiếp gói multicast (nếu có thể thống kê từ log).

Như vậy, chỉ tiêu overhead giúp đề tài đánh giá mức độ “tốn gói/tốn lần truyền” giữa unicast và multicast trong cùng một topology, từ đó làm rõ ưu thế của multicast khi cần phân phối cùng một dữ liệu đến nhiều node trong mạng 6LoWPAN.

4.6.5 Tiêu thụ năng lượng (ước lượng)

Trong mạng IoT công suất thấp, tiêu thụ năng lượng là yếu tố quan trọng vì các node thường hoạt động bằng pin và có tài nguyên hạn chế. Tuy nhiên, trong phạm vi mô phỏng trên Cooja, việc đo trực tiếp năng lượng (Joule/mAh) có thể không được thực hiện đầy đủ nếu không sử dụng các công cụ chuyên dụng. Do đó, đề tài lựa chọn cách ước lượng xu hướng tiêu thụ năng lượng dựa trên số lần truyền và nhận gói tin của các node.

Về nguyên tắc, hoạt động truyền (TX) và nhận (RX) tiêu tốn năng lượng đáng kể hơn so với trạng thái chờ (idle). Vì vậy, khi số lần TX/RX tăng thì mức tiêu thụ năng lượng của node cũng có xu hướng tăng theo. Trong đề tài này, chỉ tiêu overhead truyền thông (tổng số gói phát sinh hoặc tổng số lần truyền) được sử dụng như một đại lượng thay thế để ước lượng mức tiêu thụ năng lượng tương đối giữa các kịch bản.

Cụ thể:

- Kịch bản có overhead cao (nhiều TX/RX) được xem là có xu hướng tiêu thụ năng lượng cao hơn.
- Kịch bản giảm được số lần truyền dư thừa (đặc biệt ở node nguồn hoặc node trung gian) được xem là có lợi hơn về mặt năng lượng.

Trong phần đánh giá kết quả, đề tài sẽ dựa trên thống kê số gói gửi/nhận và số lần phát sinh gói tin trong log mô phỏng để so sánh xu hướng tiêu thụ năng lượng giữa unicast và multicast (cũng như giữa các thuật toán multicast nếu có).

4.6.6 Khả năng mở rộng theo số node

Khả năng mở rộng phản ánh mức độ ổn định và hiệu quả của cơ chế truyền dữ liệu khi số lượng node trong mạng tăng lên. Trong mạng 6LoWPAN, khi số node tăng, mật độ truyền thông và nguy cơ va chạm gói cũng tăng, từ đó có thể làm tăng độ trễ, giảm delivery ratio và tăng overhead.

Trong đề tài này, khả năng mở rộng có thể được đánh giá bằng cách giữ nguyên cấu hình và topology tổng thể, sau đó tăng số lượng node (ví dụ 7 node $\rightarrow$ 10 node hoặc 15 node) và thực hiện lại các kịch bản mô phỏng. Các chỉ tiêu theo dõi gồm:

- Độ trễ (delay)
- Tỷ lệ nhận gói (delivery ratio)
- Overhead truyền thông (số gói phát sinh/số lần truyền)

Việc so sánh kết quả theo số lượng node giúp xác định cơ chế nào phù hợp hơn khi mạng mở rộng. Trong trường hợp chưa thể triển khai đầy đủ nhiều mức số node trong phạm vi thời gian của đồ án, nội dung này sẽ được đề xuất như một hướng mở rộng trong chương kết luận và hướng phát triển.

Chương 5

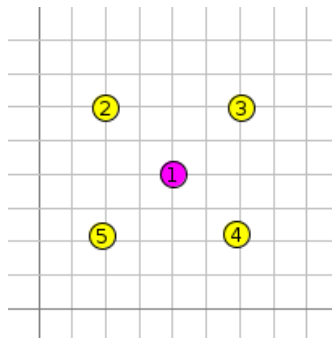
KẾT QUẢ MÔ PHỎNG VÀ THỰC NGHIỆM

5.1 Kết quả mô phỏng

Trong mục này, đề tài lần lượt trình bày kết quả mô phỏng thu được từ các kịch bản truyền dữ liệu đã thiết kế. Mỗi kịch bản được chạy trên cùng mô hình topology nhằm đảm bảo tính nhất quán. Kết quả được tổng hợp dựa trên log của các node trong cửa sổ Mote output và các thống kê cơ bản như số gói gửi/nhận, độ trễ và tỉ lệ mất gói.

5.1.1 Kết quả kịch bản Unicast

Trong kịch bản unicast, mạng được thiết lập gồm 5 node, trong đó node ID:1 đóng vai trò server đồng thời là node root của mạng RPL, còn các node ID:2 đến ID:5 là các client. Các node client thực hiện gửi gói tin UDP unicast đến node root và chờ nhận phản hồi tương ứng. Sau khi hiệu chỉnh chương trình, mỗi node client được cấu hình gửi đúng 100 request, tương ứng với các gói được đánh số từ 0 đến 99.

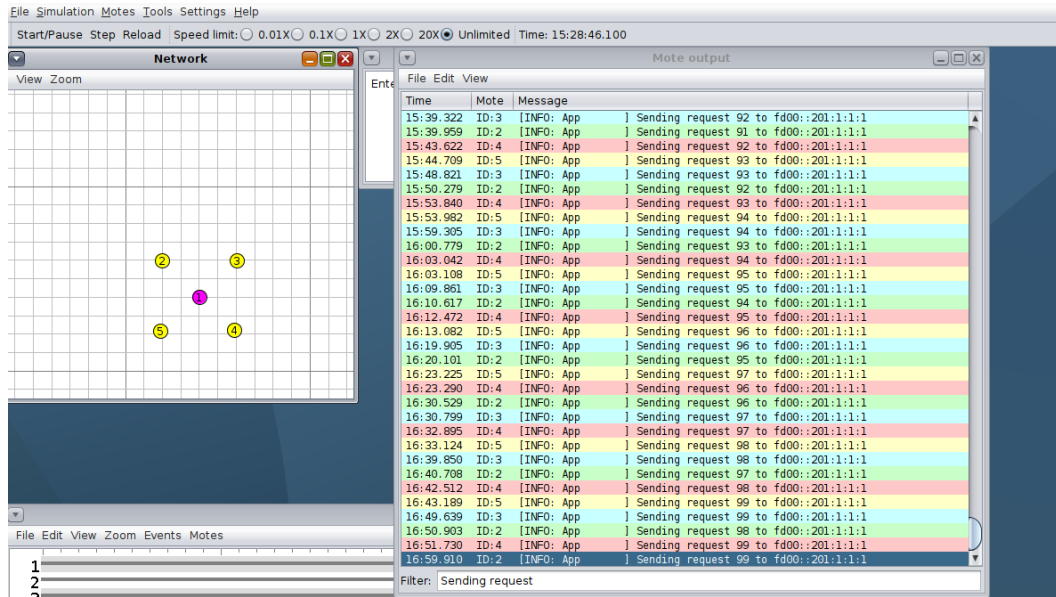


Hình 5.1: Topology mô phỏng kịch bản unicast trong Cooja

Hình trên minh họa topology sử dụng trong mô phỏng unicast. Trong đó, node ID:1 đóng vai trò node trung tâm (root/server), còn bốn node ID:2, ID:3, ID:4 và ID:5 đóng vai trò client. Các client gửi dữ liệu unicast trực tiếp về node root thông qua cơ chế UDP request/response. Cách bố trí này cho phép đánh giá khả năng truyền nhận dữ liệu giữa nhiều node gửi và một node trung tâm trong cùng một mạng RPL.

Kết quả từ cửa sổ Mote output cho thấy các node client đều gửi đủ từ request 0 đến request 99. Như vậy, mỗi client đã gửi thành công 100 gói request. Do có 4 client cùng hoạt động, tổng số gói request được gửi trong toàn mạng là:

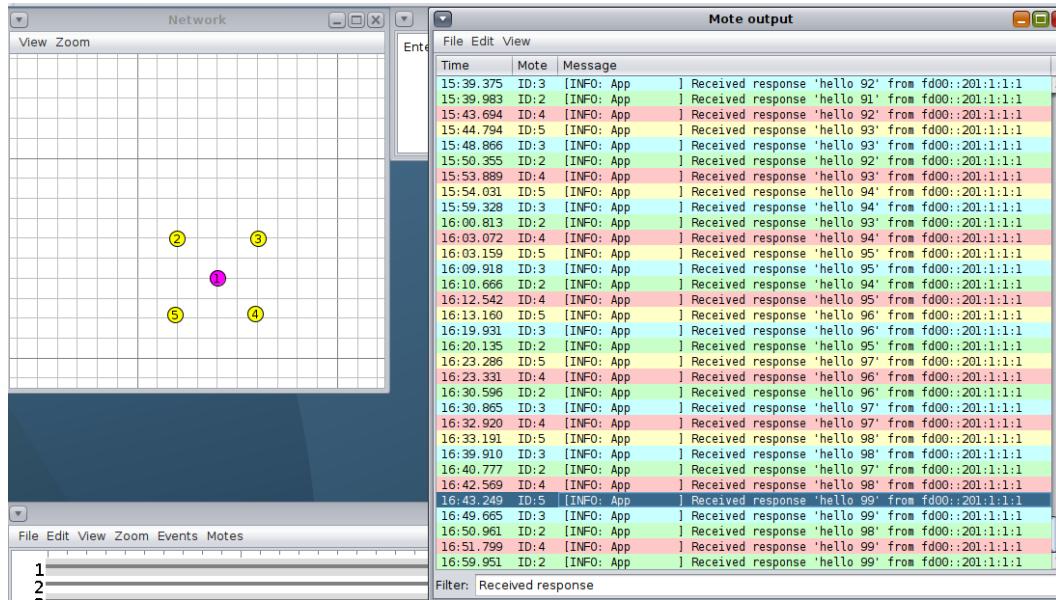
$$P_{send} = 4 \times 100 = 400$$



Hình 5.2: Log gửi request của các node client

Ngoài ra, log phản hồi cũng cho thấy các node client đều nhận được response tương ứng đến hello 99. Điều này chứng tỏ mỗi request đều có phản hồi từ node root, tức là toàn bộ quá trình truyền nhận request/response đã hoàn tất đầy đủ. Khi đó, tổng số gói response nhận được trong toàn mạng là:

$$P_{recv} = 400$$



Hình 5.3: Log nhận response của các node client

Từ các số liệu trên, tỉ lệ truyền thành công của kịch bản unicast được tính như sau:

$$\text{Delivery ratio} = \frac{P_{recv}}{P_{send}} \times 100\% = \frac{400}{400} \times 100\% = 100\%$$

Tương tự, tỉ lệ mất gói được xác định bởi:

$$\text{Packet loss ratio} = \frac{P_{send} - P_{recv}}{P_{send}} \times 100\% = \frac{400 - 400}{400} \times 100\% = 0\%$$

Để đánh giá độ trễ, đề tài sử dụng chỉ số RTT (Round-Trip Time), được tính bằng hiệu giữa thời điểm client gửi request và thời điểm nhận response tương ứng. Dựa trên log của node đại diện ID:2, RTT trung bình đo được là 49.71 ms, RTT nhỏ nhất là 15 ms và RTT lớn nhất là 83 ms. Với giả thiết thời gian lan truyền theo hai chiều gần tương đương nhau, độ trễ một chiều được ước lượng bằng một nửa RTT trung bình:

$$Delay_{1chiều} \approx \frac{RTT_{avg}}{2} = \frac{49.71}{2} \approx 24.86 \text{ ms}$$

Về overhead truyền thông, nếu xét theo số lượng request mà các client phát ra thì:

$$Overhead_{send} = P_{send} = 400$$

Nếu xét tổng số bản tin ứng dụng phát sinh trong toàn mạng, bao gồm cả request và response, thì:

$$Overhead_{app} = P_{send} + P_{recv} = 400 + 400 = 800$$

Như vậy, trong kịch bản unicast, node root phải tiếp nhận và phản hồi toàn bộ request từ các client, dẫn đến tải xử lý tập trung lớn tại node trung tâm.

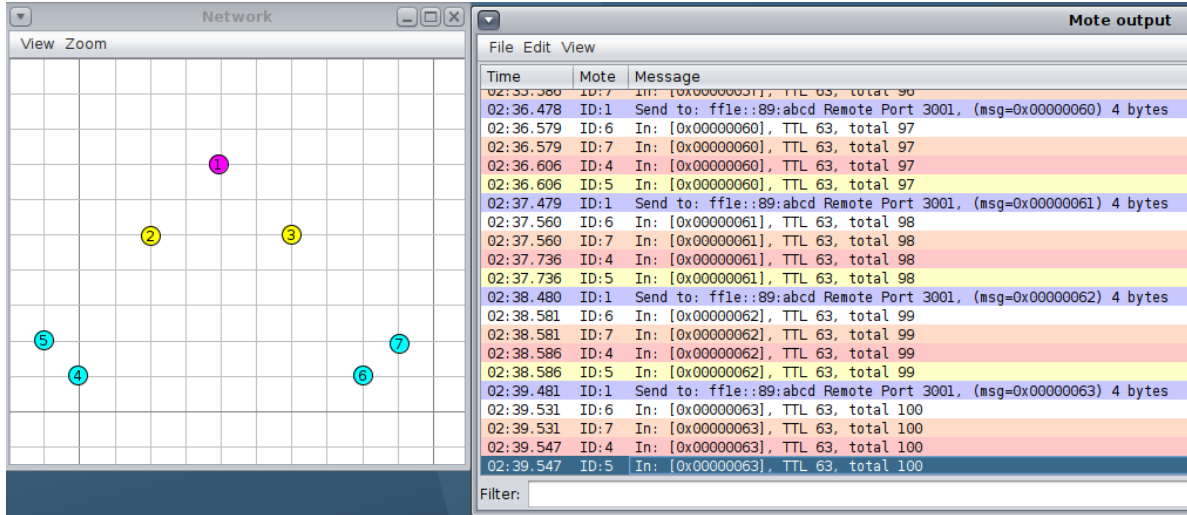
Thông số	Giá trị
Số node trong mạng	5
Node server/root	ID:1
Số node client	4 (ID:2-ID:5)
Số gói mỗi client gửi	100
Tổng số gói request gửi (P_{send})	400
Tổng số gói response nhận (P_{recv})	400
Delivery ratio	100%
Packet loss ratio	0%
RTT trung bình	49.71 ms
RTT min / max	15 ms / 83 ms
Delay 1 chiều (ước lượng)	≈ 24.86 ms
Overhead _{send}	400

Bảng 5.1: Kết quả thống kê kịch bản unicast

Từ kết quả mô phỏng có thể thấy rằng kịch bản unicast đạt độ tin cậy cao, thể hiện qua delivery ratio đạt 100% và packet loss ratio bằng 0%. Tuy nhiên, do mỗi client phải gửi request riêng đến node root và root cũng phải phản hồi riêng cho từng node, tổng số bản tin phát sinh trong mạng tương đối lớn. Điều này làm tăng overhead truyền thông và tải xử lý tại node root, đặc biệt khi số lượng client tăng lên. Đây là cơ sở để so sánh với các cơ chế multicast trong các mục tiếp theo.

5.1.2 Kết quả kịch bản Trickle Multicast

Trong kịch bản Trickle Multicast, mạng mô phỏng gồm 7 node, bao gồm 1 node gốc, 2 node trung gian và 4 node nhận. Node gốc thực hiện gửi dữ liệu multicast đến nhóm các node nhận thông qua cơ chế Trickle Multicast. Topology được bố trí theo dạng nhiều hop, trong đó dữ liệu từ node gốc được phân phối qua các node trung gian đến các sink. Cách bố trí này giúp kịch bản Trickle Multicast có cấu trúc tương đồng với các kịch bản SMRF và EMRF, từ đó thuận lợi hơn cho việc so sánh kết quả giữa các thuật toán multicast.



Hình 5.4: Kết quả mô phỏng kịch bản Trickle Multicast trong Cooja

Sau khi mạng hoàn tất quá trình khởi tạo và thiết lập định tuyến, node gốc bắt đầu gửi dữ liệu multicast đến nhóm các node nhận. Trong kịch bản này, số lần gửi từ node gốc là $K = 100$, số node nhận là $M = 4$. Kết quả mô phỏng cho thấy cả 4 node nhận đều nhận đủ 100 gói dữ liệu. Do đó, tổng số lượt nhận thành công tại các node nhận được xác định là:

$$P_{recv} = 100 + 100 + 100 + 100 = 400$$

Tổng số lượt nhận kỳ vọng được tính theo công thức:

$$P_{expected} = K \times M = 100 \times 4 = 400$$

Từ đó, tỉ lệ nhận gói được tính như sau:

$$Delivery\ Ratio = \frac{P_{recv}}{P_{expected}} \times 100\%$$

Thay số vào ta có:

$$Delivery\ Ratio = \frac{400}{400} \times 100\% = 100\%$$

Tỉ lệ mất gói được xác định theo công thức:

$$Packet\ Loss\ Ratio = 100\% - Delivery\ Ratio = 0\%$$

Kết quả trên cho thấy trong mô hình đã xây dựng, cơ chế Trickle Multicast đảm bảo khả năng phân phối dữ liệu đầy đủ đến toàn bộ các node nhận. Điều này chứng tỏ thuật toán hoạt động ổn định trong topology 7 node của đề tài.

Bên cạnh tỉ lệ nhận gói, độ trễ trong kịch bản multicast được đánh giá thông qua chỉ số $Delay_{spread}$, tức độ chênh lệch thời gian nhận cùng một gói dữ liệu giữa các node thuộc nhóm

nhận. Dựa trên log mô phỏng, gói cuối cùng được các node nhận thu được tại các thời điểm rất gần nhau, với mốc sớm nhất là 02:39.531 và muộn nhất là 02:39.547. Do đó, độ chênh lệch thời gian nhận được xác định là:

$$Delay_{spread} = 02:39.547 - 02:39.531 = 0.016 s = 16 ms$$

Giá trị này cho thấy dữ liệu được phân phối tương đối đồng đều đến các node nhận trong mạng. So với trường hợp các node nhận dữ liệu cách nhau quá xa về thời gian, mức chênh lệch 16 ms có thể xem là nhỏ và phản ánh khả năng truyền multicast ổn định trong mô hình đang xét.

Về overhead truyền thông, node gốc chỉ cần gửi 100 lần để phục vụ đồng thời toàn bộ 4 node nhận. Vì vậy, chỉ số $Overhead_{send}$ tại node gốc bằng 100. So với unicast, đây là một ưu điểm quan trọng vì dữ liệu không cần được gửi riêng đến từng node đích. Điều này góp phần giảm số lần phát tại nguồn, từ đó làm giảm tải truyền thông và tiết kiệm tài nguyên cho node gốc.

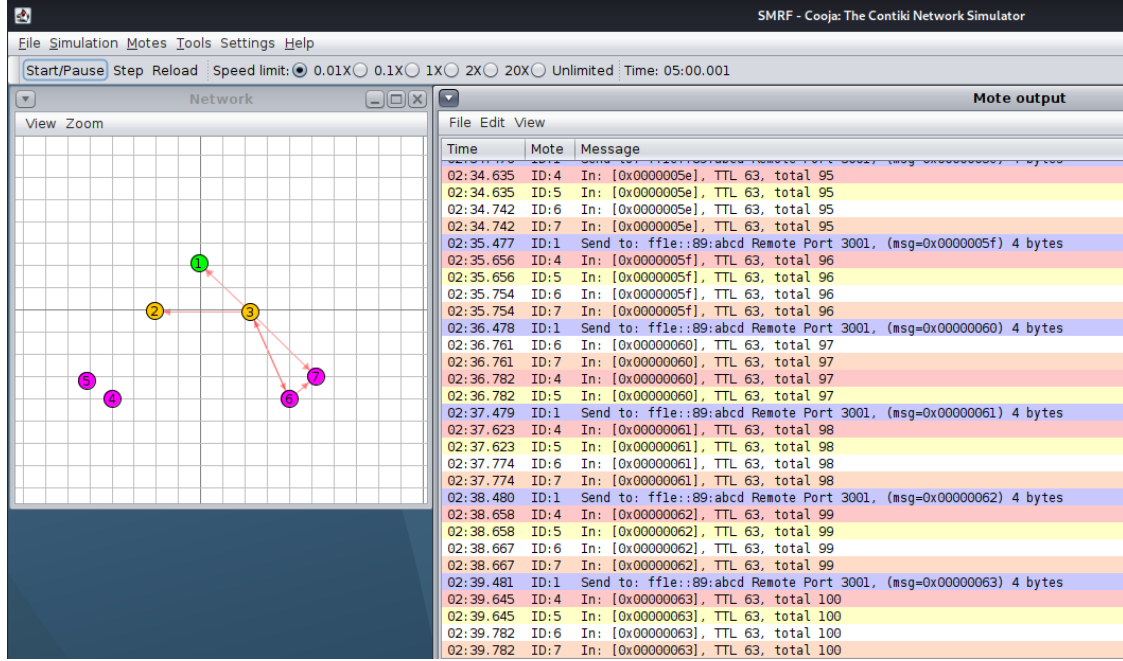
Tóm lại, kịch bản Trickle Multicast trong mô hình 7 node đã cho kết quả tốt với delivery ratio đạt 100%, packet loss ratio bằng 0%, $Delay_{spread} = 16 ms$ và $Overhead_{send} = 100$. Kết quả này cho thấy Trickle Multicast phù hợp với bài toán truyền cùng một dữ liệu đến nhiều node trong mạng 6LoWPAN, đồng thời tạo cơ sở thuận lợi để so sánh với các cơ chế multicast khác như SMRF và EMRF ở các mục tiếp theo.

Thông số	Giá trị
Tổng số node	7
Node gốc (root)	1
Node trung gian (intermediate)	2
Số node nhận (sink) – M	4
Thời gian mô phỏng	$\approx 160 s$
Số lần root gửi (K)	100
Tổng lượt nhận thành công (P_{recv})	400
Tổng lượt nhận kỳ vọng ($P_{expected} = K \times M$)	400
Delivery ratio	100%
Packet loss ratio	0%
$Delay_{spread}$	16 ms
$Overhead_{send}$	100
Năng lượng (ước lượng qua TX/RX)	root TX giảm so với unicast

Bảng 5.2: Kết quả thống kê kịch bản Trickle Multicast

5.1.3 Kết quả kịch bản SMRF

Trong kịch bản này, cơ chế multicast được cấu hình theo thuật toán SMRF. Mạng mô phỏng gồm 7 node, trong đó có 1 node gốc, 2 node trung gian và 4 node nhận. Topology được bố trí theo dạng cây nhiều-hop để phản ánh đặc trưng của SMRF, trong đó node gốc phân phối dữ liệu theo các nhánh qua các node trung gian đến các sink.



Hình 5.5: Kết quả mô phỏng kịch bản SMRF trong Cooja

Để cây định tuyến RPL/DODAG có thời gian hội tụ trước khi truyền dữ liệu, node gốc không gửi ngay khi bắt đầu mô phỏng mà chờ một khoảng $START_DELAY = 60s$. Sau đó, node gốc gửi dữ liệu multicast với chu kỳ $SEND_INTERVAL = 1s$. Trong chương trình nguồn, số lần gửi được giới hạn bởi tham số $ITERATIONS = 100$, do đó node gốc phát tổng cộng 100 gói multicast.

Dựa trên log của cửa sổ Mote Output, node gốc quan sát được gói cuối cùng mang mã 0x00000063, tương ứng với 100 lần gửi kể từ gói đầu tiên 0x00000000. Đồng thời, 4 node nhận ID:4, ID:5, ID:6 và ID:7 đều nhận đủ 100 gói, nên tổng số lượt nhận thành công trong toàn mạng là:

$$P_{recv} = 100 + 100 + 100 + 100 = 400$$

Với số lần node gốc gửi là $K = 100$ và số node nhận là $M = 4$, tổng số lượt nhận kỳ vọng được tính:

$$P_{expected} = K \times M = 100 \times 4 = 400$$

Từ đó:

$$Delivery\ ratio = \frac{P_{recv}}{P_{expected}} \times 100\% = \frac{400}{400} \times 100\% = 100\%$$

$$Packet\ loss\ ratio = 100\% - Delivery\ ratio = 0\%$$

Về độ trễ, dựa trên log của gói 0x00000061, node gốc gửi tại thời điểm 02:37.479, trong khi các sink nhận trong khoảng từ 02:37.623 đến 02:37.774. Do đó:

$$delay_{min} = 02:37.623 - 02:37.479 = 0.144s = 144ms$$

$$delay_{max} = 02:37.774 - 02:37.479 = 0.295s = 295ms$$

Vì trong bảng thống kê sử dụng chỉ số $delay_{spread}$, giá trị này được xác định bằng chênh lệch giữa thời điểm nhận sớm nhất và muộn nhất:

$$delay_{spread} = 02:37.774 - 02:37.623 = 0.151s = 151ms$$

Về thời gian, do mô phỏng sử dụng $START_DELAY = 60s$ và sau đó node gốc gửi 100 gói với chu kỳ 1 giây mỗi gói, tổng thời gian mô phỏng để hoàn tất quá trình gửi xấp xỉ:

$$60 + 100 = 160s$$

Như vậy, tổng thời gian mô phỏng của kịch bản SMRF khoảng 160 giây, trong đó 60 giây đầu là thời gian chờ hội tụ định tuyến và 100 giây sau là thời gian truyền dữ liệu thực tế.

Về overhead truyền thông, trong phạm vi thống kê hiện tại, chỉ số $Overhead_{send}$ được lấy theo số lần node gốc phát gói, do đó:

$$Overhead_{send} = 100$$

Xét về mặt năng lượng, do multicast cho phép một lần truyền từ node gốc phục vụ đồng thời nhiều node nhận, nên số lần TX tại root giảm so với kịch bản unicast. Vì vậy, năng lượng ước lượng qua hoạt động TX/RX tại root được xem là thấp hơn unicast.

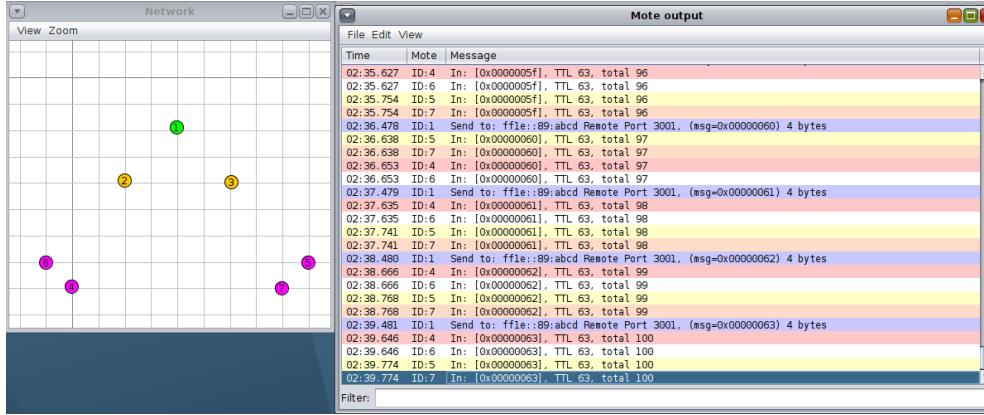
Kịch bản SMRF cho kết quả tốt trong mạng mô phỏng hiện tại với delivery ratio đạt 100%, packet loss ratio bằng 0%, $delay_{spread}$ bằng 151 ms, và 100 lần gửi từ node gốc. Kết quả này cho thấy SMRF hoạt động ổn định trên topology đã thiết lập và phù hợp với mục tiêu phân phối dữ liệu multicast trong mạng 6LoWPAN quy mô nhỏ.

Thông số	Giá trị
Tổng số node	7
Node gốc (root)	1
Node trung gian (intermediate)	2
Số node nhận (sink) – M	4
Thời gian mô phỏng	≈ 160 s
Số lần root gửi (K)	100
Tổng lượt nhận thành công (P_{recv})	400
Tổng lượt nhận kỳ vọng ($P_{expected} = K \times M$)	400
Delivery ratio	100%
Packet loss ratio	0%
$Delay_{spread}$	151 ms
$Overhead_{send}$	100
Năng lượng (ước lượng qua TX/RX)	root TX giảm so với unicast

Bảng 5.3: Kết quả thống kê kịch bản SMRF

5.1.4 Kết quả kịch bản EMRF

Trong kịch bản EMRF, mạng mô phỏng gồm 7 node, bao gồm 1 node gốc, 2 node trung gian và 4 node nhận. Node 1 đóng vai trò root, node 2 và node 3 là các node trung gian, còn node 4, node 5, node 6 và node 7 là các node sink thuộc nhóm nhận multicast. Sau thời gian khởi tạo và hội tụ cây định tuyến RPL, node gốc bắt đầu gửi dữ liệu multicast đến nhóm node nhận theo cơ chế EMRF.



Hình 5.6: Kết quả mô phỏng kịch bản EMRF trong Cooja

Từ log mô phỏng trên Cooja có thể quan sát được rằng root gửi tổng cộng 100 gói dữ liệu, đồng thời cả 4 node sink đều nhận đủ 100 gói. Giá trị total tại các node 4, 5, 6 và 7 ở cuối quá trình mô phỏng đều bằng 100, cho thấy toàn bộ các gói multicast đã được phân phối thành công đến tất cả các node nhận trong nhóm.

Với số lần root gửi là $K = 100$ và số node nhận là $M = 4$, tổng lượt nhận kỳ vọng được xác định như sau:

$$P_{expected} = K \times M = 100 \times 4 = 400$$

Do bốn node sink đều nhận đủ 100 gói, tổng lượt nhận thành công là:

$$P_{recv} = 100 + 100 + 100 + 100 = 400$$

Từ đó, tỉ lệ nhận gói của kịch bản EMRF được tính như sau:

$$Delivery\ Ratio = \frac{P_{recv}}{P_{expected}} \times 100\% = \frac{400}{400} \times 100\% = 100\%$$

Khi đó, tỉ lệ mất gói là:

$$Packet\ Loss\ Ratio = 100\% - Delivery\ Ratio = 0\%$$

Kết quả này cho thấy trong topology đang xét, cơ chế EMRF hoạt động ổn định và đảm bảo khả năng phân phối dữ liệu multicast đến toàn bộ các node đích mà không xảy ra mất gói. Đây là một kết quả tốt đối với mô hình mạng 6LoWPAN quy mô nhỏ, khi mục tiêu là truyền cùng một nội dung dữ liệu đến nhiều node nhận trong cùng một nhóm multicast.

Về độ trễ phân tán giữa các node nhận, chỉ số $Delay_{spread}$ được tính bằng hiệu giữa thời điểm nhận muộn nhất và thời điểm nhận sớm nhất của cùng một gói tin tại các sink. Xét gói cuối cùng có mã $msg = 0x00000063$, log mô phỏng cho thấy node 4 và node 6 nhận gói tại thời điểm 02:39.646, trong khi node 5 và node 7 nhận tại thời điểm 02:39.774. Do đó:

$$Delay_{spread} = 02:39.774 - 02:39.646 = 128\ ms$$

Giá trị này cho thấy chênh lệch thời gian nhận cùng một gói giữa các sink là nhỏ, tức dữ liệu được phân phối khá đồng đều trên các nhánh của topology. Trong bối cảnh mạng cảm biến không dây công suất thấp, kết quả này cho thấy EMRF không chỉ bảo đảm độ phủ gói tin mà còn duy trì được tính đồng bộ tương đối tốt giữa các node nhận.

Về thời gian mô phỏng, kịch bản EMRF sử dụng thời gian chờ hội tụ định tuyến là 60 giây và sau đó root gửi 100 gói với chu kỳ 1 giây mỗi gói. Vì vậy, tổng thời gian mô phỏng được xác định xấp xỉ:

$$T \approx 60 + 100 = 160 \text{ s}$$

Như vậy, tổng thời gian mô phỏng của kịch bản EMRF là khoảng 160 giây, trong đó 60 giây đầu dành cho quá trình ổn định mạng và 100 giây tiếp theo là quá trình truyền dữ liệu multicast thực tế.

Ngoài ra, trong phạm vi đánh giá của đề tài, chỉ số $Overhead_{send}$ được lấy theo số lần node gốc phát gói. Do root gửi tổng cộng 100 gói nên:

$$Overhead_{send} = 100$$

Xét về mặt năng lượng, EMRF vẫn là một cơ chế multicast nên một lần phát từ root có thể phục vụ đồng thời nhiều node nhận. Vì vậy, số lần truyền tại node gốc thấp hơn so với phương pháp unicast, từ đó cho thấy mức tiêu thụ năng lượng ước lượng qua hoạt động TX/RX tại root cũng giảm hơn so với kịch bản unicast.

Tổng hợp lại, kịch bản EMRF trong mô hình 7 node đã cho kết quả rất tốt với tổng lượt nhận thành công đạt 400 trên 400 lượt kỳ vọng, tương ứng delivery ratio bằng 100%, packet loss ratio bằng 0%, $Delay_{spread}$ bằng 128 ms và $Overhead_{send}$ bằng 100. Kết quả này cho thấy EMRF phù hợp với mục tiêu truyền multicast trong mạng 6LoWPAN quy mô nhỏ, đảm bảo dữ liệu được phân phối đầy đủ đến các node nhận với độ chênh lệch thời gian thấp. Tuy nhiên, do topology hiện tại khá ổn định nên kết quả của EMRF có thể không khác biệt nhiều so với SMRF. Trong các nghiên cứu tiếp theo, có thể mở rộng số lượng node hoặc bố trí topology phức tạp hơn để đánh giá rõ hơn ưu điểm của EMRF trong môi trường liên kết kém ổn định.

Thông số	Giá trị
Tổng số node	7
Node gốc (root)	1
Node trung gian (intermediate)	2
Số node nhận (sink) – M	4
Thời gian mô phỏng	$\approx 160 \text{ s}$
Số lần root gửi (K)	100
Tổng lượt nhận thành công (P_{recv})	400
Tổng lượt nhận kỳ vọng ($P_{expected} = K \times M$)	400
Delivery ratio	100%
Packet loss ratio	0%
$Delay_{spread}$	128 ms
$Overhead_{send}$	100
Năng lượng (ước lượng qua TX/RX)	root TX giảm so với unicast

Bảng 5.4: Kết quả thống kê kịch bản EMRF

5.2 So sánh kết quả

Sau khi thực hiện mô phỏng cho bốn kịch bản gồm unicast, Trickle Multicast, SMRF và EMRF, có thể tiến hành so sánh kết quả theo các tiêu chí chính là độ trễ truyền dữ liệu, tỉ lệ nhận gói, tỉ lệ mất gói và overhead truyền thông. Đây là những tiêu chí quan trọng nhằm đánh giá hiệu năng của từng cơ chế truyền dữ liệu trong mạng 6LoWPAN. $Overhead_{send}$ tại node gốc đều bằng 100.

Kết quả thu được cho thấy cả bốn kịch bản đều hoạt động ổn định trong điều kiện mô phỏng đã xây dựng. Ở kịch bản unicast, RTT trung bình đạt khoảng 49.71 ms. Đối với các kịch bản multicast, Trickle Multicast có $Delay_{spread} = 16\text{ ms}$, SMRF có $Delay_{spread} = 151\text{ ms}$ và EMRF có $Delay_{spread} = 128\text{ ms}$. Đồng thời, Trickle Multicast, SMRF và EMRF đều đạt 100% Delivery ratio, Packet loss ratio = 0%, và $Overhead_{send} = 100$ tại node gốc.

Kịch bản	Topology	Chỉ số độ trễ	Delivery ratio	Packet loss ratio	Overhead gửi từ node gốc / node gửi	Nhận xét
Unicast	5 node (1 root, 4 client)	RTT trung bình = 49.71 ms; độ trễ một chiều ước lượng $\approx 24.86\text{ ms}$	100%	0%	400 request; nếu tính cả request và response thì tổng bản tin ứng dụng là 800	Độ tin cậy cao, độ trễ thấp, nhưng overhead lớn do phải gửi riêng đến từng node đích.
Trickle Multicast	7 node (1 root, 2 trung gian, 4 sink)	Delay spread = 16 ms	100%	0%	100	Hoạt động rất tốt trong topology hiện tại, dữ liệu đến các sink gần như đồng thời, overhead thấp hơn unicast rõ rệt.
SMRF	7 node (1 root, 2 trung gian, 4 sink)	Delay spread = 151 ms	100%	0%	100	Overhead thấp, hoạt động ổn định trên cây DODAG, nhưng độ chênh lệch thời gian nhận giữa các sink cao hơn Trickle và EMRF.
EMRF	7 node (1 root, 2 trung gian, 4 sink)	Delay spread = 128 ms	100%	0%	100	Giữ được delivery ratio cao, overhead thấp như SMRF nhưng độ trễ phân tán tốt hơn SMRF trong cùng topology.

Bảng 5.5: So sánh kết quả giữa các kịch bản truyền dữ liệu

Bảng 5.5 trình bày kết quả tổng hợp của bốn kịch bản truyền dữ liệu được khảo sát. Thông qua bảng này có thể quan sát tổng quát sự khác biệt giữa unicast và các cơ chế multicast về độ trễ, độ tin cậy và overhead truyền thông. Các tiêu chí được sử dụng trong bảng cũng phù hợp với định hướng đánh giá hiệu năng đã nêu từ trước trong đề tài.

5.2.1 So sánh theo độ trễ truyền dữ liệu

Đối với kịch bản unicast, độ trễ được đánh giá thông qua chỉ số RTT, tức thời gian từ lúc node gửi request đến khi nhận lại response tương ứng. Kết quả cho thấy RTT trung bình đạt khoảng 49.71 ms, phản ánh quá trình truyền và phản hồi dữ liệu giữa các node trong mạng diễn ra ổn định.

Đối với các kịch bản multicast, độ trễ được đánh giá bằng chỉ số $Delay_{spread}$, tức độ chênh lệch thời gian nhận cùng một gói dữ liệu giữa các node thuộc nhóm nhận. Kết quả cho thấy Trickle Multicast có $Delay_{spread} = 16\text{ ms}$, SMRF có $Delay_{spread} = 151\text{ ms}$ và EMRF có $Delay_{spread}$ là 128 ms. Như vậy, trong ba cơ chế multicast được khảo sát, Trickle Multicast cho độ chênh lệch thời gian nhận nhỏ nhất, tiếp theo là EMRF và sau cùng là SMRF.

Kết quả này cho thấy sau khi sử dụng topology 7 node với 2 node trung gian, Trickle Multicast vẫn duy trì được độ đồng đều thời điểm nhận tốt hơn so với SMRF và EMRF trong điều kiện mô phỏng hiện tại. Trong khi đó, EMRF cho kết quả tốt hơn SMRF về mặt độ trễ trong cùng mô hình 7 node. Điều này cho thấy EMRF đã cải thiện phần nào hiệu năng phân phối dữ liệu so với SMRF, còn Trickle Multicast hoạt động rất tốt trong topology đã xây dựng.

Tuy nhiên, cũng cần lưu ý rằng độ trễ giữa unicast và multicast chưa được đo theo cùng một cách. Unicast được đánh giá bằng RTT, trong khi multicast được đánh giá bằng $Delay_{spread}$. Vì vậy, việc so sánh giữa unicast và multicast về độ trễ chỉ mang ý nghĩa tương đối, chủ yếu nhằm chỉ ra xu hướng khác biệt giữa hai cơ chế truyền dữ liệu.

5.2.2 So sánh theo tỉ lệ nhận gói và tỉ lệ mất gói

Tỉ lệ nhận gói và tỉ lệ mất gói là những tiêu chí quan trọng để đánh giá độ tin cậy của cơ chế truyền dữ liệu trong mạng 6LoWPAN. Kết quả thu được cho thấy cả ba kịch bản multicast gồm Trickle Multicast, SMRF và EMRF đều đạt $Delivery\ ratio = 100\%$ và $Packet\ loss\ ratio = 0\%$. Điều này cho thấy dữ liệu từ node gốc đã được phân phối đầy đủ đến toàn bộ các node nhận trong điều kiện mô phỏng hiện tại.

Đối với unicast, kết quả cũng cho thấy quá trình truyền dữ liệu diễn ra ổn định với độ tin cậy cao. Như vậy, xét theo tiêu chí delivery/loss, cả unicast và các cơ chế multicast đều đáp ứng tốt yêu cầu truyền dữ liệu trong mô hình đã xây dựng.

Việc cả ba thuật toán multicast đều đạt tỉ lệ nhận gói 100% cho thấy trong topology 7 node hiện tại, dữ liệu được phân phối thành công đến toàn bộ các sink mà không xảy ra mất gói. Tuy nhiên, kết quả này cũng phản ánh rằng môi trường mô phỏng còn khá ổn định, nên sự khác biệt về độ tin cậy giữa các thuật toán chưa thể hiện rõ rệt. Vì vậy, trong phạm vi đề tài, tiêu chí delivery ratio và packet loss ratio chủ yếu cho thấy cả ba cơ chế multicast đều hoạt động tốt trong mạng quy mô nhỏ, hơn là dùng để kết luận thuật toán nào vượt trội tuyệt đối về độ phủ gói tin.

5.2.3 So sánh theo overhead truyền thông

Overhead truyền thông phản ánh số lượng gói tin hoặc số lần truyền phát sinh trong mạng để hoàn thành việc phân phối dữ liệu. Đây là tiêu chí rất quan trọng trong mạng 6LoWPAN vì các node thường bị giới hạn về năng lượng, băng thông và khả năng xử lý. Phần lý thuyết của đề tài cũng đã chỉ ra rằng multicast được nghiên cứu nhằm giảm số lượng gói tin so với cách gửi unicast lần lượt đến từng node, từ đó tiết kiệm tài nguyên mạng.

Trong kịch bản unicast, dữ liệu phải được gửi riêng đến từng node đích nên số lượng trao đổi trong mạng lớn hơn. Khi số node nhận tăng, node nguồn phải thực hiện nhiều lần truyền riêng biệt, làm cho overhead cao hơn. Đây là hạn chế cơ bản của unicast trong các bài toán cần gửi cùng một nội dung đến nhiều node trong mạng.

Ngược lại, ở các kịch bản multicast, một lần truyền từ node gốc có thể phục vụ đồng thời nhiều node nhận. Kết quả cho thấy ở Trickle Multicast, SMRF và EMRF, chỉ số $Overhead_{send}$ tại node gốc đều bằng 100, tức root chỉ cần gửi 100 lần để phục vụ toàn bộ nhóm 4 node nhận. Điều này cho thấy multicast có ưu thế rõ rệt về số lần truyền tại nguồn so với unicast. Đồng thời, ở SMRF và EMRF, phần đánh giá cũng cho thấy số lần TX tại root giảm so với unicast, nên năng lượng ước lượng qua hoạt động TX/RX tại node gốc cũng thấp hơn.

Như vậy, xét theo tiêu chí overhead truyền thông, multicast thể hiện ưu thế rõ rệt hơn so với unicast trong bài toán gửi cùng một nội dung đến nhiều node. Trong ba cơ chế multicast được khảo sát, cả Trickle Multicast, SMRF và EMRF đều giữ được ưu điểm giảm số lần phát tại node gốc nhờ cơ chế gửi theo địa chỉ nhóm.

**Nhận xét chung*

Từ các kết quả trên có thể thấy rằng cả unicast và multicast đều hoạt động ổn định trong mô hình mô phỏng đã xây dựng. Tuy nhiên, multicast thể hiện ưu thế rõ rệt hơn khi cần gửi cùng một dữ liệu đến nhiều node, đặc biệt ở khía cạnh overhead truyền thông. Trong nhóm multicast, Trickle Multicast cho $Delay_{spread}$ nhỏ nhất với 16 ms, trong khi EMRF cho kết quả tốt

hơn SMRF về độ trễ trong cùng mô hình 7 node. Đồng thời, cả ba cơ chế multicast đều duy trì $Delivery\ ratio = 100\%$ và $Packet\ loss\ ratio = 0\%$, cho thấy khả năng phân phối dữ liệu đáng tin cậy trong mạng quy mô nhỏ.

Mặc dù vậy, do môi trường mô phỏng hiện tại còn tương đối ổn định nên sự khác biệt giữa các thuật toán multicast mới chủ yếu thể hiện ở tiêu chí độ trễ, trong khi delivery ratio và packet loss ratio chưa tạo ra sự phân hóa rõ ràng. Vì vậy, để đánh giá đầy đủ hơn hiệu năng của từng thuật toán, cần có thêm các kịch bản mô phỏng với số node lớn hơn hoặc điều kiện truyền dẫn khó khăn hơn.

5.3 Phân tích nguyên nhân

Từ các kết quả mô phỏng đã trình bày ở mục trước, có thể thấy rằng cả unicast và các cơ chế multicast đều hoạt động ổn định trong mô hình 6LoWPAN đã xây dựng. Tuy nhiên, sự khác biệt giữa các kịch bản vẫn thể hiện rõ ở hai tiêu chí chính là độ trễ và overhead truyền thông, trong khi delivery ratio và packet loss ratio gần như chưa tạo ra sự phân hóa. Để giải thích cho các kết quả này, cần xem xét đồng thời cả đặc điểm của topology mô phỏng, điều kiện môi trường truyền dẫn và nguyên lý hoạt động của từng cơ chế truyền dữ liệu.

Trước hết, nguyên nhân khiến cả ba kịch bản multicast đều đạt delivery ratio bằng 100% và packet loss ratio bằng 0% là do topology mô phỏng còn tương đối nhỏ và các liên kết giữa các node khá ổn định. Trong mô hình hiện tại, số node chưa nhiều, số node nhận chỉ ở mức 4, đồng thời khoảng cách giữa các node được bố trí đủ để dữ liệu có thể được chuyển tiếp thuận lợi qua các hop. Với điều kiện như vậy, xác suất mất gói trên đường truyền chưa lớn, nên cả Trickle Multicast, SMRF và EMRF đều có thể phân phối đầy đủ dữ liệu đến các node nhận. Điều này cũng phù hợp với cách tính chỉ tiêu delivery ratio trong đề tài, khi số lượt nhận kỳ vọng của multicast được xác định theo công thức $P_{expected} = K \times M$.

Đối với Unicast, độ tin cậy cao có thể được giải thích bởi bản chất truyền một-một của cơ chế này. Mỗi client gửi request riêng đến node gốc và nhận response tương ứng, nên quá trình trao đổi dữ liệu tương đối rõ ràng, trực tiếp và ít phụ thuộc vào việc chuyển tiếp theo nhóm. Trong mạng quy mô nhỏ, cơ chế này thường hoạt động ổn định và dễ kiểm soát, vì vậy kết quả RTT trung bình khoảng 49.71 ms và delivery ratio cao là điều phù hợp với đặc điểm của mô hình đã xây dựng. Tuy nhiên, nhược điểm của unicast nằm ở chỗ cùng một nội dung nhưng phải gửi riêng đến từng node đích, làm cho số lượng bản tin phát sinh trong mạng tăng lên đáng kể. Đây là lý do chính khiến overhead của unicast cao hơn so với multicast.

Đối với Trickle Multicast, kết quả $Delay_{spread} = 16\ ms$ cho thấy dữ liệu đến các sink khá đồng đều về mặt thời gian. Nguyên nhân là vì trong topology hiện tại, cơ chế Trickle Multicast vẫn duy trì được khả năng phát tán dữ liệu hiệu quả giữa các node lân cận. Mặc dù Trickle có cơ chế suppress dựa trên mức dư thừa và về lý thuyết có thể làm tăng độ trễ do interval, nhưng trong mạng nhỏ và ổn định, ảnh hưởng này chưa biểu hiện rõ. Ngược lại, các node nhận vẫn có thể nhận dữ liệu trong khoảng thời gian rất gần nhau. Điều này phù hợp với nhận xét ở phần lý thuyết rằng Trickle Multicast có ưu điểm giảm trùng lặp truyền thông, nhưng độ trễ còn phụ thuộc vào cách thiết lập khoảng thời gian và môi trường mạng.

Đối với SMRF, kết quả $Delay_{spread} = 151\ ms$ lớn hơn đáng kể so với Trickle Multicast. Nguyên nhân chính là do SMRF hoạt động dựa trên cấu trúc DODAG của RPL, nghĩa là gói dữ liệu chỉ được forward theo các nhánh có liên quan đến nhóm nhận. Cách làm này giúp giảm overhead trong mạng, nhưng đồng thời thời điểm nhận dữ liệu giữa các sink có thể bị lệch nhau tùy theo độ dài hành trình, vị trí relative và thứ tự chuyển tiếp. Nói cách khác, SMRF chấp nhận sự chênh lệch thời gian nhận lớn hơn để đổi lấy cơ chế forwarding gọn nhẹ và ít bản tin dư thừa hơn. Điều này cũng đúng với đặc điểm lý thuyết của SMRF, là một cơ chế stateless, overhead thấp và phù

hợp với mạng RPL ổn định.

Đối với EMRF, kết quả $Delay_{spread} = 128\text{ ms}$, nhỏ hơn so với SMRF nhưng vẫn lớn hơn Trickle Multicast. Kết quả này có thể được giải thích bởi bản chất EMRF là phiên bản nâng cao của SMRF, được thiết kế nhằm cải thiện độ phủ và tính ổn định của quá trình phân phối multicast. Trong cùng topology 7 node, EMRF vẫn giữ được ưu điểm overhead thấp như SMRF, nhưng cơ chế forwarding được cải tiến giúp dữ liệu đến các sink đồng đều hơn một mức nhất định. Vì vậy, EMRF cho kết quả tốt hơn SMRF về mặt độ trễ phân tán. Điều này cũng phù hợp với phân lý thuyết ở Chương 3, trong đó EMRF được xem là hướng cải tiến của SMRF để hoạt động tốt hơn trong môi trường lossy hoặc khi cần độ phủ cao hơn.

Một nguyên nhân quan trọng khác cần được nhấn mạnh là sự khác nhau về cách đo độ trễ giữa unicast và multicast. Trong đề tài, unicast được đánh giá bằng RTT, tức thời gian khứ hồi giữa request và response, còn multicast được đánh giá bằng $Delay_{spread}$, tức độ chênh lệch thời gian nhận cùng một gói giữa các node nhận. Do hai đại lượng này không hoàn toàn tương đương, nên không thể dùng chúng để kết luận trực tiếp rằng cơ chế này “nhanh hơn” cơ chế kia theo nghĩa tuyệt đối. Việc so sánh chỉ nên được hiểu là nhằm quan sát xu hướng hoạt động của từng cơ chế trong mô hình hiện tại. Đây cũng là một nguyên nhân khiến phần so sánh cần được diễn giải thận trọng, đặc biệt khi đối chiếu giữa unicast và multicast.

Xét theo tiêu chí overhead truyền thông, nguyên nhân khiến multicast vượt trội hơn unicast là do phương thức gửi dữ liệu theo nhóm. Trong unicast, để phục vụ nhiều node nhận, node nguồn phải gửi riêng nhiều lần đến từng node đích. Ngược lại, trong multicast, node gốc chỉ cần gửi một gói đến địa chỉ nhóm và quá trình phân phối tiếp theo được thực hiện thông qua các node trung gian hoặc cơ chế chuyển tiếp của từng thuật toán. Vì vậy, với cùng 4 node nhận và cùng 100 lần phát dữ liệu từ root, overhead gửi tại node gốc của các kịch bản multicast chỉ là 100, thấp hơn rõ rệt so với unicast. Đây là nguyên nhân quan trọng nhất lý giải vì sao multicast phù hợp hơn với mạng 6LoWPAN khi cần phân phối cùng một nội dung đến nhiều node.

Bên cạnh đó, việc cả ba kịch bản multicast đều sử dụng cùng topology 7 node với 2 node trung gian và 4 node nhận cũng giúp kết quả so sánh trở nên thống nhất hơn. Khi cùng một cấu trúc mạng được áp dụng cho nhiều thuật toán, sự khác biệt về kết quả phản ánh rõ hơn ảnh hưởng của chính cơ chế forwarding, thay vì bị chi phối quá nhiều bởi sự thay đổi topology. Đây là một điểm quan trọng làm cho phần so sánh ở mục 5.3 có cơ sở chặt chẽ hơn.

Tóm lại, nguyên nhân dẫn đến các kết quả mô phỏng trong đề tài có thể được lý giải như sau: unicast có độ tin cậy cao nhưng overhead lớn do phải gửi riêng đến từng node; Trickle Multicast cho độ trễ phân tán nhỏ nhờ khả năng phát tán dữ liệu khá đồng đều trong topology hiện tại; SMRF có overhead thấp nhưng độ lệch thời gian nhận cao hơn do phụ thuộc vào quá trình forward theo DODAG; EMRF cải thiện được phần nào độ đồng đều thời gian nhận so với SMRF nhờ cơ chế nâng cao. Đồng thời, việc môi trường mô phỏng còn nhỏ và ổn định là nguyên nhân khiến delivery ratio của cả ba cơ chế multicast đều đạt 100% và chưa thể hiện rõ sự khác biệt về mất gói.

5.4 Kiểm chứng trên FIT/IoT-LAB

5.4.1 Mục đích kiểm chứng

Mặc dù các kịch bản unicast và multicast đã được xây dựng, mô phỏng và đánh giá trên nền tảng Contiki-NG/Cooja, việc kiểm chứng lại trên FIT/IoT-LAB vẫn là cần thiết nhằm đối chiếu khả năng hoạt động của chương trình trong môi trường phần cứng thực tế. Nếu mô phỏng trên Cooja chủ yếu giúp quan sát topology, log hoạt động và các chỉ số hiệu năng trong điều kiện được kiểm soát, thì FIT/IoT-LAB cho phép triển khai trực tiếp firmware lên các node thật để kiểm tra quá trình khởi động, tham gia mạng, gửi, nhận và chuyển tiếp dữ liệu.

Bước kiểm chứng này không nhằm thay thế hoàn toàn phần mô phỏng, mà chủ yếu dùng để xác nhận rằng các chương trình đã xây dựng có thể vận hành đúng chức năng trên testbed thực. Thông qua đó, đề tài có thêm cơ sở để nhận xét về tính đúng đắn của các kịch bản unicast, Trickle Multicast, SMRF và EMRF khi chuyển từ môi trường mô phỏng sang môi trường node thật.

5.4.2 Thiết lập kiểm chứng

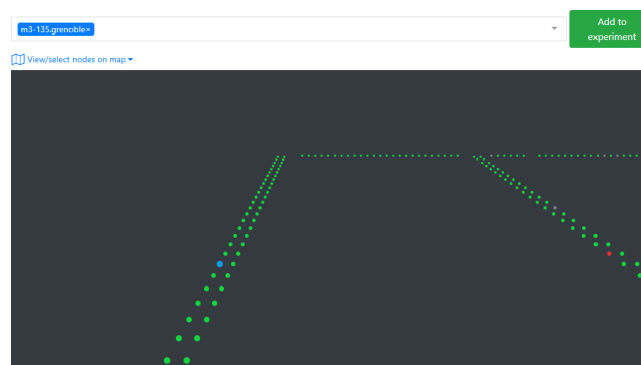
Trong phần kiểm chứng này, đề tài sử dụng các node M3 trên nền tảng FIT/IoT-LAB để nạp lại firmware được biên dịch từ Contiki-NG. Các node được lựa chọn trực tiếp trên giao diện FIT/IoT-LAB theo site, kiến trúc phần cứng và vị trí trên site map. Nhờ đó, ngoài việc biết số lượng node được sử dụng, người thực hiện còn có thể quan sát được cách bố trí vật lý của các node trong testbed trước khi submit experiment. Đây là điểm giúp phần kiểm chứng có tính thực quan hơn so với chỉ nêu danh sách node như trước.

Sau khi chọn node và gán firmware tương ứng, experiment được submit lên hệ thống. Trạng thái triển khai được theo dõi thông qua giao diện quản lý experiment, trong đó các thông tin quan trọng gồm số node được cấp phát, trạng thái running, kết quả deployment và terminal log của từng node. Dựa trên các log này, đề tài kiểm tra xem chương trình có hoạt động đúng theo kịch bản mong muốn hay không. Các node em sẽ chọn là các node Grenoble M3 nên có cùng tọa độ z thế nên chúng ta có thể xem topology của nó như đang nằm trên mặt phẳng 2D.

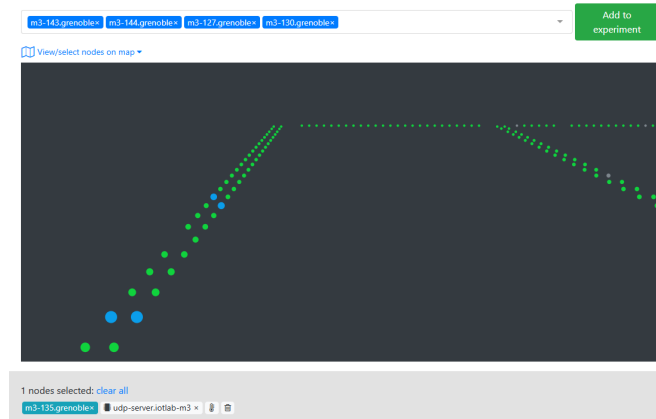
5.4.3 Kiểm chứng kịch bản Unicast

Đối với kịch bản unicast, thí nghiệm được triển khai trên FIT/IoT-LAB với 5 node M3 tại site Grenoble, gồm 1 node server và 4 node client. Các node được chọn trực tiếp trên giao diện host name / map. Trên bản đồ, các chấm xanh biểu thị node đang hoạt động, chấm đỏ là node lỗi, còn các vị trí trống là các node đã được chọn. Nhờ đó, ngoài việc chọn theo ID, người thực hiện còn có thể quan sát được topo vật lý của nhóm node tham gia thí nghiệm.

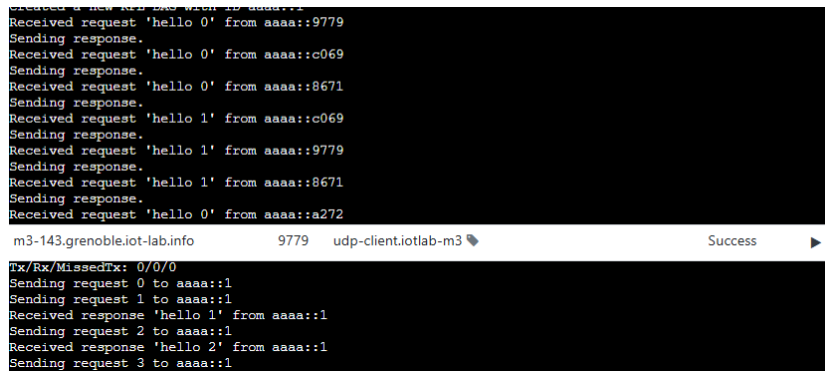
Node m3-135 được chọn làm server, còn các node m3-143, m3-144, m3-127 và m3-130 được chọn làm client. Các node được chọn trên FIT/IoT-LAB gồm m3-135, m3-143, m3-144, m3-127 và m3-130, với tọa độ lần lượt là (0.40, 14.43, -0.04), (0.40, 12.03, -0.04), (1.00, 12.03, -0.04), (0.40, 16.83, -0.04) và (1.00, 16.23, -0.04). Từ đó ước lượng khoảng cách từ node m3-135 đến các node còn lại lần lượt là 2.40m, 2.47m, 2.40m và 1.90m. Sau khi gán firmware và chạy thí nghiệm, terminal log cho thấy server đã nhận được đầy đủ 400 request từ 4 client. Ở chiều phản hồi, hầu hết các gói tin đều được các client nhận thành công. Cụ thể, tổng số response mà các client nhận được là 398 gói, tức chỉ thiếu 2 gói so với số gói phản hồi mong đợi. Kết quả này cho thấy kịch bản unicast đã hoạt động tốt trên FIT/IoT-LAB, tuy nhiên môi trường truyền thông thực tế vẫn có thể làm xuất hiện một số mất gói nhỏ so với mô phỏng.



Hình 5.7: Lựa chọn node server cho kịch bản unicast trên FIT/IoT-LAB



Hình 5.8: Lựa chọn các node client cho kịch bản unicast trên FIT/IoT-LAB



Hình 5.9: Terminal log của server và client trong quá trình kiểm chứng kịch bản unicast trên FIT/IoT-LAB

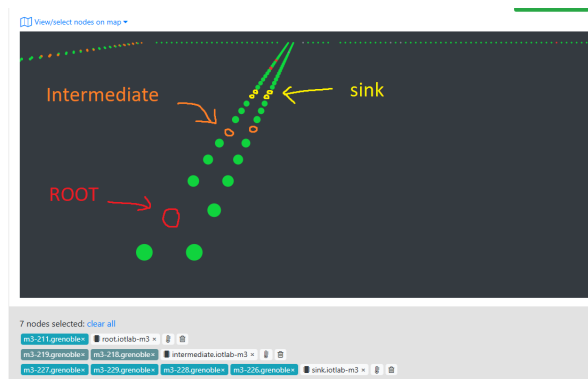
Thông số	Giá trị
Số node trong mạng	5
Số node server/root	1
Số node client	4
Số gói mỗi client gửi	100
Tổng số request client gửi	400
Số gói request server nhận được	400
Số gói response client nhận được	398
Delivery ratio ở chiều request	100%
Packet loss ratio ở chiều request	0%
Delivery ratio end-to-end	99.5%
Packet loss ratio end-to-end	0.5%
Overhead _{send}	400

Bảng 5.6: Kết quả tổng hợp kịch bản unicast trên FIT/IoT-LAB

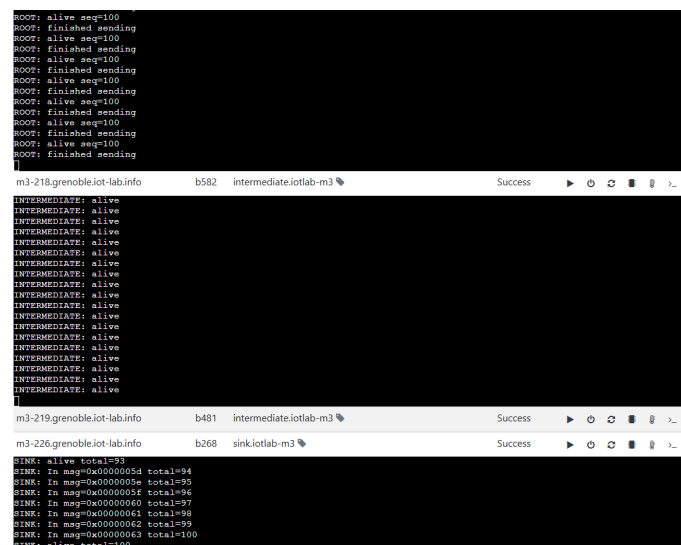
5.4.4 Trickle Multicast

Đối với kịch bản Trickle Multicast, thí nghiệm được triển khai trên FIT/IoT-LAB với 7 node M3 tại site Grenoble, gồm 1 node root, 2 node intermediate và 4 node sink. Các node được chọn trực tiếp trên giao diện host name / map, nhờ đó có thể quan sát được vị trí tương đối của các node trên site map. Trong lần kiểm chứng này, node root được đặt ở vị trí gần nhất, 2 node intermediate ở khu vực giữa, còn 4 node sink được chọn ở xa hơn để tạo đường truyền nhiều chặng. Cách bố trí này phù hợp với mục tiêu kiểm chứng cơ chế multicast có chuyển tiếp qua node trung gian trên testbed thực.

Node m3-211 được chọn làm root, hai node m3-218 và m3-219 đóng vai trò intermediate, còn bốn node m3-226, m3-227, m3-228 và m3-229 được dùng làm sink. Các node được chọn trên FIT/IoT-LAB gồm m3-211, m3-218, m3-219, m3-226, m3-227, m3-228 và m3-229, với tọa độ lần lượt là (18.35, 3.03, -0.04), (18.95, 5.43, -0.04), (18.35, 5.43, -0.04), (18.95, 7.83, -0.04), (18.35, 7.83, -0.04), (18.95, 8.43, -0.04) và (18.35, 8.43, -0.04). Từ đó, khoảng cách từ node m3-211 đến các node còn lại lần lượt là 2.47m, 2.40m, 4.84m, 4.80m, 5.43m và 5.40m. Sau khi gán firmware và chạy thí nghiệm, terminal log cho thấy node root gửi đủ 100 gói, các node intermediate hoạt động ổn định, và các sink nhận được đầy đủ dữ liệu. Trong lần chạy này, không ghi nhận mất gói, tức là cả 4 sink đều nhận đủ 100 gói. Kết quả này cho thấy kịch bản Trickle Multicast hoạt động tốt trên FIT/IoT-LAB và phù hợp với xu hướng đã quan sát trong mô phỏng.



Hình 5.10: Cấu hình 7 node cho kịch bản Trickle Multicast trên FIT/IoT-LAB



Hình 5.11: Terminal log của node root, các node intermediate và các node sink trong quá trình kiểm chứng Trickle Multicast trên FIT/IoT-LAB

Thông số	Giá trị
Số node trong mạng	7
Node nguồn/root	1
Số node intermediate	2
Số node nhận	4
Số gói dữ liệu nguồn phát ra	100
Tổng số gói kỳ vọng tại các node nhận	400
Tổng số gói nhận thành công	400
Delivery ratio	100%
Packet loss ratio	0%
Overhead _{send}	100

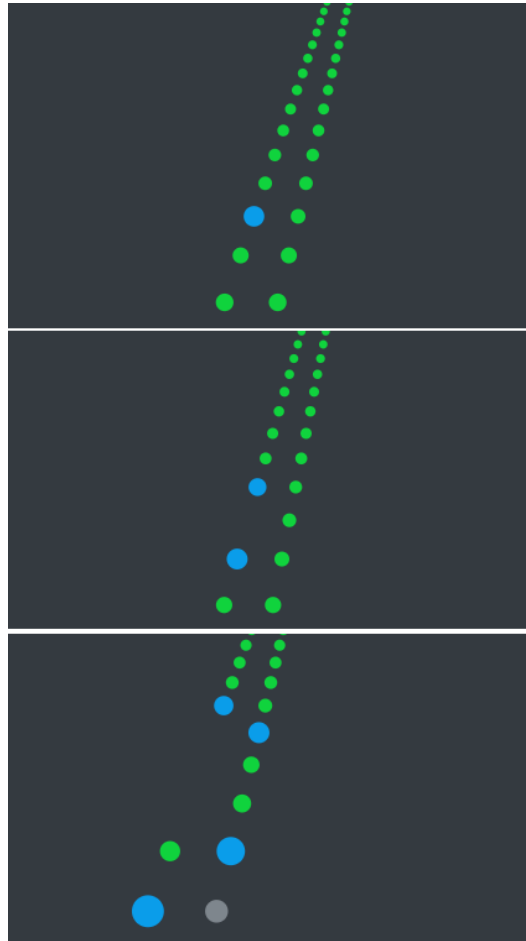
Bảng 5.7: Kết quả tổng hợp kịch bản Trickle Multicast trên FIT/IoT-LAB

5.4.5 Kiểm chứng kịch bản SMRF

Đối với kịch bản SMRF, thí nghiệm được triển khai trên FIT/IoT-LAB với cách chọn node và bố trí topo tương tự kịch bản Trickle Multicast, gồm 1 node root, 2 node intermediate và 4 node sink. Các node được chọn trực tiếp trên giao diện host name / map, trong đó node root được đặt ở vị trí gần, hai node intermediate nằm ở khu vực giữa và các node sink được bố trí ở phía xa hơn để tạo đường truyền nhiều chặng. Tuy nhiên, báo cáo không lặp lại hình chọn topo mà chỉ tập trung trình bày kết quả terminal log của quá trình kiểm chứng.

Trong lần kiểm chứng này, node m3-213 được chọn làm root, hai node m3-215 và m3-211 đóng vai trò intermediate, còn bốn node m3-207, m3-208, m3-214 và m3-217 được dùng làm sink. Các node được chọn trên FIT/IoT-LAB gồm m3-213, m3-215, m3-211, m3-207, m3-208, m3-214 và m3-217, với tọa độ lần lượt là (18.35, 3.63, -0.04), (18.35, 4.23, -0.04), (18.35, 3.03, -0.04), (18.35, 1.83, -0.04), (18.95, 2.43, -0.04), (18.95, 4.23, -0.04) và (18.35, 4.83, -0.04). Từ đó, khoảng cách từ node m3-213 đến các node còn lại lần lượt là 0.60m, 0.60m, 1.80m, 1.34m, 0.85m và 1.20m. Kết quả log cho thấy node root đã gửi đủ 100 gói, tuy nhiên số gói mà các sink nhận được không đồng đều, lần lượt là 85, 82, 83 và 84 gói. Như vậy, tổng số gói nhận thành công tại các node sink là 334/400 gói.

Mặc dù các node đã được bố trí tương đối gần nhau trên site map, kết quả kiểm chứng vẫn cho thấy hiện tượng mất gói khá rõ. Điều này cho thấy với kịch bản SMRF, chất lượng truyền dữ liệu trên FIT/IoT-LAB không chỉ phụ thuộc vào khoảng cách topo mà còn chịu ảnh hưởng của liên kết vô tuyến thực tế và quá trình chuyển tiếp qua các node intermediate. So với kịch bản unicast đã kiểm chứng trước đó, SMRF trong lần chạy này cho kết quả kém ổn định hơn. Tuy nhiên, nhận xét này chỉ áp dụng cho lần kiểm chứng thực nghiệm hiện tại, không nên khái quát tuyệt đối rằng SMRF luôn kém hơn unicast trong mọi trường hợp. Trong phần lý thuyết của báo cáo, multicast trong 6LoWPAN được xây dựng để giảm số lần phát lại và tiết kiệm tài nguyên mạng, nhưng trong môi trường LLN thực tế, các liên kết không ổn định vẫn có thể làm giảm delivery ratio.



Hình 5.12: Cấu hình node root, intermediate và sink cho kịch bản SMRF trên FIT/IoT-LAB

Thông số	Giá trị
Số node trong mạng	7
Node nguồn/root	1
Số node intermediate	2
Số node nhận	4
Số gói dữ liệu nguồn phát ra	100
Tổng số gói kỳ vọng tại các node nhận	400
Tổng số gói nhận thành công	334
Delivery ratio	83.5%
Packet loss ratio	16.5%
Overhead _{send}	100

Bảng 5.8: Kết quả tổng hợp kịch bản SMRF trên FIT/IoT-LAB

5.4.6 Kiểm chứng kịch bản EMRF

Trong kịch bản EMRF, em tiến hành kiểm chứng trên nền tảng FIT/IoT-LAB với cách bố trí topology tương tự kịch bản SMRF, gồm một node root, một node intermediate và các node sink được đặt theo mô hình truyền đa chặng. Kết quả terminal log cho thấy node gốc có thể phát gói multicast, trong khi node sink khởi động bình thường, được gán địa chỉ IPv6, tham gia nhóm

multicast và mở cổng UDP để lắng nghe dữ liệu. Tuy nhiên, trong quá trình thử nghiệm, em chưa ghi nhận được gói tin multicast được nhận thành công tại node sink, số lượng gói nhận vẫn bằng 0. Nguyên nhân có thể xuất phát từ sự khác biệt giữa môi trường mô phỏng và môi trường phần cứng thực, đặc biệt ở quá trình hình thành tuyến multicast, cơ chế chuyển tiếp tại node intermediate hoặc khả năng tương thích của EMRF với cấu hình Contiki/FIT đang sử dụng. Vì vậy, kịch bản EMRF chưa được kiểm chứng thành công trên FIT/IoT-LAB ở mức truyền nhận dữ liệu đầu cuối, và phần đánh giá EMRF trong đồ án chủ yếu dựa trên kết quả mô phỏng Cooja.

```

m3-95.grenoble.iot-lab.info a770 root.iotlab-m3 Success
ROOT: Send to ff1e::89:abcd remote port 3001 msg=0x00000038 bytes=4
ROOT: Send to ff1e::89:abcd remote port 3001 msg=0x00000039 bytes=4
ROOT: Send to ff1e::89:abcd remote port 3001 msg=0x0000003a bytes=4
ROOT: Send to ff1e::89:abcd remote port 3001 msg=0x0000003b bytes=4
ROOT: alive, seq=60
ROOT: Send to ff1e::89:abcd remote port 3001 msg=0x0000003c bytes=4
ROOT: Send to ff1e::89:abcd remote port 3001 msg=0x0000003d bytes=4
ROOT: Send to ff1e::89:abcd remote port 3001 msg=0x0000003e bytes=4
ROOT: Send to ff1e::89:abcd remote port 3001 msg=0x0000003f bytes=4
ROOT: Send to ff1e::89:abcd remote port 3001 msg=0x00000040 bytes=4
ROOT: alive, seq=65
ROOT: Send to ff1e::89:abcd remote port 3001 msg=0x00000041 bytes=4
ROOT: Send to ff1e::89:abcd remote port 3001 msg=0x00000042 bytes=4
ROOT: Send to ff1e::89:abcd remote port 3001 msg=0x00000043 bytes=4
ROOT: Send to ff1e::89:abcd remote port 3001 msg=0x00000044 bytes=4
ROOT: Send to ff1e::89:abcd remote port 3001 msg=0x00000045 bytes=4
ROOT: alive, seq=70
ROOT: Send to ff1e::89:abcd remote port 3001 msg=0x00000046 bytes=4

m3-101.grenoble.iot-lab.info 9181 intermediate.iotlab-m3 Success
INTERMEDIATE: alive
INTERMEDIATE: alive
INTERMEDIATE: alive
INTERMEDIATE: alive
INTERMEDIATE: alive
INTERMEDIATE: alive
INTERMEDIATE: alive
INTERMEDIATE: alive

m3-102.grenoble.iot-lab.info a881 intermediate.iotlab-m3 Success

m3-103.grenoble.iot-lab.info 9881 sink.iotlab-m3 Success
SINK: alive total=0
    
```

Hình 5.13: Terminal log trong quá trình thử kiểm chứng kịch bản EMRF trên FIT/IoT-LAB

5.4.7 Nhận xét chung

Qua quá trình kiểm chứng trên FIT/IoT-LAB, em nhận thấy các kịch bản unicast, Trickle Multicast và SMRF đều có thể triển khai trên testbed thực và quan sát được quá trình truyền nhận dữ liệu giữa các node. Điều này cho thấy mô hình mạng, cách lựa chọn node và quy trình nạp chương trình là phù hợp để kiểm chứng thực nghiệm các cơ chế truyền dữ liệu trong mạng 6LoWPAN. Tuy nhiên, kết quả trên testbed thực vẫn có sự khác biệt so với môi trường mô phỏng Cooja do còn chịu ảnh hưởng của quá trình hình thành tuyến, trạng thái kết nối giữa các node và điều kiện hoạt động của phần cứng.

Đối với kịch bản EMRF, em đã triển khai với topology tương tự SMRF và theo dõi terminal log trong quá trình hoạt động của các node. Kết quả cho thấy chương trình có thể khởi động trên testbed, node gốc phát gói multicast và node sink có thể tham gia nhóm multicast, nhưng chưa ghi nhận được dữ liệu được nhận thành công tại node đích. Vì vậy, có thể nhận xét rằng FIT/IoT-LAB phù hợp để kiểm chứng một số kịch bản multicast cơ bản, còn với EMRF vẫn cần tiếp tục nghiên cứu và hoàn thiện thêm để có thể triển khai ổn định trong môi trường thực tế.

5.5 Đánh giá chung

Qua các kết quả mô phỏng và kiểm chứng trong phạm vi đề tài, có thể thấy multicast là hướng truyền dữ liệu phù hợp hơn unicast khi cần gửi cùng một nội dung đến nhiều node trong mạng 6LoWPAN. So với unicast, các cơ chế multicast giúp giảm số lượng gói tin phát ra từ node gốc, từ đó làm giảm overhead truyền thông và sử dụng tài nguyên mạng hiệu quả hơn. Đây là điểm phù hợp với mục tiêu đánh giá của đề tài, vốn tập trung vào độ trễ, tỉ lệ nhận gói, tỉ lệ mất gói và overhead truyền thông.

Trong các kịch bản đã khảo sát, Trickle Multicast cho thấy khả năng hoạt động ổn định trong mô hình mạng đơn giản, dễ triển khai và cho kết quả tốt khi topology chưa quá phức tạp. SMRF cũng cho thấy hiệu quả trong cơ chế truyền multicast dựa trên RPL, đồng thời thể hiện được ưu điểm khi truyền dữ liệu theo hướng từ node gốc đến nhiều node nhận trong mạng nhiều chặng. Trong khi đó, EMRF cho thấy tiềm năng về mặt thuật toán trong môi trường mô phỏng, tuy nhiên kết quả kiểm chứng thực tế chưa ổn định như mong muốn. Bên cạnh đó, kết quả cũng cho thấy hiệu quả của từng cơ chế không chỉ phụ thuộc vào bản thân thuật toán mà còn chịu ảnh hưởng bởi topology mạng, khoảng cách giữa các node, chất lượng liên kết và quá trình hình thành tuyến trong mạng. Vì vậy, cùng là multicast nhưng mỗi cơ chế sẽ có đặc điểm riêng về độ trễ, độ tin cậy truyền dữ liệu và mức overhead tạo ra trong từng điều kiện thử nghiệm cụ thể.

Đối với phần kiểm chứng trên FIT/IoT-LAB, các kịch bản unicast, Trickle Multicast và SMRF đều cho thấy có thể triển khai và quan sát được hoạt động truyền nhận dữ liệu trên testbed thực. Riêng với EMRF, nhóm mới chỉ kiểm chứng được ở mức khởi động chương trình, tham gia nhóm multicast và phát gói từ node gốc, nhưng chưa ghi nhận được dữ liệu nhận thành công tại node đích. Do đó, phần đánh giá EMRF trong đồ án chủ yếu vẫn nên dựa trên kết quả mô phỏng Cooja, còn kết quả trên testbed thực được xem là bước thử nghiệm triển khai ban đầu. Tổng thể, đề tài đã cho thấy multicast có nhiều ưu điểm trong mạng 6LoWPAN khi so với unicast trong bài toán truyền dữ liệu đến nhiều node. Đồng thời, kết quả đạt được cũng cho thấy việc lựa chọn cơ chế multicast phù hợp cần gắn với điều kiện mạng cụ thể và mục tiêu triển khai thực tế của hệ thống.

Chương 6

KẾT LUẬN VÀ HƯỚNG PHÁT TRIỂN

6.1 Kết luận

Trong đồ án này, em đã nghiên cứu cơ sở lý thuyết liên quan đến mạng 6LoWPAN, IPv6, UDP, multicast IPv6 và các cơ chế multicast trong mạng IoT công suất thấp. Trên cơ sở đó, em đã xây dựng mô hình mô phỏng trên nền tảng Contiki-NG/Cooja, thiết kế các kịch bản truyền dữ liệu gồm unicast, Trickle Multicast, SMRF và EMRF để đánh giá hiệu quả của multicast trong mạng 6LoWPAN. Việc đánh giá được thực hiện dựa trên các chỉ tiêu chính gồm độ trễ truyền dữ liệu, tỉ lệ nhận gói, tỉ lệ mất gói và overhead truyền thông.

Kết quả mô phỏng cho thấy multicast là cơ chế phù hợp hơn unicast khi cần truyền cùng một nội dung đến nhiều node trong mạng. So với unicast, các cơ chế multicast giúp giảm số lượng gói phát ra từ node gốc, qua đó giảm overhead truyền thông và sử dụng tài nguyên mạng hiệu quả hơn. Trong các cơ chế đã khảo sát, Trickle Multicast cho thấy khả năng hoạt động ổn định trong mô hình mạng đơn giản, trong khi SMRF thể hiện được hiệu quả trong mô hình multicast dựa trên RPL. Đối với EMRF, kết quả mô phỏng cho thấy cơ chế này có tiềm năng về mặt thuật toán, tuy nhiên việc triển khai và kiểm chứng thực tế vẫn còn gặp khó khăn.

Bên cạnh phần mô phỏng, đồ án cũng đã thực hiện kiểm chứng trên nền tảng FIT/IoT-LAB. Kết quả cho thấy các kịch bản unicast, Trickle Multicast và SMRF đều có thể triển khai và quan sát được hoạt động truyền nhận dữ liệu trên testbed thực. Riêng với EMRF, nhóm mới chỉ kiểm chứng được ở mức khởi động chương trình, tham gia nhóm multicast và phát gói từ node gốc, nhưng chưa ghi nhận được dữ liệu nhận thành công tại node đích. Điều này cho thấy giữa môi trường mô phỏng và môi trường phần cứng thực vẫn tồn tại những khác biệt nhất định, đặc biệt với các cơ chế multicast phức tạp hơn. Nhìn chung, đồ án đã đạt được mục tiêu chính là xây dựng mô hình, thực hiện mô phỏng, phân tích kết quả và bước đầu kiểm chứng khả năng triển khai multicast trong mạng 6LoWPAN. Các kết quả thu được là cơ sở để khẳng định vai trò của multicast trong việc nâng cao hiệu quả truyền dữ liệu đến nhiều node trong mạng IoT công suất thấp.

6.2 Hạn chế

Mặc dù đã đạt được các kết quả nhất định, đồ án vẫn còn một số hạn chế. Thứ nhất, quy mô mạng được khảo sát còn tương đối nhỏ nên chưa phản ánh đầy đủ khả năng hoạt động của các cơ chế multicast khi số lượng node tăng lên. Thứ hai, các chỉ tiêu đánh giá mới tập trung vào độ trễ, tỉ lệ nhận gói, tỉ lệ mất gói và overhead truyền thông, trong khi một số yếu tố khác như mức tiêu thụ năng lượng thực tế hay độ ổn định lâu dài của mạng chưa được phân tích sâu.

Bên cạnh đó, kết quả giữa môi trường mô phỏng và môi trường phần cứng thực vẫn còn tồn tại khác biệt. Một số kịch bản có thể hoạt động tốt trong Cooja nhưng khi triển khai trên FIT/IoT-

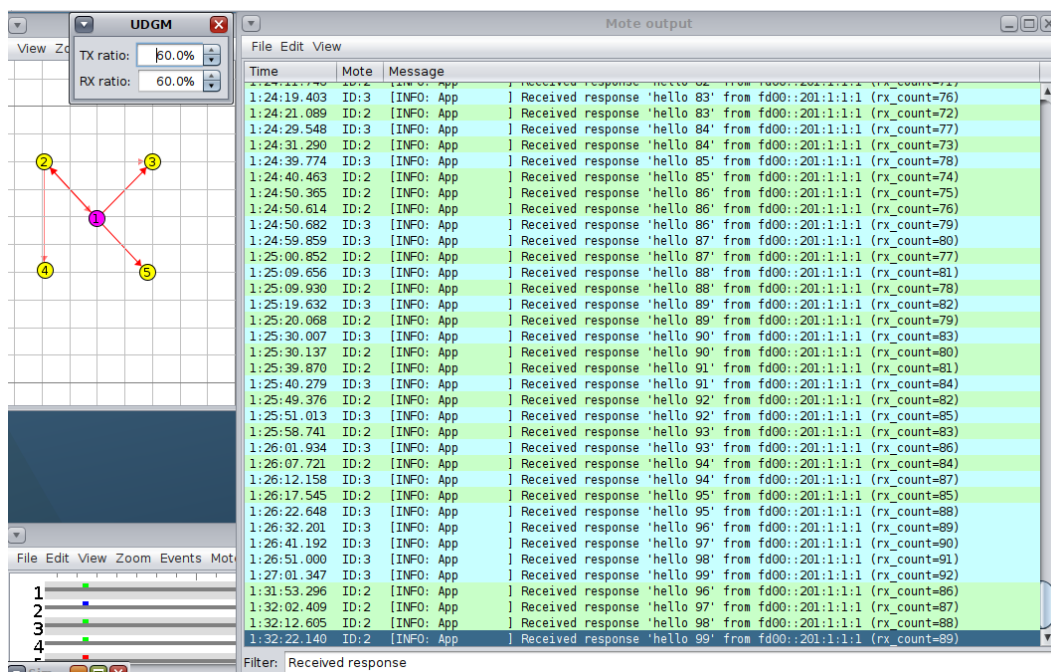
LAB lại phát sinh khó khăn do ảnh hưởng của phần cứng, trạng thái kết nối giữa các node và quá trình hình thành tuyến trong mạng. Đặc biệt, với kịch bản EMRF, nhóm mới chỉ kiểm chứng được ở mức khởi động chương trình, tham gia nhóm multicast và cấu hình chức năng cơ bản, nhưng chưa ghi nhận được dữ liệu được nhận thành công tại node đích. Vì vậy, phần đánh giá EMRF trên testbed thực vẫn còn hạn chế và chưa thể đối chiếu đầy đủ với kết quả mô phỏng.

Bên cạnh đó, kết quả mô phỏng trong Cooja và kết quả kiểm chứng trên FIT/IoT-LAB vẫn có sự khác biệt nhất định. Một số kịch bản có thể hoạt động tốt trong môi trường mô phỏng nhưng khi triển khai trên testbed thực lại phát sinh khó khăn do ảnh hưởng của phần cứng, trạng thái kết nối giữa các node và quá trình hình thành tuyến trong mạng. Đặc biệt, với kịch bản EMRF, nhóm mới chỉ kiểm chứng được ở mức khởi động chương trình và cấu hình cơ bản, chưa ghi nhận được quá trình truyền nhận dữ liệu thành công tại node đích. Vì vậy, phần đánh giá EMRF trên testbed thực vẫn còn hạn chế và chưa thể đối chiếu đầy đủ với kết quả mô phỏng.

6.3 Hướng phát triển

Kết quả ở Chương 5 cho thấy các kịch bản được khảo sát nhìn chung hoạt động ổn định trong điều kiện mô phỏng hiện tại. Ở kịch bản unicast, hệ thống đạt delivery ratio 100%, packet loss ratio 0% và RTT trung bình khoảng 49.71ms. Đối với multicast, Trickle Multicast cũng cho kết quả tốt về độ phủ gói tin, độ trễ và overhead truyền thông. Điều này cho thấy mô hình đã đáp ứng được mục tiêu đánh giá của đề tài và có thể dùng làm cơ sở cho các khảo sát mở rộng trong những điều kiện khó hơn. Tuy nhiên, do môi trường mô phỏng hiện tại vẫn tương đối thuận lợi nên sự khác biệt về khả năng chịu lỗi giữa unicast và multicast chưa được thể hiện thật rõ.

Vì vậy, một hướng phát triển tiếp theo của đề tài là mở rộng mô phỏng sang điều kiện truyền dẫn có mất gói bằng cách giảm success ratio của radio medium, nhưng vẫn giữ nguyên topology để bảo đảm tính nhất quán khi so sánh. Cách tiếp cận này giúp đánh giá rõ hơn ảnh hưởng của chất lượng liên kết đến delivery ratio, packet loss ratio và overhead truyền thông của từng cơ chế. Trong phạm vi mở rộng này, Trickle Multicast được chọn làm cơ chế đại diện cho nhóm multicast nhằm tập trung so sánh trực tiếp với unicast trong cùng điều kiện, qua đó làm rõ hơn ưu thế của multicast khi mạng hoạt động trong môi trường bất lợi.



Hình 6.1: Cấu hình mô phỏng unicast với radio medium đặt TX ratio = 60% và RX ratio = 60%

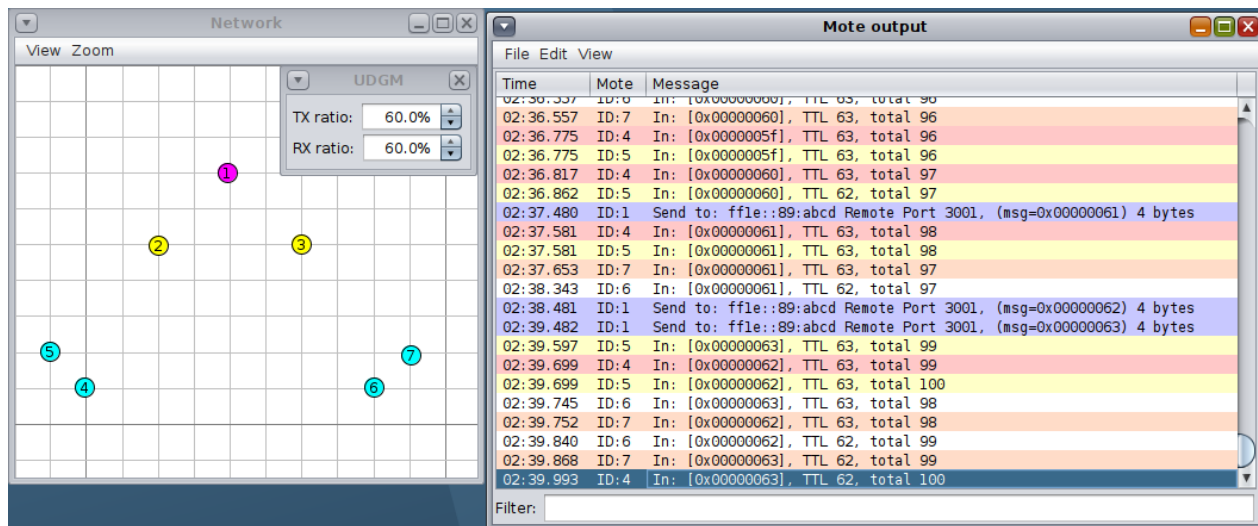
Đối với kịch bản unicast, trong thử nghiệm mở rộng ban đầu, radio medium được cấu hình theo hướng giảm TX success ratio và RX success ratio xuống 60% nhằm tạo môi trường truyền dẫn kém ổn định hơn so với kịch bản gốc. Với cấu hình này, tổng số request kỳ vọng vẫn là 400 gói do có 4 client và mỗi client gửi 100 request. Kết quả thử nghiệm cho thấy server chỉ nhận được 382/400 request, tương ứng delivery ratio ở chiều request đạt 95.5% và packet loss ratio là 4.5%. Đồng thời, tổng số response mà các client nhận được là 379, tương ứng delivery ratio end-to-end đạt 94.75% và packet loss ratio là 5.25%. Kết quả này cho thấy khi điều kiện truyền dẫn xấu đi, cơ chế unicast UDP không còn duy trì được mức truyền thành công tuyệt đối như trong điều kiện lý tưởng; mất gói có thể xuất hiện ở cả chiều client gửi request đến server và chiều server phản hồi về client.

Thông số	Giá trị
Số node trong mạng	5
Node server/root	ID:1
Số node client	4 (ID:2-ID:5)
Số gói mỗi client gửi	100
Tổng số gói request gửi	400
Tổng số gói request server nhận được	382
Tổng số gói response client nhận được	379
Delivery ratio ở chiều request	95.5%
Packet loss ratio ở chiều request	4.5%
Delivery ratio end-to-end	94.75%
Packet loss ratio end-to-end	5.25%
Overhead _{send}	400

Bảng 6.1: Kết quả thử nghiệm mở rộng cho kịch bản unicast khi tăng loss trong radio medium

Từ kết quả trên có thể thấy rằng việc tăng loss rate trong radio medium là cần thiết để đánh giá thực tế hơn độ bền của cơ chế unicast. Trong điều kiện mô phỏng thuận lợi ở Chương 5, unicast cho độ tin cậy cao và độ trễ thấp. Tuy nhiên, khi mức mất gói tăng lên, delivery ratio của unicast đã suy giảm rõ rệt. Điều này cho thấy hiệu năng của cơ chế unicast phụ thuộc đáng kể vào chất lượng liên kết vô tuyến, đặc biệt khi quá trình truyền dữ liệu diễn ra theo mô hình request/response.

Đối với multicast, đề tài lựa chọn Trickle Multicast làm trường hợp đại diện để tiếp tục khảo sát trong cùng điều kiện suy giảm kênh truyền. Sau khi đã thực hiện thử nghiệm mở rộng với unicast, cùng một mức loss trong radio medium được áp dụng cho Trickle Multicast, tức là vẫn giữ nguyên topology và tiếp tục sử dụng cấu hình TX success ratio = 60% và RX success ratio = 60%. Việc giữ nguyên các điều kiện mô phỏng này là cần thiết để bảo đảm tính công bằng khi so sánh giữa unicast và multicast.



Hình 6.2: Cấu hình mô phỏng Trickle Multicast trong điều kiện tăng loss với TX ratio = 60% và RX ratio = 60%

Kết quả mô phỏng cho thấy node nguồn phát ra 100 gói dữ liệu, trong khi 4 node nhận đạt tổng cộng 398/400 gói kỳ vọng, tương ứng delivery ratio đạt 99.5% và packet loss ratio là 0.5%. Cụ thể, hai node ID:4 và ID:5 nhận đủ 100 gói, trong khi ID:6 và ID:7 nhận được 99 gói. Kết quả này cho thấy trong cùng điều kiện tăng loss trong radio medium, Trickle Multicast vẫn duy trì được độ phủ gói tin rất cao.

Thông số	Giá trị
Số node trong mạng	7
Node nguồn/root	ID:1
Số node nhận	4 (ID:4–ID:7)
Số gói dữ liệu nguồn phát ra	100
Tổng số gói kỳ vọng tại các node nhận	400
Tổng số gói nhận thành công	398
Delivery ratio	99.5%
Packet loss ratio	0.5%
Overhead _{send}	100

Bảng 6.2: Kết quả thử nghiệm mở rộng cho kịch bản Trickle Multicast khi tăng loss trong radio medium

So với unicast trong cùng điều kiện mô phỏng, multicast đại diện bởi Trickle Multicast cho thấy độ tin cậy cao hơn rõ rệt. Trong khi unicast chỉ đạt delivery ratio end-to-end 94.75%, thì Trickle Multicast đạt 99.5%, đồng thời overhead gửi từ node nguồn chỉ là 100, thấp hơn đáng kể so với 400 ở unicast. Điều này cho thấy trong điều kiện mạng có mất gói, Trickle Multicast không chỉ duy trì độ phủ tốt hơn mà còn tiết kiệm chi phí truyền thông hơn so với cơ chế unicast request/response.

Tóm lại, hướng phát triển này nhằm mở rộng phạm vi đánh giá của đề tài từ điều kiện mô phỏng thuận lợi sang điều kiện có mất gói, qua đó phản ánh thực tế hơn chất lượng hoạt động của hai cơ chế truyền dữ liệu là unicast và multicast trong mạng 6LoWPAN. Kết quả thử nghiệm mở rộng cho thấy khi tăng loss trong radio medium, cơ chế unicast bắt đầu suy giảm rõ rệt

về delivery ratio, trong khi Trickle Multicast vẫn duy trì được độ phủ gói tin rất cao. Cụ thể, unicast đạt delivery ratio end-to-end 94.75%, trong khi Trickle Multicast đạt 99.5%. Bên cạnh đó, overhead gửi từ node nguồn của Trickle Multicast cũng thấp hơn nhiều so với unicast. Kết quả này cho thấy trong cùng điều kiện mạng bất lợi, multicast đại diện bởi Trickle Multicast có ưu thế hơn về cả độ tin cậy lẫn hiệu quả truyền thông, từ đó làm rõ hơn giá trị thực tiễn của cơ chế multicast trong mạng 6LoWPAN.

6.4 Kết thúc

Đồ án Multicast trong mạng 6LoWPAN đã hoàn thành các nội dung chính theo mục tiêu đề ra. Trong quá trình thực hiện, em đã lần lượt tìm hiểu cơ sở lý thuyết, xây dựng mô hình mô phỏng, thiết kế các kịch bản truyền dữ liệu, thu thập số liệu và phân tích kết quả đối với các cơ chế unicast, Trickle Multicast, SMRF và EMRF. Bên cạnh đó, đề tài cũng đã thực hiện bước kiểm chứng trên FIT/IoT-LAB nhằm đối chiếu giữa môi trường mô phỏng và môi trường triển khai thực tế, qua đó giúp nội dung nghiên cứu có tính thực tiễn hơn.

Những kết quả đạt được trong đồ án đã góp phần làm rõ đặc điểm và hiệu quả của multicast trong mạng 6LoWPAN, đặc biệt trong bài toán truyền cùng một nội dung đến nhiều node trong mạng IoT công suất thấp. Mặc dù vẫn còn một số hạn chế nhất định, quá trình thực hiện đề tài đã giúp em củng cố kiến thức về mạng IoT, 6LoWPAN, RPL và các cơ chế multicast, đồng thời rèn luyện thêm kỹ năng mô phỏng, phân tích số liệu và tiếp cận hệ thống testbed thực. Đây là cơ sở quan trọng để em tiếp tục hoàn thiện kiến thức và phát triển các nghiên cứu liên quan trong thời gian tới.

Tài liệu tham khảo

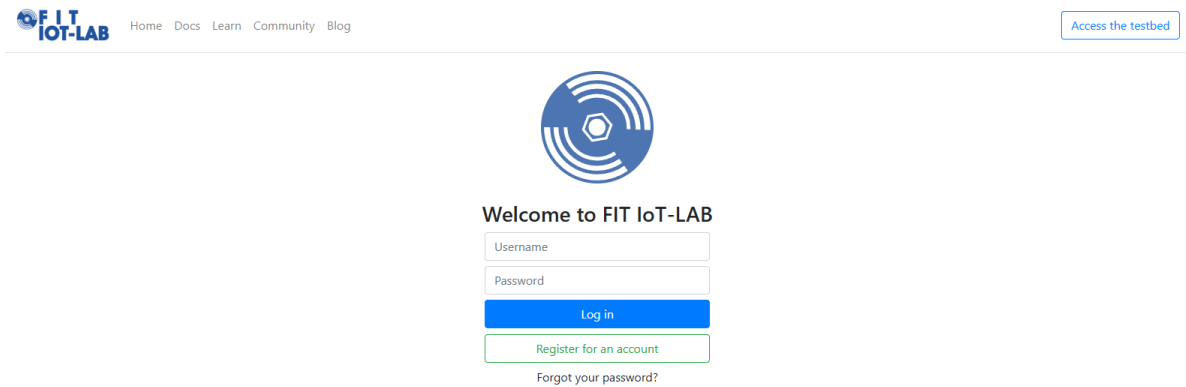
- [1] D. Gil, A. Ferrández, H. Mora-Mora, and J. Peral, “Internet of Things: A Review of Surveys Based on Context Aware Intelligent Services,” *Sensors*, 2016.
- [2] C. Bormann, M. Ersue, and A. Keränen, “Terminology for Constrained-Node Networks,” RFC 7228, IETF, 2014.
- [3] T. Winter et al., “RPL: IPv6 Routing Protocol for Low-Power and Lossy Networks,” RFC 6550, IETF, 2012.
- [4] J.-P. Vasseur et al., “Terms Used in Routing for Low-Power and Lossy Networks,” RFC 7102, IETF, 2014.
- [5] E. Kim, D. Kaspar, C. Gomez, and C. Bormann, “Problem Statement and Requirements for IPv6 over Low-Power Wireless Personal Area Network (6LoWPAN) Routing,” RFC 6606, IETF, 2012.
- [6] G. Montenegro, N. Kushalnagar, J. Hui, and D. Culler, “Transmission of IPv6 Packets over IEEE 802.15.4 Networks,” RFC 4944, IETF, 2007.
- [7] J. Hui and P. Thubert, “Compression Format for IPv6 Datagrams over IEEE 802.15.4-Based Networks,” RFC 6282, IETF, 2011.
- [8] S. Deering and R. Hinden, “Internet Protocol, Version 6 (IPv6) Specification,” RFC 8200, IETF, 2017.
- [9] R. Hinden and S. Deering, “IP Version 6 Addressing Architecture,” RFC 4291, IETF, 2006.
- [10] S. Thomson, T. Narten, and T. Jinmei, “IPv6 Stateless Address Autoconfiguration,” RFC 4862, IETF, 2007.
- [11] J. Postel, “User Datagram Protocol,” RFC 768, IETF, 1980.
- [12] R. Vida and L. Costa, “Multicast Listener Discovery Version 2 (MLDv2) for IPv6,” RFC 3810, IETF, 2004.
- [13] J. Hui and R. Kelsey, “Multicast Protocol for Low-Power and Lossy Networks (MPL),” RFC 7731, IETF, 2016.
- [14] Contiki-NG, “IPv6 multicast,” Contiki-NG Official Documentation.
- [15] Contiki-NG, “Running Contiki-NG in Cooja,” Contiki-NG Official Documentation.

PHỤ LỤC

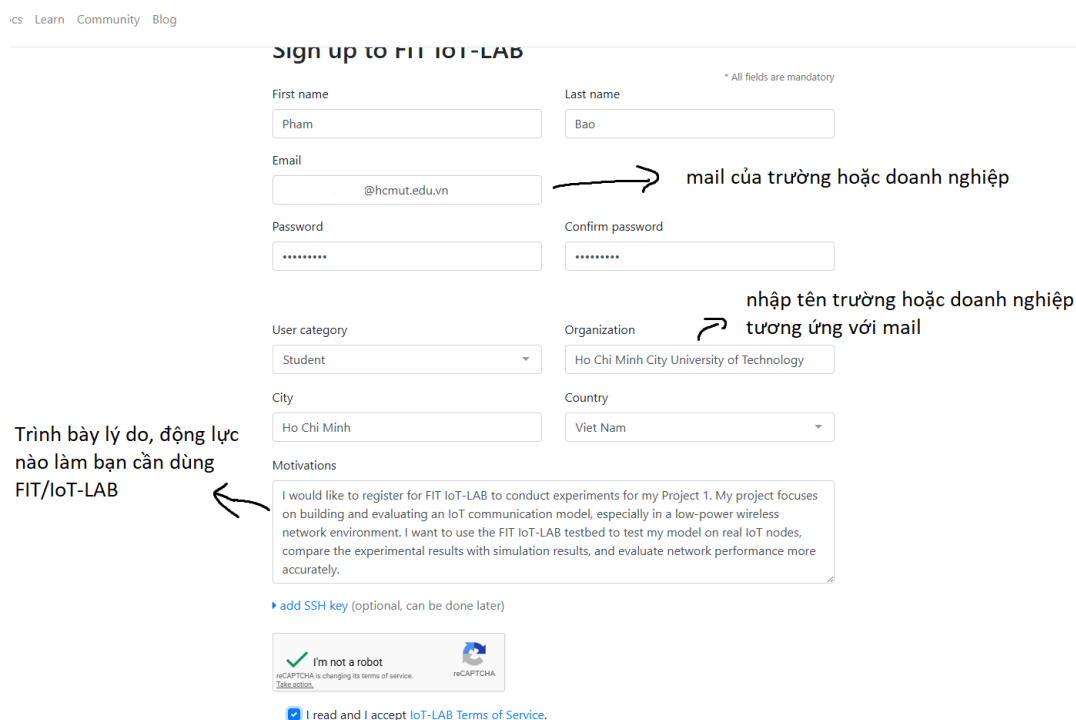
Hướng dẫn triển khai thực nghiệm trên FIT/IoT-LAB

1. Đăng nhập và đăng kí tài khoản trên FIT/IoT-LAB

Đầu tiên, người dùng truy cập vào trang đăng nhập của FIT/IoT-LAB theo đường dẫn:
<https://www.iot-lab.info/testbed/login>



Sau khi truy cập vào link, nếu có tài khoản thì đăng nhập với Username là mail của trường Password là mật khẩu đã đặt. Nếu chưa có tài khoản thì nhấn vào Register for an account.



Trình bày lý do, động lực
nào làm bạn cần dùng
FIT/IoT-LAB

Nhập thông tin theo hướng dẫn trên rồi nhấn Sign up

Sign up to FIT IoT-LAB

✓ An email has been sent to @hcmut.edu.vn

Check your inbox and follow the link to activate your account.

Sau khi đã thành công thì sẽ xuất hiện màn hình như này lúc này bạn chỉ cần vô gmail nhấn mở tất cả thư đã đến tìm mail của FIT/IoT-LAB rồi xác nhận tài khoản.

2. Tạo thí nghiệm trên FIT/IoT-LAB

Khi đăng nhập vào thì ở màn hình chính của FIT/IoT-LAB nó sẽ như thế này

0 Running 0 Scheduled 57 Completed

New experiment

Recent

Id	Name	Date	Duration	Nodes	State
436287		2026-04-15 12:47	20 minutes	7	Terminated
436286		2026-04-15 12:37	9 minutes	7	Stopped
436285		2026-04-15 12:04	35 minutes	7	Stopped
436284		2026-04-15 12:03	32 minutes	7	Stopped
436283		2026-04-15 11:27	35 minutes	7	Terminated

Load more...

Để tạo thí nghiệm thì ta bấm vào New experiment

Schedule 20 min, as soon as possible

Name New experiment

Duration 20 minutes hours

Start ☒ As soon as possible ☐ Scheduled

(nên để As soon as possible nếu cần làm ngay lập tức)

Nodes 0

Summary

Your experiment on 0 nodes is set to start as soon as possible for 20 minutes.

Submit experiment

Sau đó làm theo hướng dẫn trên hình

3. Lựa chọn node và cấu hình testbed

Để thêm node thì ta bấm vào Nodes

Schedule 20 min, as soon as possible

Nodes 0

Select by: node properties | **host name / map** | node id

Select an architecture, site and quantity.

Architecture Site + Add to experiment

Summary

Your experiment on 0 nodes is set to start as soon as possible for 20 minutes.

Submit experiment

Ở mục node properties thì bạn có thể tạo 1 lúc nhiều node máy sẽ tự chọn. Còn ở node id thì bạn phải add từng node 1 thủ công. Cuối cùng là host name / map thì ngoài việc bạn có thể add node và load firmware thì bạn còn có thể lựa chọn vị trí các node trên topology. Architecture và Site thì bạn sẽ chọn tương ứng với mô hình mạng bạn đang làm. Ví dụ: Với mô hình 6LoWPAN Chọn **Architecture** là **M3 (AT86RF231)** vì node M3 hỗ trợ truyền thông không dây IEEE 802.15.4, phù hợp để triển khai mô hình IoT/6LoWPAN.

Chọn **Site** là **Grenoble** vì site này có nhiều node M3, thuận tiện cho việc chọn số lượng node và bố trí thí nghiệm.

4. Biên dịch firmware cho node

Để biên dịch từ bên contiki-NG qua firmware cần thiết của FIT/IoT-LAB đầu tiên ta mở terminal lên và gõ

```
git clone https://github.com/iot-lab/iot-lab.git
```

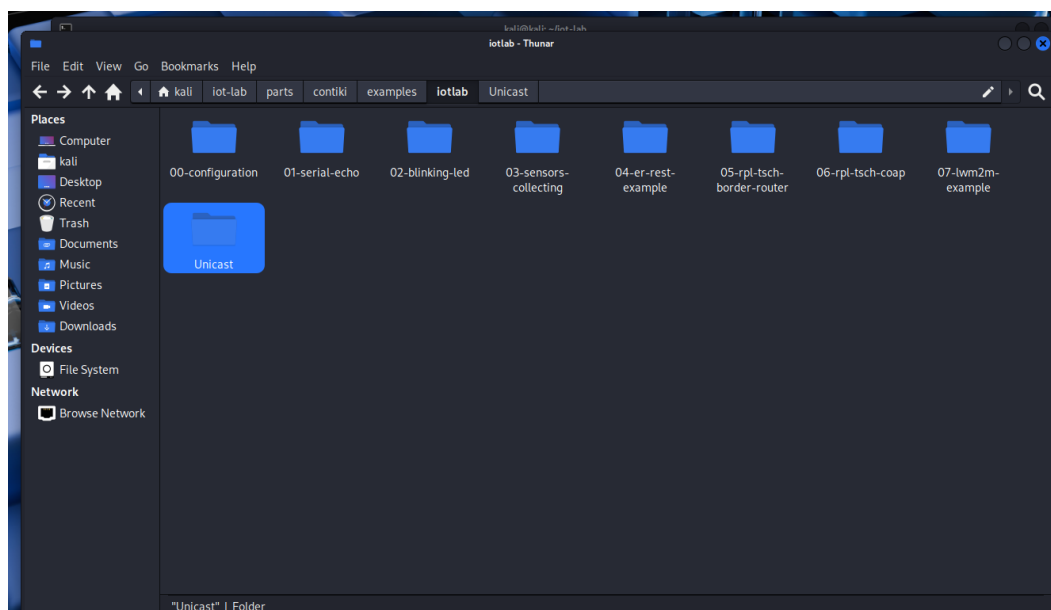
Sau khi đã tạo được thư mục iot-lab thành công thì ta truy cập vào và cài đặt contiki bằng lệnh

```
cd iot-lab
```

```
make
```

```
make setup-contiki
```

Sau đó thì tạo 1 thư mục mới trong đường dẫn /home/kali/iot-lab/parts/contiki/examples/iotlab/ như hình dưới, như thư mục của mình là Unicast

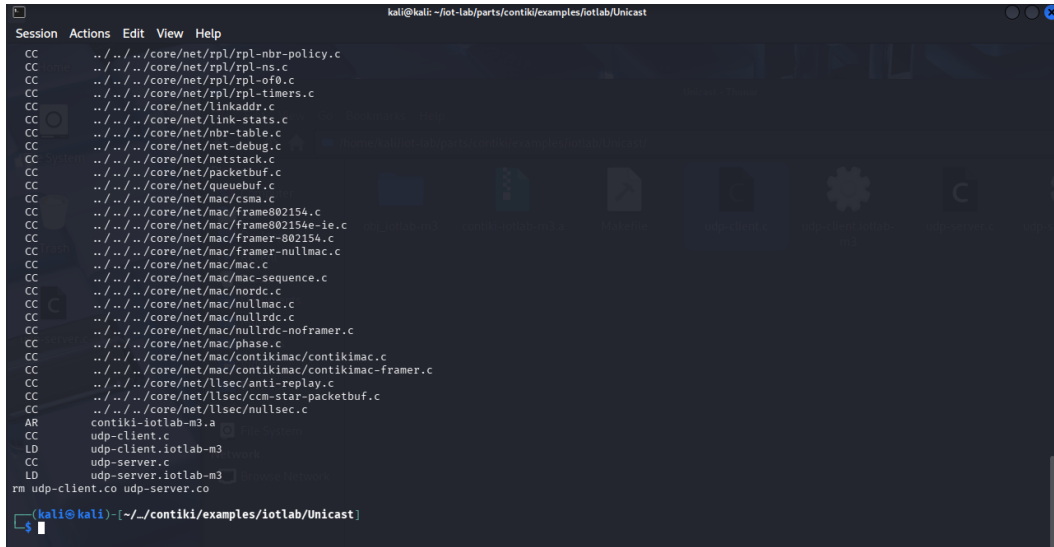


Rồi tiếp tục gõ

```
cd /home/kali/iot-lab/parts/contiki/examples/iotlab/Unicast/
```

Sau đó bạn bỏ file cần thiết để compile vào, sửa code các file từ bên contiki sao cho phù hợp để compile. Sau đó ta compile bằng lệnh:

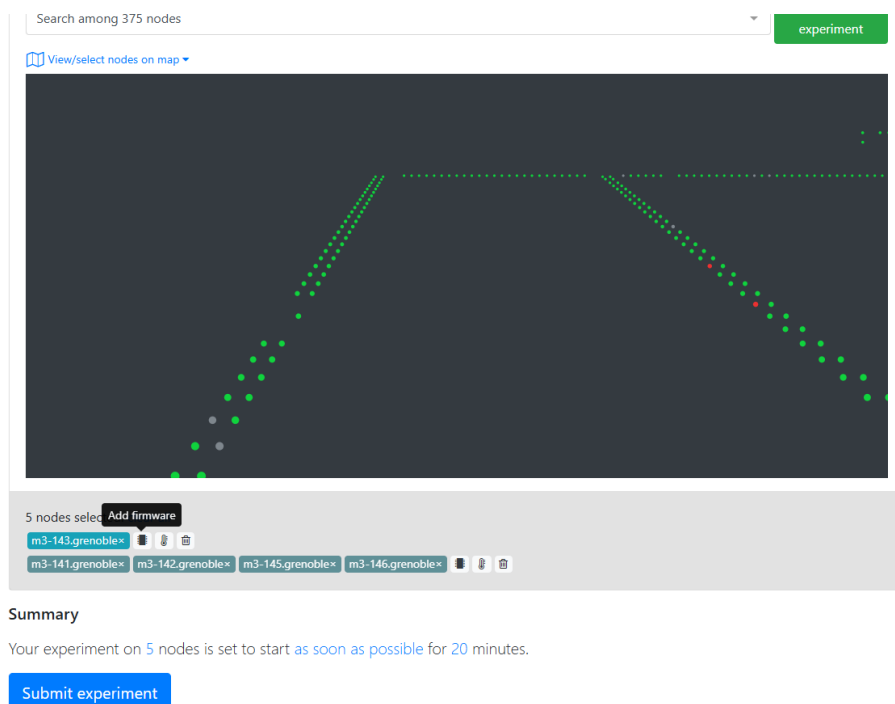
```
make TARGET=iotlab-m3
```



Khi compile thành công thì nó sẽ ra kết quả như hình trên lúc này bạn chỉ cần lấy file có đuôi .iotlab-m3

5. Nạp firmware lên các node

Đầu tiên ta phải chọn node trước sau đó để load firmware thì bạn load bằng cách nhấn vào icon hình IC để add firmware. Khi add thì chọn file .iotlab-m3 vừa compile lúc này



Sau khi load xong thì nhấn Submit experiment rồi đợi đến khi cột State chuyển sang Running rồi bấm vào để xem kết quả.

0 Running 1 Scheduled 57 Completed

Scheduled

Id	Name	Date	Duration	Nodes	State
436985		2026-04-26 13:35	20 minutes (0%)	5	Running

New experiment

Recent

Id	Name	Date	Duration	Nodes	State
436287		2026-04-15 12:47	20 minutes	7	Terminated
436286		2026-04-15 12:37	9 minutes	7	Stopped
436285		2026-04-15 12:04	35 minutes	7	Stopped
436284		2026-04-15 12:03	32 minutes	7	Stopped
436283		2026-04-15 11:27	35 minutes	7	Terminated

Load more...

6. Mở terminal và ghi nhận log

Đến bước này thì ta chỉ cần nhấn nút con icon terminal lên thì cửa sổ terminal sẽ hiện ra

```
m3-143.grenoble.iot-lab.info 9779 udp-server.iotlab-m3 Success
Sending response.
Received request 'hello 26' from aaaa::8579
Sending response.
Received request 'hello 26' from aaaa::a783
Sending response.
Received request 'hello 26' from aaaa::a668
Sending response.
Received request 'hello 26' from aaaa::9680
Sending response.
Received request 'hello 27' from aaaa::8579
Sending response.
Received request 'hello 27' from aaaa::a783
Sending response.
Received request 'hello 27' from aaaa::a668
Sending response.
Received request 'hello 27' from aaaa::9680
Sending response.
Received request 'hello 28' from aaaa::8579
Sending response.

m3-145.grenoble.iot-lab.info 9680 udp-client.iotlab-m3 Success
Sending request 19 to aaaa::1
Received response 'hello 19' from aaaa::9779
Tx/Rx/MissedTx: 20/20/0
Sending request 20 to aaaa::1
Received response 'hello 20' from aaaa::9779
Sending request 21 to aaaa::1
Received response 'hello 21' from aaaa::9779
Sending request 22 to aaaa::1
Received response 'hello 22' from aaaa::9779
Sending request 23 to aaaa::1
Received response 'hello 23' from aaaa::9779
Sending request 24 to aaaa::1
Received response 'hello 24' from aaaa::9779
Sending request 25 to aaaa::1
Received response 'hello 25' from aaaa::9779
Sending request 26 to aaaa::1
Received response 'hello 26' from aaaa::9779
Sending request 27 to aaaa::1
Received response 'hello 27' from aaaa::9779
```

Do cấu hình của mình là Unicast các client sẽ gửi request đến server và server sẽ response đủ 100 cái (do code của mình để là gửi đủ 100 gói). Thế nên để ghi nhận kết quả thì chỉ cần ghi thông số đã gửi bao nhiêu cái response đã được gửi từ server và bao nhiêu cái received response đã được nhận ở node client để phân tích xem có bị loss không. Multicast thì tương tự vậy. (bạn cũng có thể code sao cho dễ nhìn để đọc gói trên terminal hơn)

7. Cách đo khoảng cách giữa các node

Để xem khoảng cách giữa các node thì bạn phải cài tool của FIT/IoT/LAB

Để cài tool thì gõ các dòng lệnh sau:

```
sudo apt update
```

Nhập mật khẩu của hệ điều hành rồi enter, sau đó nhập tiếp

```
sudo apt install -y pipx
pipx ensurepath
```

Xong rồi thì tắt terminal và mở lại rồi gõ:

```
pipx install iotlabcli
```

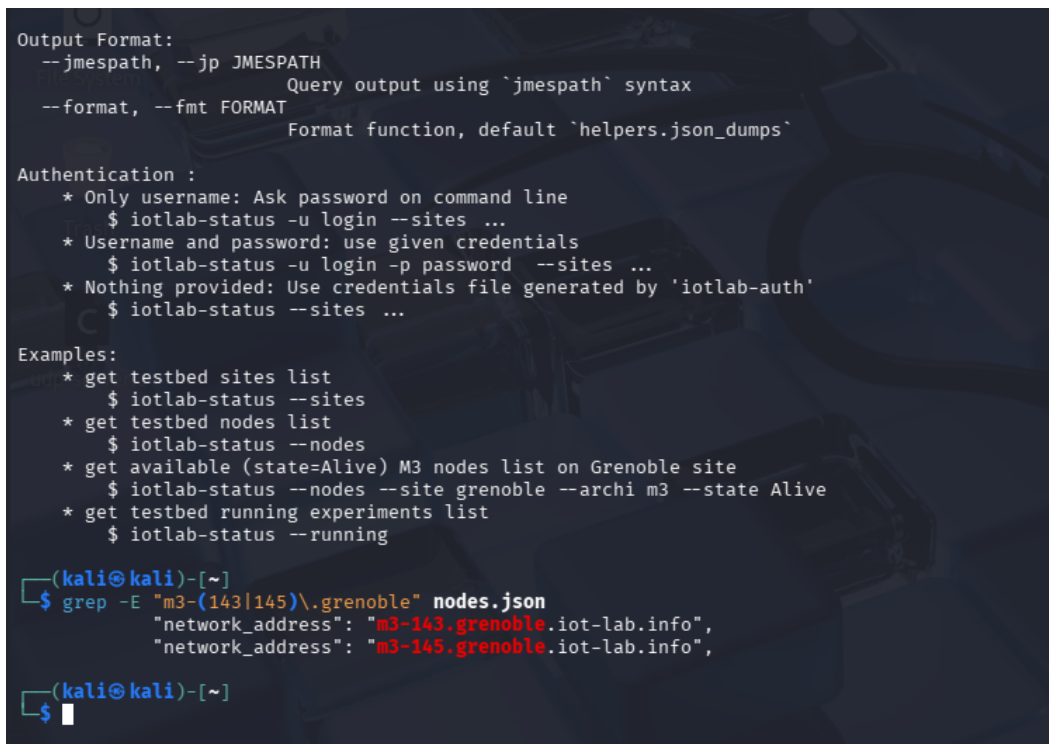
Để kiểm tra cài đặt đã thành công chưa thì gõ:

```
which iotlab-status
iotlab-status --help
```

Cuối cùng là mình sẽ xác định tọa độ node ví dụ node mình đang chọn là m3-143 và m3-145 thì mình làm như sau:

Để kiểm tra xem là trong máy hiện tại có node mình cần tìm tọa độ thì gõ lệnh:

```
grep -E "m3-(143|145)\.grenoble" nodes.json
```



```
Output Format:
--jmespath, --jp JMESPATH          Query output using `jmespath` syntax
--format, --fmt FORMAT             Format function, default `helpers.json_dumps`

Authentication :
* Only username: Ask password on command line
  $ iotlab-status -u login --sites ...
* Username and password: use given credentials
  $ iotlab-status -u login -p password --sites ...
* Nothing provided: Use credentials file generated by 'iotlab-auth'
  $ iotlab-status --sites ...

Examples:
* get testbed sites list
  $ iotlab-status --sites
* get testbed nodes list
  $ iotlab-status --nodes
* get available (state=Alive) M3 nodes list on Grenoble site
  $ iotlab-status --nodes --site grenoble --archi m3 --state Alive
* get testbed running experiments list
  $ iotlab-status --running

(kali@kali)-[~]
$ grep -E "m3-(143|145)\.grenoble" nodes.json
  "network_address": "m3-143.grenoble.iot-lab.info",
  "network_address": "m3-145.grenoble.iot-lab.info",

(kali@kali)-[~]
$
```

Nếu có thì nó sẽ hiển thị như hình

Còn cách để xem tọa độ thì mình gõ lệnh sau:

```
grep -A 40 "m3-143.grenoble" nodes.json
grep -A 40 "m3-145.grenoble" nodes.json
```

```
(kali@kali)-[~]
$ grep -A 40 "m3-143.grenoble" nodes.json
    "network_address": "m3-143.grenoble.iot-lab.info",
    "power_consumption": 1,
    "power_control": 1,
    "radio_sniffing": 1,
    "site": "grenoble",
    "state": "Alive",
    "uid": "9779",
    "x": "0.40",
    "y": "12.03",
    "z": "-0.04"
  },
  {
    "archi": "m3:at86rf231",
    "camera": 0,
    "mobile": 0,
    "mobility_type": " ",
    "network_address": "m3-144.grenoble.iot-lab.info",
    "power_consumption": 1,
    "power_control": 1,
    "radio_sniffing": 1,
    "site": "grenoble",
    "state": "Alive",
    "uid": "c069",
    "x": "1.00",
    "y": "12.03",
    "z": "-0.04"
```

Dòng x,y,z đầu tiên chính là tọa độ của m3-143 thực hiện tương tự với m3-145. Các node m3-143 và m3-145 có tọa độ lần lượt là (0.40, 12.03, -0.04) và (0.40, 11.43, -0.04). Khoảng cách giữa hai node được tính theo công thức khoảng cách Euclid trong không gian ba chiều:

$$d = \sqrt{(x_2 - x_1)^2 + (y_2 - y_1)^2 + (z_2 - z_1)^2}$$

Thay số vào, ta được:

$$d = \sqrt{(0.40 - 0.40)^2 + (11.43 - 12.03)^2 + (-0.04 - (-0.04))^2}$$

$$d = \sqrt{0^2 + (-0.60)^2 + 0^2}$$

$$d = \sqrt{0.36} = 0.60 \text{ m}$$